

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Stratify: Plataforma
Full Stack para el
diseño de estrategias
de trading
algorítmico en
criptoactivos

Curso 2024/2025

Alumno/a:

Pablo Gómez Rivas

Director/es:

Manel Mena Vicente

Luis Fernando Iribarne Martínez



Agradecimientos

Quisiera dedicar unas palabras de reconocimiento a todas aquellas personas que, de una forma u otra, han contribuido a que este trabajo y esta etapa lleguen a buen término.

En primer lugar, agradezco profundamente a mis seres queridos por estar siempre presentes, por ofrecerme su ánimo cuando más lo he necesitado y por enseñarme el valor de la constancia y la confianza. Su presencia ha sido un pilar fundamental en este camino.

A quienes han compartido conmigo esta experiencia académica, gracias por el compañerismo, por las conversaciones que han aliviado el estrés y por todos los recuerdos que me llevo. Sin duda, todo ha sido más llevadero gracias a vosotros.

No quiero dejar de mostrar mi gratitud a la persona que ha dirigido este trabajo, por su disponibilidad, su compromiso y la claridad de sus orientaciones, que han sido clave en este proceso. Y también al resto del profesorado, cuyo conocimiento y dedicación han dejado huella en mi formación.

Gracias a quienes habéis estado ahí durante estos años.



Índice de Contenidos

	Página
1. Introducción	8
1.1. Objetivos	8
1.2. Motivación	8
1.3. Planificación temporal.....	9
1.4. Estado del arte	11
2. Metodologías y herramientas	12
2.1. Metodología ágil.....	12
2.2. Elección tecnológica.....	13
2.2.1. Backend: Python, Django y Django REST Framework	13
2.2.2. Frontend web: React y Vite	14
2.2.3. Base de datos	14
2.2.4. Despliegue en producción: Microsoft Azure y Nginx	15
2.2.5. Librería externa para integración con exchanges	15
2.3. Herramientas	16
3. Análisis funcional	17
3.1. Estudio de viabilidad técnica.....	17
3.2. Análisis de requisitos	17
3.2.1. Objetivos	18
3.2.2. Actores	19
3.2.3. Requisitos funcionales.....	19
3.2.4. Casos de uso	24
3.2.5. Requisitos de información	27
3.2.6. Requisitos no funcionales.....	31
3.3. Prototipado inicial. Mockups	32
4. Fundamentos del análisis técnico	34
4.1. Funcionamiento y estructura de las velas japonesas	34
4.2. Funcionamiento y clasificación de indicadores técnicos.....	35
5. Desarrollo del servidor (backend)	37
5.1. Arquitectura del backend con Django REST Framework	37
5.1.1. Definición de modelos de datos	39
5.1.2. Implementación de vistas	40
5.1.3. Configuración de rutas	42
5.1.4. Serialización y validación de datos	42
5.2. Seguridad y autenticación	43
5.2.1. Autenticación mediante JWT	43
5.2.2. Persistencia de tokens mediante cookies	43
5.2.3. Inicio de sesión con Google y GitHub	44
5.2.4. Verificación de correo electrónico mediante código y protocolo SMTP	45
5.3. API REST: Endpoints	46
5.4. Integración con librerías y servicios externos	48
5.4.1. Integración con plataformas de intercambio mediante CCXT	48
5.4.2. Cálculo de indicadores técnicos a través de TA-Lib	49
5.4.3. Algoritmo de procesamiento de velas para simulación y trading automático	51
6. Desarrollo del cliente (frontend)	52
6.1. Arquitectura general del frontend.....	52
6.1.1. Node.js como entorno base para React.....	52
6.1.2. Configuración del entorno: Vite y TypeScript.....	53
6.1.3. Estructura de componentes con Shadcn/ui y Tailwind CSS	54
6.1.4. Integración de gráfico de velas japonesas con Lightweight Charts.....	55
6.2. Gestión de rutas y navegación en React	56
6.3. Autenticación y manejo de cookies en el navegador	56
6.4. Integración con la API REST del backend	57
6.5. Interfaz de usuario	58
7. Pruebas	69



7.1. Introducción a Postman	69
7.2. Creación de pruebas en Postman	69
7.3. Listado de pruebas en Postman	71
7.4. Pruebas de usabilidad	74
8. Proceso de despliegue	76
8.1. Despliegue con Docker y Nginx	76
8.2. Configuración de Microsoft Azure	77
8.3. Estimación de costes	78
9. Conclusiones	79
9.1. Resultados	79
9.2. Trabajos futuros	79
Anexo 1	81
Anexo 2	82
Anexo 3	83
10. Bibliografía	84



Índice de Figuras

Página

Figura 1: Línea temporal de la planificación del proyecto	9
Figura 2: UC-1, Usuario no Autenticado.....	24
Figura 3: UC-2, Usuario Autenticado, Configurar Perfil	24
Figura 4: UC-3, Usuario Autenticado, Consultar panel global	24
Figura 5: UC-4, Usuario Autenticado, Gestionar claves API	25
Figura 6: UC-5, Usuario Autenticado, Explorar estrategias	25
Figura 7: UC-6, Usuario Autenticado, Configurar estrategia.....	25
Figura 8: UC-7, Usuario Autenticado, Explorar gráfico de velas	26
Figura 9: UC-8, Usuario Autenticado, Administrar ejecución	26
Figura 10: Mockups iniciales	33
Figura 11: Estructura de una vela japonesa alcista y bajista	34
Figura 12: Boceto explicativo de los indicadores RSI y Bollinger Bands sobre el gráfico de velas.....	35
Figura 13: Estructura Modelo-Vista-Plantilla (MVT).....	38
Figura 14: Estructura de carpetas en un proyecto Django.....	38
Figura 15: Tabla User del modelo de datos	39
Figura 16: Clase User en models.py.....	40
Figura 17: Endpoint de listar ejecuciones en views.py	41
Figura 18: Interfaz generada por Django al realizar la llamada al endpoint	41
Figura 19: Configuración general de prefijos en backend/urls.py.....	42
Figura 20: Url asociada al endpoint de listar ejecuciones	42
Figura 21: Serializador para la validación de nombre de usuario y correo	43
Figura 22: Flujo de autenticación con Google y GitHub.....	44
Figura 23: Correo de verificación recibido	45
Figura 24: Entrada en la interfaz para verificación del código.....	45
Figura 25: Endpoints definidos en la API.....	47
Figura 26: Declaración de la clase Exchange.....	49
Figura 27: Ejemplo de cálculo del indicador RSI con TA-Lib	50
Figura 28: Sección de configuración y arranque del package.json	53
Figura 29: Configuración de Vite con Tailwind y alias de rutas en vite.config.ts.....	54
Figura 30: Gestión de rutas con react-router-dom en App.tsx.....	56
Figura 31: Instancia de Axios con cookies incluidas en cada petición.....	57
Figura 32: Gestión global de errores y llamada a la API.....	57
Figura 33: Página principal para usuario no autenticado	58
Figura 34: Interfaz en modo oscuro	58
Figura 35: Formulario de registro	59
Figura 36: Formulario de inicio de sesión.....	59
Figura 37: Recuperación de contraseña para usuario no autenticado.....	60
Figura 38: Recuperación de contraseña con correo prellenado	60
Figura 39: Dashboard con métricas en modo simulado	61
Figura 40: Dashboard con métricas en modo real	61
Figura 41: Listado de estrategias públicas.....	62
Figura 42: Exploración con filtros aplicados.....	62
Figura 43: Listado de claves API por exchange	63
Figura 44: Formulario de gestión de clave API.....	63
Figura 45: Configuración de datos de usuario.....	64
Figura 46: Selección de zona horaria	64
Figura 47: Vista editable de estrategia propia	65
Figura 48: Vista restringida de estrategia ajena.....	65
Figura 49: Parámetros generales de estrategia y vista de gráfico de velas japonesas	66
Figura 50: Modificación de parámetros de un indicador.....	66
Figura 51: Configuración de condiciones antes de ejecutar	67
Figura 52: Visualización de condiciones tras ejecución	67
Figura 53: Métricas de rendimiento de una ejecución.....	68
Figura 54: Lista de operaciones ejecutadas	68



Figura 55: Ejemplo de prueba para el endpoint de consulta de estrategias personales	70
Figura 56: Resultado de ejecución de pruebas en Postman.....	73
Figura 57: Página principal del sistema desplegado en Azure	77



Índice de Tablas

Página

Tabla 1: Planificación de las tareas	10
Tabla 2: OBJ-001, Crear un entorno visual para diseñar estrategias de trading algorítmico	18
Tabla 3: OBJ-002, Permitir explorar, gestionar y clonar estrategias	18
Tabla 4: OBJ-003, Ejecutar operaciones mediante integración con exchanges	18
Tabla 5: OBJ-004, Mostrar resultados y métricas de ejecución de estrategias	18
Tabla 6: OBJ-005, Centralizar funciones clave en un panel de control operativo	18
Tabla 7: ACT-001, Usuario no autenticado	19
Tabla 8: ACT-002, Usuario autenticado	19
Tabla 9: RF-001, Gestionar usuarios y autenticación	19
Tabla 10: RF-002, Configurar perfil de usuario	20
Tabla 11: RF-003, Visualizar panel de control con métricas globales	20
Tabla 12: RF-004, Gestionar claves API de plataformas de intercambio	20
Tabla 13: RF-005, Explorar y filtrar estrategias	21
Tabla 14: RF-006, Administrar y personalizar estrategia	21
Tabla 15: RF-007, Clonar estrategia pública o propia	21
Tabla 16: RF-008, Visualizar gráfico de velas japonesas	22
Tabla 17: RF-009, Gestionar indicadores técnicos en el gráfico	22
Tabla 18: RF-010, Gestionar condiciones para órdenes	22
Tabla 19: RF-011, Administrar ejecuciones de estrategias	23
Tabla 20: RF-012, Gestionar historial de ejecuciones	23
Tabla 21: RI-001, Usuario	27
Tabla 22: RI-002, Claves Api	27
Tabla 23: RI-003, Estrategia	28
Tabla 24: RI-004, Ejecución Estrategia	29
Tabla 25: RI-005, Operación	30
Tabla 26: RI-006, Vela	31
Tabla 27: RNF-001, Robustez ante fallos	31
Tabla 28: RNF-002, Adaptabilidad de la interfaz	31
Tabla 29: RNF-003, Manejo seguro de credenciales	31
Tabla 30: RNF-004, Usabilidad para perfiles no técnicos	32
Tabla 31: RNF-005, Rendimiento y tiempos de respuesta adecuados	32
Tabla 32: RNF-006, Escalabilidad técnica	32
Tabla 33: HU-001, Página principal y tema oscuro	58
Tabla 34: HU-002, Identificación en la plataforma	59
Tabla 35: HU-003, Recuperación de contraseña	60
Tabla 36: HU-004, Portal de métricas globales	61
Tabla 37: HU-005, Exploración de estrategias	62
Tabla 38: HU-006, Gestión de claves API	63
Tabla 39: HU-007, Configuración de usuario	64
Tabla 40: HU-008, Visualización de estrategia	65
Tabla 41: HU-009, Configuración general e indicadores	66
Tabla 42: HU-010, Configuración y visualización de condiciones de ejecución	67
Tabla 43: HU-011, Resultados y operaciones de una ejecución	68
Tabla 44: Pruebas de Autenticación de Usuario	71
Tabla 45: Pruebas de Preferencias del Usuario	71
Tabla 46: Pruebas de API Keys	71
Tabla 47: Pruebas de Datos de Mercado	72
Tabla 48: Pruebas de Estrategias	72
Tabla 49: Pruebas de Ejecución de Estrategias	73
Tabla 50: Pruebas de usabilidad realizadas	74
Tabla 51: Participantes de las pruebas de usabilidad	75
Tabla 52: Estimación costes mensuales en producción	78



1. Introducción

El presente documento tiene como objetivo recoger de manera ordenada y coherente el proceso de desarrollo de una plataforma web integral destinada a la gestión de estrategias de trading algorítmico (compra y venta de activos con fines especulativos de manera automática) aplicadas al mercado de criptomonedas. A lo largo de los distintos apartados se detallarán los fines perseguidos por el proyecto, las razones técnicas y personales que lo motivan, así como la planificación llevada a cabo para su ejecución. También se incluye un estudio previo del contexto tecnológico y del panorama actual en torno al uso de algoritmos de inversión, con el propósito de situar esta propuesta dentro de una perspectiva actualizada. Este apartado introductorio proporciona el marco necesario para entender el enfoque adoptado, el valor diferencial del trabajo y los fundamentos sobre los que se ha construido la solución planteada.

1.1. Objetivos

El presente Trabajo de Fin de Grado tiene como propósito el diseño y desarrollo de Stratify, una plataforma web Full Stack orientada a la gestión de estrategias de trading algorítmico para criptomonedas. Este proyecto busca facilitar el acceso y la utilización de algoritmos de inversión basados en indicadores técnicos, democratizando así una herramienta tradicionalmente reservada a expertos y grandes inversores. De esta forma, Stratify pretende transformar la volatilidad característica del mercado de criptomonedas, convirtiéndola en una oportunidad accesible y controlada para usuarios con diferentes niveles de experiencia.

La elección de una plataforma web responde a la necesidad de ofrecer una solución accesible, flexible y compatible con múltiples dispositivos, evitando la dependencia de software específico o instalaciones complicadas. Dado que la gestión y análisis en tiempo real son elementos clave en el trading algorítmico, esta aplicación web permite a los usuarios consultar, modificar y ejecutar sus estrategias desde cualquier lugar con conexión a internet, fomentando la autonomía y la participación en un mercado dinámico y complejo.

Este proyecto también representa un desafío técnico significativo, ya que integra tecnologías actuales tanto en el backend (lado del servidor, que brinda los datos), encargado de procesar y almacenar los datos relacionados con las estrategias y el mercado, como en el frontend (lado del cliente, es decir, la aplicación utilizada por los usuarios), que proporciona una interfaz intuitiva y funcional para el usuario final. Entre las funcionalidades desarrolladas se incluyen la configuración de estrategias personalizables, la visualización de indicadores técnicos y la monitorización continua de resultados, aportando un enfoque práctico y formativo al trabajo.

Los objetivos principales que guían el desarrollo del proyecto son los siguientes:

- Crear una plataforma web que permita a los usuarios diseñar, probar y gestionar estrategias de trading algorítmico basadas en indicadores técnicos para criptomonedas.
- Implementar un sistema seguro de autenticación y gestión de usuarios, que facilite la personalización y el control sobre las estrategias desarrolladas.
- Desarrollar una API RESTful [1] robusta y escalable que soporte la interacción con APIs externas de datos financieros y asegure la correcta ejecución de las operaciones de trading.
- Adquirir experiencia en tecnologías Full Stack, incluyendo el desarrollo de interfaces reactivas y la gestión eficiente de datos, con el fin de consolidar habilidades técnicas y aplicar conocimientos teóricos en un contexto práctico.

1.2. Motivación

En la última década, el mercado de criptomonedas ha experimentado un crecimiento exponencial, caracterizado por una alta volatilidad y un volumen creciente de transacciones diarias. En este contexto, el trading algorítmico ha emergido como una herramienta fundamental, representando aproximadamente el 60% del volumen total de operaciones en los mercados financieros globales, siendo incluso mayor esta proporción en las criptomonedas. Sin embargo, esta tecnología, que automatiza y optimiza las decisiones de inversión

mediante algoritmos basados en indicadores técnicos, sigue siendo mayormente inaccesible para el inversor promedio debido a la complejidad técnica y la falta de plataformas intuitivas y accesibles.

El elevado dinamismo e imprevisibilidad inherentes a las criptomonedas presentan una oportunidad y un desafío: mientras la volatilidad puede implicar riesgos significativos, también abre la puerta a posibilidades de beneficio si se gestionan adecuadamente. No obstante, la mayoría de los inversores minoristas carecen de las herramientas necesarias para aprovechar esta volatilidad de forma estratégica y segura, limitando su capacidad para competir en igualdad de condiciones con grandes fondos e inversores institucionales.

Este proyecto nace de la necesidad de democratizar el acceso al trading algorítmico, transformando la volatilidad de las criptomonedas de una amenaza en una oportunidad para el inversor común. Stratify se plantea como una plataforma web que permite diseñar, gestionar y ejecutar estrategias algorítmicas basadas en indicadores técnicos, con un enfoque intuitivo y accesible para usuarios con diferentes niveles de experiencia. La idea es que cualquier persona pueda acceder a herramientas avanzadas de inversión automatizada sin requerir conocimientos profundos en programación o finanzas cuantitativas.

Más allá del desarrollo tecnológico, este proyecto responde a una visión social: empoderar a los usuarios para que tomen decisiones informadas y optimicen sus inversiones en un mercado tan disruptivo como el de las criptomonedas. La plataforma no solo busca ofrecer funcionalidad, sino también promover una cultura de transparencia, control y aprendizaje continuo, facilitando que los usuarios comprendan y adapten sus estrategias a la evolución del mercado.

A nivel personal y académico, este trabajo ha supuesto un desafío integral que ha permitido consolidar competencias en desarrollo web, análisis de datos financieros y diseño de algoritmos, además de abordar aspectos fundamentales como la experiencia de usuario y la seguridad en entornos digitales. En definitiva, Stratify nace de la convicción de que la tecnología puede y debe ser un vehículo para ampliar las oportunidades financieras, haciendo que el acceso a la inversión automatizada sea una realidad para todos.

1.3. Planificación temporal

El proyecto se ha desarrollado a lo largo de un periodo estimado de tres meses y medio, en el que se han ido alcanzando distintos hitos relevantes. La Figura 1 recoge una representación visual de esta línea temporal, destacando los momentos clave que marcaron el progreso del trabajo.

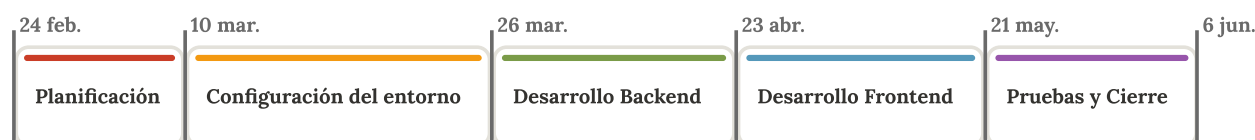


Figura 1: Línea temporal de la planificación del proyecto

Para complementar la información ofrecida en dicha figura, en la Tabla 1 se detalla la planificación inicial a través de una subdivisión de tareas más concreta. Esta descomposición facilita una mejor comprensión del proceso de trabajo y de los distintos componentes abordados en cada fase. Asimismo, en el Anexo 1 se incluye una versión gráfica de esta planificación en forma de diagrama de Gantt, que permite visualizar de forma orientativa las dependencias entre tareas.

Desde el inicio se planteó una distribución de carga horaria basada en una dedicación diaria de 4 horas, de lunes a viernes, lo que se traduce en unas 20 horas semanales. Bajo este esquema de trabajo, el proyecto se estructuró en 15 semanas, iniciándose el lunes 24 de febrero de 2025 y finalizando el viernes 6 de junio de 2025, sin contemplar los fines de semana. La Tabla 1 muestra de manera clara esta organización temporal.

Nº de tarea	Nombre de tarea	Duración (días)	Inicio	Fin	Predecesoras
1	Planificación	10	24/02/2025 (lun)	07/03/2025 (vie)	-
2	Definición y análisis de requisitos	4	24/02/2025 (lun)	27/02/2025 (jue)	-



3	Análisis y elección tecnológica	4	28/02/2025 (vie)	05/03/2025 (mié)	2
4	Diseño de diagrama de casos de uso	2	06/03/2025 (jue)	07/03/2025 (vie)	3
5	Configuración del entorno	12	10/03/2025 (lun)	25/03/2025 (mar)	1
6	Inicialización del repositorio Git	1	10/03/2025 (lun)	10/03/2025 (lun)	4
7	Instalación y configuración de herramientas de desarrollo	3	11/03/2025 (mar)	13/03/2025 (jue)	6
8	Diseño de la estructura de carpetas y entorno virtual	4	14/03/2025 (vie)	19/03/2025 (mié)	7
9	Despliegue inicial en Azure	4	20/03/2025 (jue)	25/03/2025 (mar)	8
10	Desarrollo del backend	20	26/03/2025 (mié)	22/04/2025 (mar)	5
11	Modelado de la base de datos	4	26/03/2025 (mié)	31/03/2025 (lun)	9
12	Integración de API RESTful	5	01/04/2025 (mar)	07/04/2025 (lun)	11
13	Implementación de endpoints	8	08/04/2025 (mar)	17/04/2025 (jue)	12
14	Pruebas a la API con Postman	3	18/04/2025 (vie)	22/04/2025 (mar)	13
15	Desarrollo del frontend	20	23/04/2025 (mié)	20/05/2025 (mar)	10
16	Diseño y maquetación de interfaces	4	23/04/2025 (mié)	28/04/2025 (lun)	14
17	Configuración de librería de componentes	3	29/04/2025 (mar)	01/05/2025 (jue)	16
18	Integración de gráfico de velas japonesas	5	02/05/2025 (vie)	08/05/2025 (jue)	17
19	Desarrollo de lógica de negocio y conexión con backend	8	09/05/2025 (vie)	20/05/2025 (mar)	18
20	Finalización	13	21/05/2025 (mié)	06/06/2025 (vie)	15
21	Pruebas finales del proyecto	3	21/05/2025 (mié)	23/05/2025 (vie)	19
22	Redacción de memoria del TFG	10	26/05/2025 (lun)	06/06/2025 (vie)	21

Tabla 1: Planificación de las tareas

No obstante, como ocurre frecuentemente en proyectos de este tipo, hubo desviaciones respecto al plan original. Algunas tareas iniciales, relativas a la planificación general, la preparación técnica y el desarrollo del backend, se completaron antes de lo previsto, en parte gracias a la experiencia previa con las



herramientas seleccionadas. Por otro lado, la fase de desarrollo del frontend requirió más tiempo del estimado inicialmente, debido a ciertas dificultades técnicas que se explicarán más adelante. A pesar de estos ajustes, el proyecto logró mantenerse dentro del plazo global establecido.

1.4. Estado del arte

En la actualidad, existen diversas plataformas orientadas al trading algorítmico en criptomonedas, aunque presentan limitaciones significativas que dificultan su uso generalizado. Entre las más conocidas, Trading-View destaca como una herramienta potente para análisis técnico, pero su personalización de estrategias requiere programación en Pine Script, lo que limita su accesibilidad para usuarios sin conocimientos técnicos. Además, su interfaz suele estar sobrecargada con funciones adicionales que distraen del foco principal: el desarrollo preciso de estrategias de trading.

Asimismo, la mayoría de los servicios similares o plataformas especializadas suelen ser de pago y requieren conocimientos técnicos avanzados para diseñar y ajustar algoritmos, lo que excluye a una gran parte de inversores minoristas. Muchas opciones disponibles se enfocan exclusivamente en la ejecución automática o en la visualización de indicadores, sin ofrecer una plataforma integrada para la creación interactiva y la gestión personalizada de estrategias.

Otra barrera importante en muchas soluciones existentes es el proceso de conexión con exchanges (plataformas de intercambio) de criptomonedas. Si bien algunas plataformas permiten la automatización del trading mediante claves API, en la mayoría de los casos esta funcionalidad se encuentra detrás de planes de suscripción de pago. Esto implica que el usuario no solo debe diseñar su estrategia, sino también asumir un coste adicional para poder llevarla a cabo de forma automatizada, lo cual contradice la idea de accesibilidad que debería caracterizar a este tipo de herramientas.

Además, el enfoque de estas plataformas suele ser poco visual o intuitivo. Algunas muestran gráficas y permiten observar el comportamiento histórico de indicadores, pero muy pocas ofrecen un entorno en el que el usuario pueda definir reglas lógicas condicionales de forma interactiva, sin código, y con retroalimentación visual clara. Esta falta de usabilidad limita el potencial de aprendizaje y experimentación de muchos usuarios interesados en iniciarse en el trading algorítmico sin experiencia previa en programación o en herramientas técnicas complejas.

Frente a este panorama, Stratify propone una solución diferenciada con las siguientes ventajas:

- Permite la personalización de estrategias mediante la configuración interactiva de condiciones booleanas, sin necesidad de escribir código.
- Ofrece un entorno sencillo y enfocado, sin la saturación de funciones que habitualmente caracteriza a otras plataformas.
- Democratiza el acceso al trading algorítmico, evitando barreras técnicas y económicas.
- Integra en un solo espacio todas las actividades relacionadas con la creación, gestión y evaluación de estrategias basadas en indicadores técnicos, facilitando su uso.
- Fomenta la autonomía del inversor minorista, facilitando una experiencia intuitiva y transparente.
- Permite conectar la cuenta de múltiples plataformas de intercambio de forma gratuita, manteniendo la operativa accesible incluso en fases iniciales.
- Prioriza la experiencia de usuario, con una interfaz clara, limpia y orientada a la acción.

A diferencia de otras plataformas fragmentadas, de pago o con barreras técnicas significativas, *Stratify* incorpora todas estas características en una interfaz amigable y accesible, centrada en empoderar al usuario para que convierta la volatilidad de las criptomonedas en oportunidades reales de inversión. Su enfoque unificado, orientado a la simplicidad y a la acción directa, permite superar las limitaciones actuales del sector y ofrece una alternativa funcional, educativa y abierta a un espectro mucho más amplio de usuarios.

2. Metodologías y herramientas

Antes de iniciar el desarrollo de un proyecto software, es imprescindible definir con claridad el enfoque de trabajo y las herramientas técnicas que se emplearán durante el proceso. En esta sección se analizan las decisiones metodológicas y tecnológicas tomadas, explicando el porqué de cada elección. Se destaca la adopción de una metodología ágil, considerada la opción más adecuada para este tipo de desarrollo, así como la selección de tecnologías utilizadas, justificando su elección frente a otras alternativas ampliamente reconocidas en el ámbito del desarrollo web.

2.1. Metodología ágil

Para desarrollar un proyecto de software con garantías de organización y eficiencia, resulta imprescindible establecer desde el inicio una estrategia de trabajo que permita avanzar de forma estructurada y adaptable. Frente a esquemas clásicos como el modelo en cascada (que propone una secuencia cerrada de fases sin posibilidad de retroceso), los métodos ágiles ofrecen un enfoque más flexible, donde el progreso se construye de forma progresiva mediante iteraciones breves y evaluables.

Estos enfoques han ganado popularidad en distintos contextos gracias a su capacidad para facilitar el seguimiento continuo del desarrollo, detectar errores tempranos y adaptarse con rapidez a nuevas circunstancias. Entre los marcos más representativos se encuentran Scrum [2], Kanban [3] y XP (Extreme Programming) [4], todos ellos centrados en maximizar la entrega de valor real, fomentar la colaboración constante y mantener una visión clara del producto en cada etapa. Aunque suelen aplicarse en entornos colaborativos, su lógica también puede adaptarse eficazmente a proyectos individuales con las adaptaciones necesarias.

Dado que este trabajo ha sido desarrollado por una sola persona en un entorno académico, se optó por una versión simplificada de Scrum, orientada a la planificación por ciclos breves, llamados sprints, en los que se establecían metas concretas y tareas bien definidas. Cada uno de estos ciclos tiene una duración de entre una y dos semanas, y concluye con una revisión para analizar los resultados obtenidos, ajustar prioridades si era necesario y planificar el siguiente bloque de trabajo en coordinación con el tutor del proyecto.

La elección de este enfoque respondió a la necesidad de mantener una estructura clara sin sacrificar flexibilidad. Permitir cambios sobre la marcha, reorganizar tareas cuando era necesario y mantener control constante sobre el ritmo de desarrollo fueron factores clave para el éxito del proyecto. Además, esta dinámica permitió mantener buena comunicación con el tutor, establecer objetivos realistas y avanzar de forma ordenada, reduciendo el riesgo de acumulación de errores o desvíos significativos en los tiempos.

Si bien algunos componentes tradicionales del marco Scrum, como la definición de roles específicos o las reuniones diarias, no eran aplicables en un proyecto unipersonal, otros elementos resultaron muy útiles. La división del trabajo en bloques manejables, la definición anticipada de entregables por ciclo y las evaluaciones periódicas facilitaron tanto la organización como el seguimiento del progreso. Esta estructura también sirvió como base para mantener la motivación a lo largo del proyecto, permitiendo ver resultados concretos en plazos relativamente cortos.

Los sprints fueron definidos en relación directa con la planificación general del proyecto, que se recoge en la Tabla 1. Se intentó que cada ciclo abarcara uno o varios hitos importantes. Por ejemplo, tareas iniciales como la preparación del entorno o la selección de tecnologías se completaron en ciclos breves, mientras que etapas más amplias, como el desarrollo del backend o la interfaz del usuario, se fragmentaron en varios sprints consecutivos, cada uno enfocado en objetivos técnicos específicos (como el diseño de la base de datos, la creación de la API o la implementación de componentes visuales).

La organización del trabajo de esta manera contribuyó no solo a mantener la trazabilidad de lo realizado, sino también a documentar el proceso con mayor precisión. Al finalizar cada ciclo, se realizaba una sesión con el tutor del proyecto que funcionaba como una revisión externa del trabajo desarrollado. Estas sesiones resultaron valiosas tanto para obtener retroalimentación como para corregir el rumbo en caso necesario y asegurar que el proyecto se mantenía dentro de los objetivos previstos en cada fase.

2.2. Elección tecnológica

Para la realización de este proyecto, se llevó a cabo una selección meticulosa de las tecnologías que mejor se adaptaran a los objetivos planteados y a las características propias del sistema que se pretendía desarrollar. La intención principal fue construir una solución robusta y eficiente, empleando un conjunto limitado pero coherente de herramientas que pudieran integrarse de manera armoniosa, garantizando así un equilibrio adecuado entre rendimiento, facilidad de mantenimiento y escalabilidad futura.

En este apartado se presentan las tecnologías principales implementadas, organizadas según su papel en la arquitectura general del sistema. Se incluyen las relacionadas con el procesamiento y la lógica del servidor, la gestión y visualización de la interfaz de usuario, la administración y almacenamiento de datos, y las herramientas complementarias que facilitan el desarrollo y despliegue del proyecto. Además, se exponen las razones que fundamentaron la elección de estas tecnologías específicas frente a otras opciones disponibles en el mercado. Se consideraron factores como la experiencia previa, la curva de aprendizaje de cada tecnología, la compatibilidad entre componentes y la capacidad para afrontar los retos técnicos y funcionales en cada fase del desarrollo.

La adopción de lenguajes de programación modernos y ampliamente respaldados, como Python [5] y TypeScript [6], permitió la construcción de una plataforma sólida y escalable, con una clara separación entre la lógica del backend y la experiencia del usuario en el frontend. Este enfoque facilitó la aplicación de buenas prácticas de desarrollo, promoviendo la modularidad, la reutilización de código y un control de versiones riguroso. Estas características fueron fundamentales para gestionar de manera efectiva un proyecto con una complejidad y alcance superiores a los habituales en el contexto académico, permitiendo además mantener un ritmo constante de trabajo y asegurar la calidad técnica del producto final durante todas las etapas del proceso de desarrollo.

2.2.1. Backend: Python, Django y Django REST Framework

Para el desarrollo del backend se optó por utilizar Python debido a su naturaleza accesible y su fuerte presencia en ámbitos técnicos diversos. Su sintaxis clara, junto con la extensa documentación disponible y el respaldo de una comunidad consolidada, lo convierten en una herramienta especialmente útil para proyectos donde la eficiencia y la rapidez de implementación son factores relevantes.

En este caso concreto, se decidió trabajar con Django REST Framework [7], una extensión del ecosistema Django [8] especializada en la construcción de APIs. Esta elección se basó tanto en motivos estratégicos como prácticos. Por un lado, ya se contaba con experiencia previa en proyectos desarrollados sobre esta tecnología, lo que permitía avanzar con mayor seguridad y eficiencia. Empezar este tipo de trabajo final con herramientas completamente nuevas podría haber supuesto una inversión excesiva de tiempo en la fase de aprendizaje, con el riesgo añadido de comprometer plazos o la estabilidad del sistema.

Django REST Framework aporta una estructura sólida para construir interfaces de programación robustas, y su integración con Django facilita una organización clara del código, ideal para sistemas con múltiples módulos. Además, proporciona mecanismos integrados para tareas recurrentes como la autenticación, la serialización de datos o la gestión de permisos, lo que permite centrarse en el desarrollo funcional del proyecto sin necesidad de construir soluciones desde cero.

A nivel de seguridad, este conjunto tecnológico ofrece un entorno razonablemente protegido desde el inicio, incorporando medidas automáticas que previenen vulnerabilidades comunes sin requerir configuraciones complejas. Esta característica resulta fundamental cuando se manejan datos sensibles o se requiere una interacción constante con servicios externos.

Otra razón para inclinarse por este entorno es la facilidad con la que se puede integrar con librerías auxiliares del ecosistema Python, lo que ha facilitado el desarrollo de funcionalidades complementarias como la obtención de datos de fuentes externas, la automatización de tareas paralelas o la transformación de datos para su análisis posterior.

En la fase de evaluación de alternativas, se contemplaron otras opciones como FastAPI [9], NestJS o Spring Boot [10]. FastAPI ofrecía una propuesta moderna dentro del ecosistema Python, con un alto rendimiento y una sintaxis muy clara, pero requería una configuración más detallada y carecía de algunas utilidades integradas que Django REST Framework ya proporciona de serie. NestJS, por otro lado, encajaba bien con



un stack basado en React [11] gracias a su uso de TypeScript y su enfoque modular, aunque su complejidad inicial y la necesidad de definir manualmente muchas capas del sistema lo hacían menos eficiente para un desarrollo centrado en resultados rápidos. En cuanto a Spring Boot, su robustez y uso en entornos empresariales son indiscutibles, aunque su curva de aprendizaje elevada y la necesidad de trabajar con Java lo hacían menos apropiado para un desarrollo individual con un enfoque práctico y formativo.

En conjunto, Django REST Framework ha resultado ser una opción equilibrada, capaz de ofrecer rapidez de desarrollo, claridad estructural y funcionalidades avanzadas sin añadir una carga técnica excesiva, ajustándose perfectamente a los objetivos del proyecto.

2.2.2. Frontend web: React y Vite

Para la implementación del frontend se eligió React, un framework de código abierto desarrollado por Meta que facilita la creación de interfaces web dinámicas y reactivas mediante el uso de componentes modulares en JavaScript o TypeScript. Este framework, combinado con Vite como herramienta de construcción y desarrollo, proporciona un entorno moderno y eficiente que mejora significativamente los tiempos de recarga y la experiencia durante el desarrollo.

La elección de React se fundamenta principalmente en su capacidad para construir aplicaciones web escalables y de alto rendimiento. Su arquitectura basada en componentes reutilizables permite organizar el código de forma clara y modular, favoreciendo tanto el mantenimiento como la ampliación futura del proyecto. Además, su manejo eficaz del estado y la renderización optimizada contribuyen a ofrecer una interfaz ágil y fluida, un requisito indispensable para la gestión y visualización dinámica de estrategias de trading.

Otro factor decisivo fue la robusta comunidad que respalda React, junto con la amplia disponibilidad de librerías y herramientas compatibles que facilitan la integración de funcionalidades complejas, como la comunicación con APIs externas y el control del estado global. El uso de Vite complementa este ecosistema al proporcionar un sistema de construcción ligero y rápido, lo que agiliza el ciclo de desarrollo y reduce significativamente los tiempos de espera durante la prueba y depuración del código.

Además, la naturaleza web de React junto con Vite facilita el despliegue inmediato y la compatibilidad con cualquier dispositivo con navegador, sin necesidad de instalaciones adicionales o configuraciones específicas del sistema operativo. Esto resulta especialmente relevante para un proyecto académico, donde la accesibilidad y facilidad de uso para los usuarios finales son prioritarias. Gracias a esta característica, la plataforma puede ser consultada y gestionada desde diferentes entornos (ordenadores, tabletas o móviles) asegurando una experiencia coherente y homogénea. De esta forma, se evita la fragmentación que supone desarrollar aplicaciones nativas separadas, optimizando el tiempo y los recursos disponibles.

Al evaluar alternativas populares como Angular o Vue.js, React destacó por ofrecer un equilibrio óptimo entre flexibilidad, rendimiento y soporte comunitario. Angular, aunque completo, implica mayor complejidad inicial y un enfoque más rígido, mientras que Vue.js, sencillo y ligero, no cuenta con el mismo nivel de madurez y adopción en proyectos a gran escala, factores clave para un trabajo de estas características.

2.2.3. Base de datos

En el desarrollo de esta plataforma, la gestión eficiente y estructurada de los datos resulta fundamental. Por ello, se optó por utilizar PostgreSQL [12], un sistema gestor de bases de datos relacional que destaca por su estabilidad y su soporte avanzado para consultas SQL. Esta decisión se apoya en la experiencia acumulada durante el grado en bases de datos relacionales, lo que permitió aprovechar conocimientos ya adquiridos y evitar la complejidad que implica adoptar tecnologías desconocidas.

Uno de los factores clave para elegir PostgreSQL fue su excelente compatibilidad con Django, especialmente mediante su ORM, que facilita la interacción con la base de datos a través de objetos en Python. Esto permite que la estructura de datos se modifique y evolucione sin escribir sentencias SQL manualmente, gracias al sistema de migraciones automatizadas. Además, la naturaleza relacional de PostgreSQL facilita la representación clara de las relaciones entre entidades del sistema, como usuarios y estrategias de trading, asegurando la integridad y consistencia de la información.

Aunque se exploraron otras tecnologías, tales como MySQL [13], Oracle Database [14] y bases de datos NoSQL como Cassandra [15], ninguna satisfizo de manera óptima los requisitos específicos del proyecto. MySQL, pese a su popularidad, no aportaba mejoras significativas respecto a PostgreSQL para el contexto

planteado. Oracle Database, por su parte, se descartó debido a su complejidad de configuración y al coste asociado a su licencia, factores poco compatibles con los recursos disponibles. En cuanto a Cassandra, su arquitectura orientada a modelos NoSQL no es adecuada para la aplicación, que requiere un manejo preciso y estructurado de datos altamente interrelacionados.

Por último, el respaldo comunitario y la documentación accesible de PostgreSQL han sido vitales para resolver problemas durante el desarrollo. Su capacidad para manejar grandes volúmenes de datos y escalar con el proyecto la convierten en una opción segura para su despliegue en producción, donde se espera un aumento en la demanda y complejidad de los datos.

2.2.4. Despliegue en producción: Microsoft Azure y Nginx

Para el despliegue en producción se ha elegido Microsoft Azure [16], la plataforma de servicios en la nube de Microsoft. Su elección se debe a su buena integración con los servicios necesarios y a la posibilidad de gestionar fácilmente entornos escalables y seguros. Azure ofrece herramientas específicas para aplicaciones web, monitorización, gestión de bases de datos y control de identidad, lo que permite mantener una infraestructura sólida sin una configuración excesivamente compleja.

Una de las principales razones para esta elección ha sido la disponibilidad de una suscripción gratuita anual de 100 € ofrecida por la Universidad de Almería, lo que permite utilizar recursos en la nube sin coste adicional durante el desarrollo y pruebas del proyecto.

Se valoraron otras opciones como Amazon Web Services [17], Google Cloud [18] y Heroku. Sin embargo, la ausencia de apoyo económico institucional en estos casos y una curva de aprendizaje más pronunciada en algunos servicios hicieron que Azure resultase la opción más equilibrada y viable para este proyecto.

Como servidor web se ha utilizado Nginx [19], configurado como proxy inverso para canalizar las peticiones entre el cliente y los distintos servicios de la aplicación. Su capacidad para manejar múltiples conexiones simultáneas de forma eficiente y su bajo consumo de recursos lo convierten en una opción sólida para entornos de producción.

Asimismo, Nginx permite servir contenido estático, redirigir tráfico y gestionar certificados SSL/TLS [20], facilitando la implementación de conexiones seguras. Su configuración clara y su amplia documentación han contribuido a integrar esta herramienta sin añadir complejidad innecesaria al despliegue.

2.2.5. Librería externa para integración con exchanges

Para la interacción con los mercados de criptomonedas en este proyecto, se ha optado por utilizar la librería de código abierto CCXT (CryptoCurrency eXchange Trading Library), una solución consolidada y ampliamente utilizada en el ámbito del trading algorítmico. De los más de 100 exchanges que soporta CCXT, se han seleccionado 42 que cumplen con los requisitos funcionales necesarios para el correcto desarrollo y operación de la plataforma.

Las funcionalidades imprescindibles que estos exchanges deben soportar, y que han sido aprovechadas mediante CCXT, son las siguientes:

- Autenticación segura mediante claves API proporcionadas por los usuarios.
- Acceso a la información sobre los mercados disponibles en cada plataforma, permitiendo la selección adecuada de pares de criptomonedas.
- Obtención de datos de precios, tanto históricos como actuales, en formatos estándar (OHLCV), fundamentales para el análisis y simulación de estrategias.
- Ejecución en tiempo real de órdenes de compra y venta, lo que posibilita la automatización completa y eficiente del trading.

CCXT fue elegido por ser casi la única librería con soporte uniforme para múltiples plataformas de intercambio, facilitando la integración mediante su capa de abstracción. Al ser código abierto, su comunidad mantiene las APIs actualizadas y soluciona problemas. Esto permitió crear una arquitectura flexible y escalable que centraliza la conexión a los exchanges en una sola herramienta, evita código duplicado y facilita añadir mercados o cambios sin modificar la estructura base. Así, CCXT acortó los tiempos de desarrollo y garantizó la fiabilidad necesaria para el trading automatizado.



2.3. Herramientas

- **Visual Studio Code**

Visual Studio Code ha sido la herramienta elegida para la edición y gestión del código a lo largo del desarrollo. Su popularidad radica en su rapidez, soporte para numerosos lenguajes y un extenso catálogo de extensiones que enriquecen la experiencia del desarrollador. Además, su integración directa con sistemas de control de versiones y terminales incorporados permite una gestión eficiente del proyecto sin necesidad de alternar entre aplicaciones. La capacidad de adaptar el entorno a necesidades específicas mediante plugins (extensiones que se añaden a un programa para extender su funcionalidad) ha sido esencial para optimizar la calidad del código y la productividad durante todo el proceso.

- **Docker**

El uso de Docker [21] ha resultado fundamental para crear entornos de desarrollo consistentes y fácilmente replicables, independientemente del sistema operativo utilizado. Esta tecnología de contenedores facilita el despliegue de componentes backend y servicios auxiliares, asegurando que las dependencias y configuraciones sean idénticas en cualquier máquina. Docker simplifica también la escalabilidad y mantenimiento del sistema, permitiendo aislar servicios y evitar conflictos, lo que es especialmente valioso en proyectos con arquitecturas complejas y múltiples dependencias.

- **Postman**

Postman [22] se ha empleado como la herramienta principal para la prueba y validación de la API desarrollada. Su interfaz intuitiva y sus funcionalidades avanzadas, como la gestión de colecciones, variables de entorno y automatización de pruebas, permiten una verificación rápida y organizada de los endpoints (URLs concretas de una API utilizada para enviar o recibir información). Estas características facilitan tanto la detección temprana de errores como la documentación dinámica del comportamiento esperado, apoyando el proceso de desarrollo ágil y garantizando la calidad de la comunicación entre frontend y backend.

- **Git y GitHub**

El sistema de control de versiones Git ha sido fundamental para el control y seguimiento del código fuente, permitiendo mantener un historial detallado y gestionar versiones mediante ramas de forma eficiente. GitHub ha servido como repositorio remoto para almacenar el proyecto de manera segura y facilitar la sincronización entre diferentes dispositivos personales. Este sistema ha sido clave para mantener la integridad del código, permitiendo experimentar con nuevas funcionalidades, resolver conflictos y recuperar versiones anteriores, asegurando así un desarrollo ordenado y controlado.

- **Figma y Draw.io**

En el proceso de diseño y documentación visual del proyecto, se han utilizado Figma [23] y Draw.io, dos herramientas que cubren distintas necesidades. Figma ha permitido crear y validar prototipos de la interfaz de usuario, facilitando la planificación de aspectos gráficos y de usabilidad. Por otro lado, Draw.io ha sido empleado para elaborar diagramas técnicos, como diagrama de casos de uso y modelos de datos, que ayudan a entender la estructura y funcionalidad del sistema. El uso combinado de ambas herramientas ha contribuido a mantener una documentación clara y precisa a lo largo del desarrollo.



3. Análisis funcional

El análisis funcional constituye una fase esencial dentro del ciclo de desarrollo, ya que permite establecer de forma precisa los comportamientos esperados del sistema, las necesidades a cubrir y las interacciones que se producirán entre los diferentes componentes y usuarios. En este apartado se definen los objetivos específicos del sistema, se identifican los requisitos técnicos y operativos, y se delimitan los elementos necesarios para garantizar una experiencia satisfactoria y eficaz en el uso de la plataforma. Además, se incorporan los primeros esquemas de diseño y prototipos visuales elaborados como referencia para guiar las fases iniciales de implementación.

3.1. Estudio de viabilidad técnica

Antes de comenzar el proceso de desarrollo, se realizó una evaluación detallada de la viabilidad técnica del proyecto, con el objetivo de comprobar su factibilidad en términos de tiempo, recursos y complejidad funcional. Esta revisión permitió valorar si el alcance planteado era razonable dentro de los márgenes establecidos por el calendario académico y la carga de trabajo asumible en un contexto individual.

El proyecto presenta una notable complejidad, tanto por el número de funcionalidades integradas como por la variedad de tecnologías involucradas. Entre los retos técnicos se encuentran el desarrollo de una API RESTful, la interacción con múltiples plataformas de intercambio de criptomonedas a través de la librería CCXT, la gestión segura de usuarios mediante autenticación basada en tokens, la implementación de una interfaz dinámica basada en React, y el diseño de una estructura robusta de base de datos que permita manejar datos históricos y en tiempo real.

A pesar de estos desafíos, se concluyó que el proyecto era técnicamente viable gracias a una planificación modular que permitió dividir el trabajo en bloques de desarrollo consecutivos. Cada etapa se diseñó de forma progresiva, permitiendo adaptar la carga de trabajo a lo largo del tiempo. En total, se estimaron unas 300 horas de dedicación efectiva distribuidas en 15 semanas, con jornadas de trabajo de aproximadamente 4 horas diarias de lunes a viernes. Esta organización permitió destinar tiempo específico a validación, pruebas funcionales y ajustes finales antes del despliegue.

En cuanto a la infraestructura técnica, se optó por herramientas de código abierto y tecnologías bien consolidadas dentro del ámbito del desarrollo web, lo que redujo la curva de aprendizaje y garantizó la disponibilidad de documentación y soporte comunitario. Esta elección estratégica no solo facilitó la implementación, sino que también aseguró una mayor estabilidad y sostenibilidad del sistema a largo plazo.

En resumen, considerando la experiencia previa, el marco temporal previsto, la disponibilidad de recursos tecnológicos y la definición estructurada del trabajo, se determinó que el desarrollo del proyecto era factible técnicamente, siempre que se mantuviera una disciplina metodológica constante durante el proyecto.

3.2. Análisis de requisitos

En esta sección se identifican y describen los objetivos principales, los actores involucrados, y los requisitos funcionales y no funcionales que orientan el desarrollo del sistema. En un entorno profesional, es común realizar una fase rigurosa de recopilación y validación de requisitos con el cliente o un equipo especializado. Sin embargo, en este proyecto individual, ese proceso se ha desarrollado de forma más dinámica y flexible, adaptándose durante la planificación y ejecución de los módulos.

Por ello, en lugar de contar con una lista fija y definitiva de requisitos, se ha optado por una evolución progresiva, seleccionando y perfeccionando las funcionalidades según su factibilidad técnica y relevancia respecto a los objetivos del sistema.

Para presentar ordenadamente el comportamiento esperado del software, se incluyen tablas que detallan funciones, requisitos, objetivos y actores. Esta información se complementa con diagramas de casos de uso que facilitan la comprensión de las relaciones y responsabilidades de cada actor. Para un análisis más integral de las funcionalidades del diagrama completo, véase el Anexo 2.



3.2.1. Objetivos

OBJ-001	Crear un entorno visual para diseñar estrategias de trading algorítmico
Descripción	El sistema debe permitir crear estrategias con condiciones lógicas, mostrando el gráfico de velas japonesas e indicadores técnicos del par seleccionado.
Estabilidad	Alta
Comentarios	Las órdenes ejecutadas deben mostrarse visualmente sobre el gráfico de velas.

Tabla 2: OBJ-001, Crear un entorno visual para diseñar estrategias de trading algorítmico

OBJ-002	Permitir explorar, gestionar y clonar estrategias
Descripción	La plataforma debe ofrecer la posibilidad de consultar un conjunto de estrategias públicas o personales, así como clonarlas para adaptarlas a nuevas necesidades o criterios.
Estabilidad	Alta
Comentarios	El creador decide si la estrategia será privada o visible para otros usuarios.

Tabla 3: OBJ-002, Permitir explorar, gestionar y clonar estrategias

OBJ-003	Ejecutar operaciones mediante integración con exchanges
Descripción	El sistema debe ejecutar órdenes automáticamente usando claves API del usuario, conectando con exchanges compatibles de forma segura.
Estabilidad	Alta
Comentarios	Una configuración incorrecta puede comprometer la operativa. Por ello, se requiere una validación robusta y mecanismos de control sobre las órdenes generadas.

Tabla 4: OBJ-003, Ejecutar operaciones mediante integración con exchanges

OBJ-004	Mostrar resultados y métricas de ejecución de estrategias
Descripción	La aplicación debe generar reportes visuales y numéricos tras la ejecución de una estrategia, reflejando el rendimiento, las señales activadas y los resultados obtenidos.
Estabilidad	Alta
Comentarios	Ofrecer datos claros y comprensibles ayuda al usuario a identificar patrones, errores o aciertos, fomentando una mejora continua basada en la experiencia.

Tabla 5: OBJ-004, Mostrar resultados y métricas de ejecución de estrategias

OBJ-005	Centralizar funciones clave en un panel de control global
Descripción	El sistema debe mostrar métricas de los resultados globales y las operaciones recientes para facilitar el análisis.
Estabilidad	Alta
Comentarios	Este panel actúa como punto de entrada principal a la plataforma, mejorando la navegación y facilitando el seguimiento del estado general del sistema.

Tabla 6: OBJ-005, Centralizar funciones clave en un panel de control operativo



3.2.2. Actores

ACT-001	Usuario no autenticado
Rol	Usuario potencial que aún no posee cuenta.
Descripción	Corresponde a cualquier persona que accede a la aplicación sin estar registrada o sin haber iniciado sesión. Solo puede acceder a las funciones básicas como iniciar sesión, registrarse, recuperar contraseña y visualizar la página principal.
Comentarios	Ninguno.

Tabla 7: ACT-001, Usuario no autenticado

ACT-002	Usuario autenticado
Rol	Usuario con cuenta que puede acceder a funcionalidades avanzadas.
Descripción	Usuario que ha iniciado sesión y tiene acceso al dashboard (panel principal), configuración de usuario, gestión de claves API, personalización y exploración de estrategias, ejecución de estrategias y visualización de resultados.
Comentarios	Ninguno.

Tabla 8: ACT-002, Usuario autenticado

3.2.3. Requisitos funcionales

En el proceso de definición de los requisitos funcionales de la plataforma, se ha considerado conveniente evitar una descripción exhaustiva de cada funcionalidad secundaria, con el objetivo de mantener la claridad y concisión del documento. Por esta razón, se ha optado por sintetizar los elementos funcionales más relevantes en formato tabular, priorizando aquellos de mayor impacto dentro del sistema y que, de manera implícita, agrupan diversas operaciones complementarias.

Esta presentación permite ofrecer una visión general estructurada de las capacidades fundamentales de la aplicación sin recurrir al detalle técnico de cada componente menor. Para una descripción más precisa de las acciones que puede realizar el usuario, se remite a los diagramas de casos de uso incluidos en el apartado correspondiente, concretamente en las Figuras 2 a 11. Asimismo, en el Anexo 2 se proporciona un esquema completo de estos diagramas, y en las historias de usuario se ejemplifican de manera práctica los distintos escenarios de interacción con el sistema.

RF-001	Gestionar usuarios y autenticación
Objetivos relacionados	OBJ-001
Requisitos de información relacionados	RI-001
Actores relacionados	ACT-001
Descripción	El sistema deberá permitir a los usuarios registrarse, iniciar y cerrar sesión, recuperar contraseña, así como verificación de correo electrónico. La autenticación se realizará mediante tokens JWT.
Comentarios	Este requisito incluye todas las funcionalidades necesarias para la gestión de sesiones de usuario y su acceso a la plataforma.

Tabla 9: RF-001, Gestionar usuarios y autenticación



RF-002	Configurar perfil de usuario
Objetivos relacionados	OBJ-001
Requisitos de información relacionados	RI-001
Actores relacionados	ACT-002
Descripción	El sistema permitirá a usuarios autenticados modificar datos personales, como nombre de usuario, correo electrónico, contraseña y zona horaria.
Comentarios	Todas las opciones relacionadas con el perfil estarán accesibles desde una única sección para mejorar la experiencia del usuario.

Tabla 10: RF-002, Configurar perfil de usuario

RF-003	Visualizar panel de control con métricas globales
Objetivos relacionados	OBJ-005
Requisitos de información relacionados	RI-003, RI-004, RI-005
Actores relacionados	ACT-002
Descripción	El sistema mostrará al usuario autenticado un panel con métricas generales sobre sus estrategias, pudiendo filtrar por estrategias ejecutadas en simulación o en modo real. Además, se mostrará las últimas órdenes ejecutadas en un rango de tiempo configurable.
Comentarios	Este panel centraliza información útil para el análisis global de la actividad del usuario, promoviendo una visión general, clara y accesible.

Tabla 11: RF-003, Visualizar panel de control con métricas globales

RF-004	Gestionar claves API de plataformas de intercambio
Objetivos relacionados	OBJ-003
Requisitos de información relacionados	RI-002
Actores relacionados	ACT-002
Descripción	El sistema permitirá al usuario conectar su cuenta de exchange mediante la introducción de claves API, que serán almacenadas de forma segura y utilizadas exclusivamente para ejecutar operaciones autorizadas.
Comentarios	La gestión de claves consistirá en un formulario para su almacenamiento.

Tabla 12: RF-004, Gestionar claves API de plataformas de intercambio



RF-005	Explorar y filtrar estrategias
Objetivos relacionados	OBJ-002
Requisitos de información relacionados	RI-003
Actores relacionados	ACT-002
Descripción	El sistema ofrecerá al usuario una sección para visualizar un catálogo de estrategias públicas o propias, con opción a filtrar por nombre, exchange, par de monedas y ordenarlas por número de copias.
Comentarios	Esta funcionalidad facilitará el descubrimiento de nuevas estrategias y fomentará la personalización del portafolio del usuario.

Tabla 13: RF-005, Explorar y filtrar estrategias

RF-006	Administrar y personalizar estrategia
Objetivos relacionados	OBJ-002
Requisitos de información relacionados	RI-003
Actores relacionados	ACT-002
Descripción	El sistema permitirá a los usuarios crear, editar y eliminar estrategias, pudiendo definir atributos como el exchange, par de monedas y duración de vela en los que operará, así como gestionar la visibilidad de la estrategia.
Comentarios	Todas estas funcionalidades estarán agrupadas en una única interfaz para simplificar su acceso y mejorar la usabilidad.

Tabla 14: RF-006, Administrar y personalizar estrategia

RF-007	Clonar estrategia pública o propia
Objetivos relacionados	OBJ-002
Requisitos de información relacionados	RI-003
Actores relacionados	ACT-002
Descripción	El sistema permitirá a los usuarios clonar estrategias existentes, ya sean propias o públicas, para modificarlas y adaptarlas a sus preferencias sin alterar la original.
Comentarios	Esta funcionalidad promueve la reutilización y adaptación de estrategias existentes, permitiendo una curva de aprendizaje más rápida.

Tabla 15: RF-007, Clonar estrategia pública o propia



RF-008	Visualizar gráfico de velas japonesas
Objetivos relacionados	OBJ-001, OBJ-004
Requisitos de información relacionados	RI-006
Actores relacionados	ACT-002
Descripción	El sistema permitirá visualizar la evolución histórica de un par de criptomonedas en un exchange determinado mediante un gráfico de velas japonesas, siendo ajustable la duración de cada vela.
Comentarios	Los datos visualizados serán obtenidos en tiempo real desde APIs externas, permitiendo al usuario analizar el comportamiento del mercado.

Tabla 16: RF-008, Visualizar gráfico de velas japonesas

RF-009	Gestionar indicadores técnicos en el gráfico
Objetivos relacionados	OBJ-001, OBJ-004
Requisitos de información relacionados	RI-003, RI-006
Actores relacionados	ACT-002
Descripción	El sistema permitirá añadir, eliminar y configurar indicadores técnicos sobre el gráfico de velas, ajustando sus parámetros desde la interfaz.
Comentarios	La configuración de indicadores será visual y reactiva, permitiendo ver los cambios aplicados directamente sobre el gráfico.

Tabla 17: RF-009, Gestionar indicadores técnicos en el gráfico

RF-010	Gestionar condiciones para órdenes
Objetivos relacionados	OBJ-001, OBJ-003
Requisitos de información relacionados	RI-003, RI-004
Actores relacionados	ACT-002
Descripción	El sistema permitirá al usuario establecer condiciones lógicas que determinen la ejecución de órdenes de compra o venta, basadas en los indicadores seleccionados y diversas condiciones del mercado.
Comentarios	Las condiciones podrán combinarse mediante operadores lógicos, y el sistema validará que sean coherentes antes de su activación.

Tabla 18: RF-010, Gestionar condiciones para órdenes



RF-011	Administrar ejecuciones de estrategias
Objetivos relacionados	OBJ-003
Requisitos de información relacionados	RI-003, RI-004, RI-005
Actores relacionados	ACT-002
Descripción	El sistema permitirá activar o detener la ejecución de las estrategias configuradas, mostrando su estado en tiempo real y permitiendo el control manual del usuario.
Comentarios	La plataforma mantendrá un registro continuo del estado de ejecución para cada estrategia.

Tabla 19: RF-011, Administrar ejecuciones de estrategias

RF-012	Gestionar historial de ejecuciones
Objetivos relacionados	OBJ-003, OBJ-004
Requisitos de información relacionados	RI-004, RI-005
Actores relacionados	ACT-002
Descripción	El sistema ofrecerá al usuario acceso a un historial detallado de ejecuciones pasadas, incluyendo fecha, par de criptomonedas, resultados obtenidos y el conjunto de órdenes ejecutadas.
Comentarios	Esta funcionalidad permite analizar el rendimiento histórico de cada estrategia y mejorar la toma de decisiones futura.

Tabla 20: RF-012, Gestionar historial de ejecuciones

3.2.4. Casos de uso

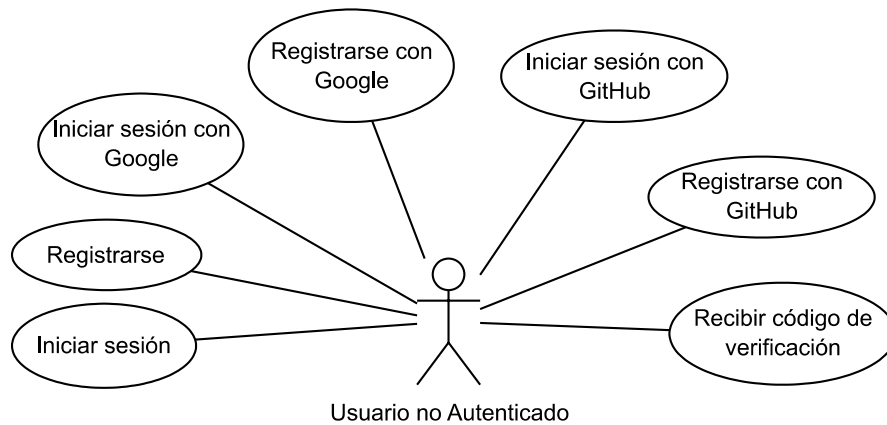


Figura 2: UC-1, Usuario no Autenticado

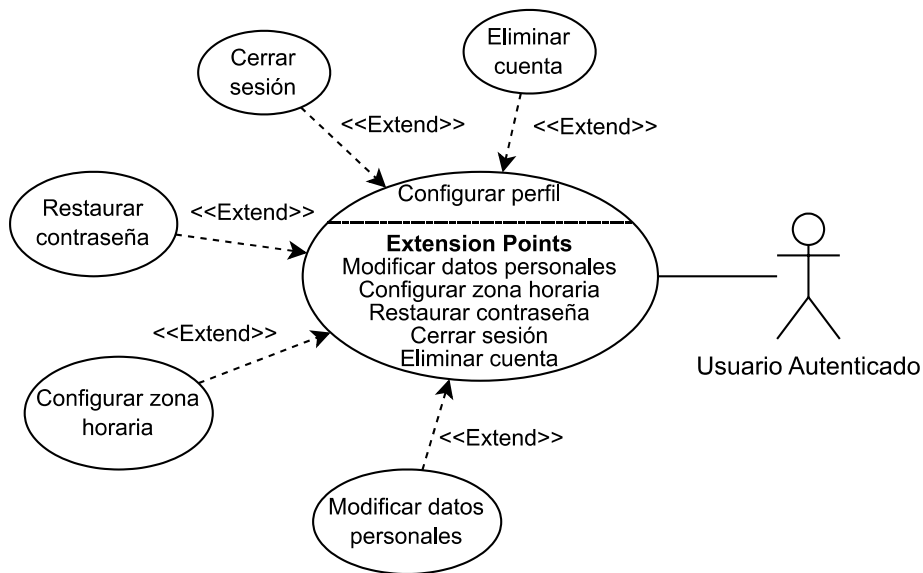


Figura 3: UC-2, Usuario Autenticado, Configurar Perfil

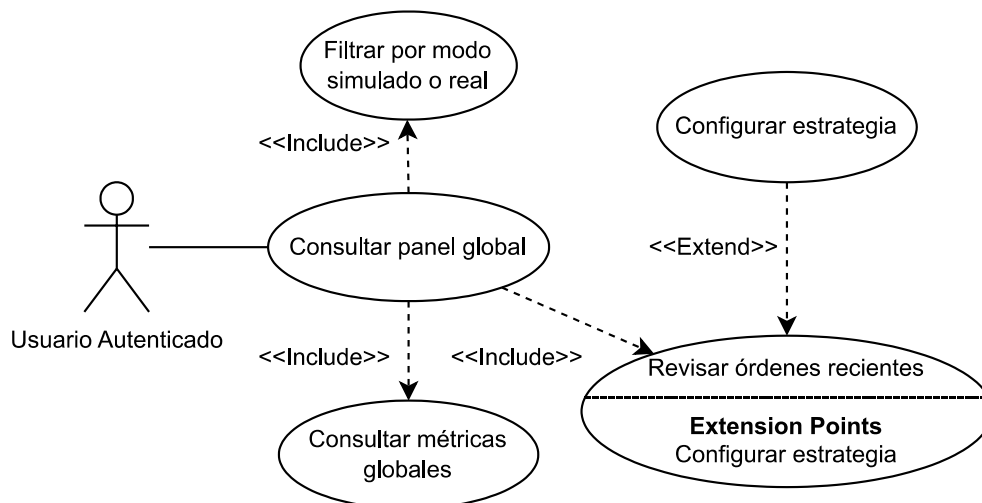


Figura 4: UC-3, Usuario Autenticado, Consultar panel global

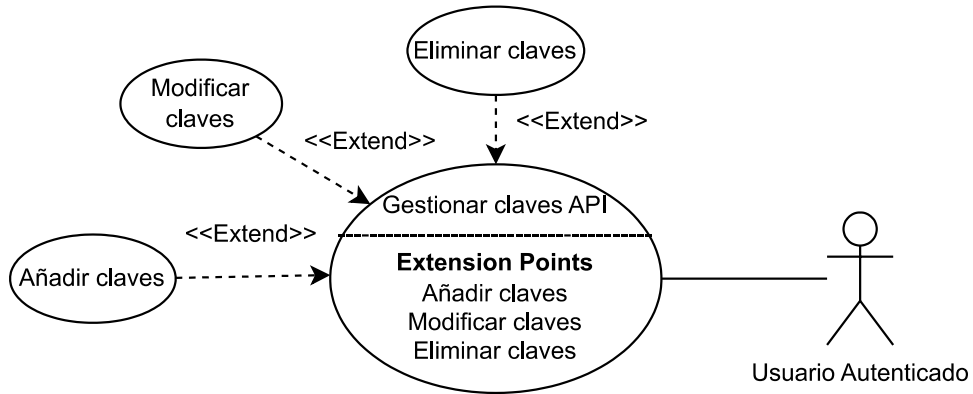


Figura 5: UC-4, Usuario Autenticado, Gestionar claves API

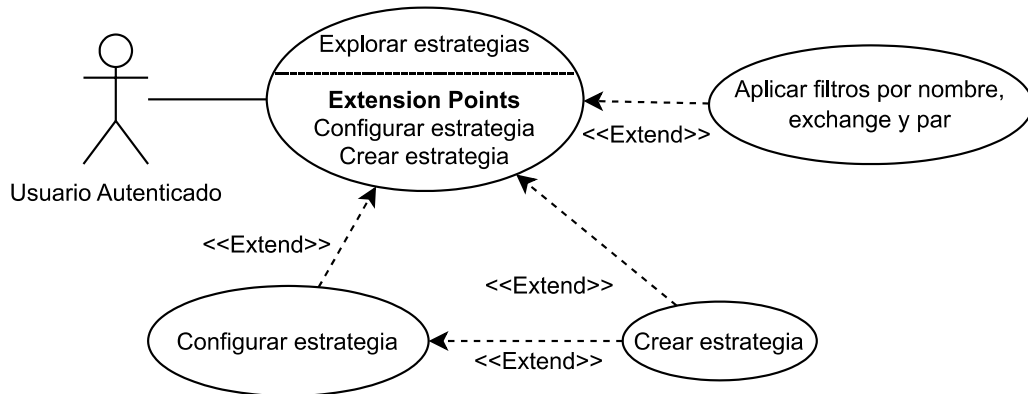


Figura 6: UC-5, Usuario Autenticado, Explorar estrategias

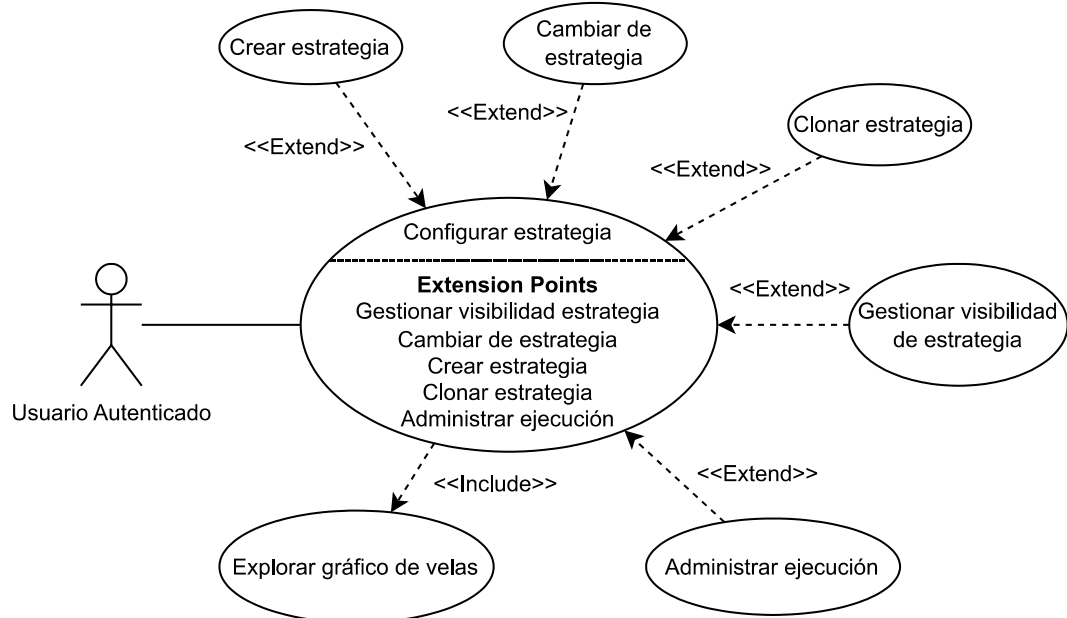


Figura 7: UC-6, Usuario Autenticado, Configurar estrategia

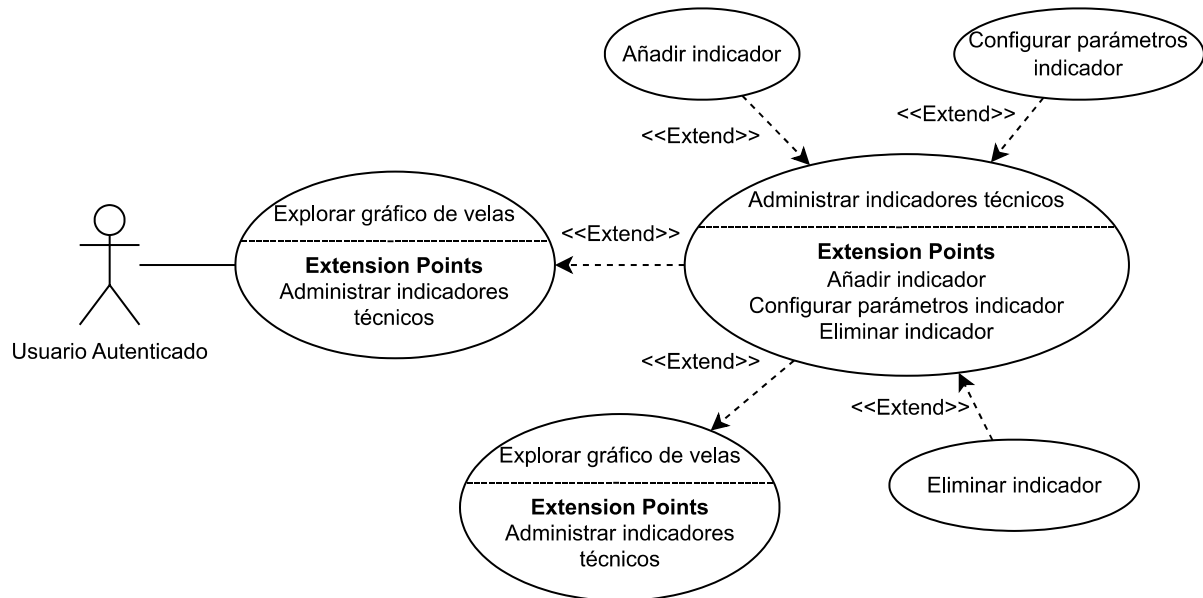


Figura 8: UC-7, Usuario Autenticado, Explorar gráfico de velas

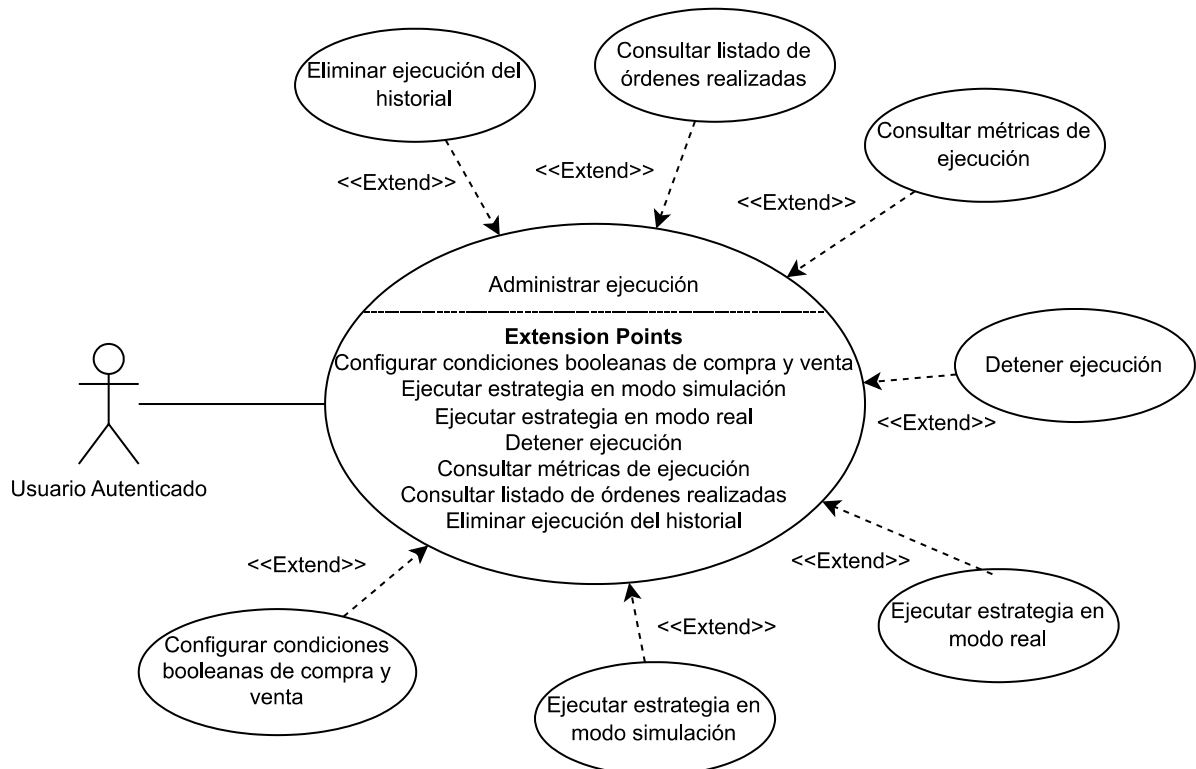


Figura 9: UC-8, Usuario Autenticado, Administrar ejecución



3.2.5. Requisitos de información

Esta sección define los datos que deben guardarse de forma persistente dentro del sistema. Se identifican los elementos fundamentales sobre los cuales es necesario conservar información, así como los atributos específicos relevantes. Estos requisitos deben detallar con claridad qué contenido debe registrar la aplicación para permitir el desarrollo de las funcionalidades planteadas. Para consultar una representación más general de estos requisitos en un diagrama de base de datos, véase el Anexo 3.

RI-001	Usuario
Objetivos asociados	OBJ-001
Requisitos asociados	RF-001, RF-002
Descripción	El sistema almacenará la información necesaria para la autenticación, configuración y personalización del perfil del usuario.
Datos específicos	• Id • Correo_electronico • Nombre_de_usuario • Contraseña • Zona_horaria • Tema_oscuro • Panel_operativa_real • Id_google • Id_github
Importancia	Alta
Prioridad	Alta
Comentarios	Este requisito es clave para garantizar el acceso seguro y personalizado a la plataforma, incluyendo autenticación por terceros (Google, GitHub).

Tabla 21: RI-001, Usuario

RI-002	Claves Api
Objetivos asociados	OBJ-003
Requisitos asociados	RF-004
Descripción	El sistema almacenará las claves API que permiten la conexión entre el usuario y las plataformas de intercambio externas, garantizando su uso seguro y restringido a las operaciones autorizadas.
Datos específicos	• Id • Usuario_id • Exchange • Clave_api • Secreto • Contraseña • Uid
Importancia	Alta
Prioridad	Alta
Comentarios	Este requisito es esencial para la ejecución automatizada de estrategias en exchanges reales, y requiere medidas de seguridad robustas.

Tabla 22: RI-002, Claves Api



RI-003	Estrategia
Objetivos asociados	OBJ-001, OBJ-002
Requisitos asociados	RF-005, RF-006, RF-007
Descripción	El sistema almacenará la información completa de cada estrategia creada o clonada por un usuario, incluyendo su configuración operativa y visibilidad.
Datos específicos	• Id • Usuario_id • Nombre • Exchange • Par • Duración_vela • Marca_tiempo_inicio • Marca_tiempo_fin • Indicadores • Es_pública • Cantidad_clonaciones
Importancia	Alta
Prioridad	Alta
Comentarios	Este requisito respalda la creación, gestión y exploración de estrategias, favoreciendo la reutilización y personalización.

Tabla 23: RI-003, Estrategia



RI-004	Ejecución Estrategia
Objetivos asociados	OBJ-003, OBJ-004
Requisitos asociados	RF-010, RF-011, RF-012
Descripción	El sistema almacenará los datos asociados a la ejecución de una estrategia concreta, incluyendo resultados de rendimiento y condiciones activadas.
Datos específicos	• Id • Estrategia_id • Tipo • Condiciones_orden • En_ejecución • Exchange • Par • Duración_vela • Marca_tiempo_inicio • Marca_tiempo_fin • Indicadores • Comisión_maker • Comisión_taker • Valor_inicial_comerciable • Apalancamiento • Marca_tiempo_ejecución • Ganancia_neta_absoluta • Ganancia_neta_relativa • Total_operaciones_cerradas • Tasa_operaciones_ganadoras • Factor_ganancia • Ganancia_media_absoluta • Ganancia_media_relativa • Mayor_serie_ganancias_absoluta • Mejor_serie_ganancias_relativa • Mayor_serie_pérdidas_absoluta • Mayor_serie_pérdidas_relativa
Importancia	Alta
Prioridad	Alta
Comentarios	Este requisito permite un análisis detallado de cada ejecución, ayudando a evaluar la eficacia de las estrategias.

Tabla 24: RI-004, Ejecución Estrategia



RI-005	Operación
Objetivos asociados	OBJ-003, OBJ-004
Requisitos asociados	RF-011, RF-012
Descripción	El sistema almacenará cada operación generada durante la ejecución de una estrategia, incluyendo información económica y de rentabilidad individual.
Datos específicos	• Id • Ejecución_estrategia_id • Tipo • Dirección • Marca_tiempo • Precio • Cantidad • Coste • Precio_entrada_promedio • Ganancia_absoluta • Ganancia_relativa • Ganancia_acumulada_absoluta • Ganancia_acumulada_relativa • Ganancia_hodling_absoluta • Ganancia_hodling_relativa • Serie_ganancias_absoluta • Serie_ganancias_relativa • Serie_pérdidas_absoluta • Serie_pérdidas_relativa
Importancia	Alta
Prioridad	Alta
Comentarios	Este requisito es fundamental para construir las métricas agregadas de rendimiento y realizar un seguimiento detallado de cada acción en el mercado.

Tabla 25: RI-005, Operación



RI-006	Vela
Objetivos asociados	OBJ-001, OBJ-004
Requisitos asociados	RF-008, RF-009
Descripción	El sistema almacenará la información de cada vela japonesa utilizada para la visualización del comportamiento del mercado y para la lógica de las condiciones de trading.
Datos específicos	• Id • Exchange • Par • Duración_vela • Marca_tiempo • Apertura • Máximo • Mínimo • Cierre • Volumen
Importancia	Alta
Prioridad	Alta
Comentarios	Este requisito permite el análisis visual y técnico de los datos históricos del mercado, siendo la base del análisis gráfico de estrategias.

Tabla 26: RI-006, Vela

3.2.6. Requisitos no funcionales

RNF-001	Robustez ante fallos
Descripción	La aplicación debe mantener su estabilidad general ante errores inesperados durante la operación, evitando bloqueos totales o cierres forzados.
Comentarios	El sistema debe evitar estados críticos que impidan el uso continuo.

Tabla 27: RNF-001, Robustez ante fallos

RNF-002	Adaptabilidad de la interfaz
Descripción	La interfaz gráfica debe adaptarse a distintas resoluciones y tamaños de pantalla, permitiendo su uso en ordenadores y móviles sin pérdida de funcionalidad.
Comentarios	Se garantiza el acceso a las funcionalidades esenciales desde múltiples dispositivos.

Tabla 28: RNF-002, Adaptabilidad de la interfaz

RNF-003	Manejo seguro de credenciales
Descripción	Las contraseñas deben almacenarse mediante funciones hash seguras. Las claves API deben cifrarse y restringirse exclusivamente a procesos autorizados.
Comentarios	No debe permitirse el acceso a datos sensibles en texto plano desde ningún componente del sistema.

Tabla 29: RNF-003, Manejo seguro de credenciales



RNF-004	Usabilidad para perfiles no técnicos
Descripción	El sistema debe facilitar la creación, configuración y supervisión de estrategias mediante interfaces visuales e intuitivas, sin requerir conocimientos avanzados.
Comentarios	La curva de aprendizaje debe ser baja, permitiendo a nuevos usuarios operar sin necesidad de documentación extensa.

Tabla 30: RNF-004, Usabilidad para perfiles no técnicos

RNF-005	Rendimiento y tiempos de respuesta adecuados
Descripción	La ejecución de simulaciones no debe superar los 30 segundos en escenarios con hasta 50.000 velas. Las acciones frecuentes, como cargar gráficos o consultar resultados, deben responder en menos de 5 segundos en condiciones normales.
Comentarios	Estos límites aseguran una experiencia fluida tanto en análisis como en navegación.

Tabla 31: RNF-005, Rendimiento y tiempos de respuesta adecuados

RNF-006	Escalabilidad técnica
Descripción	El sistema deberá escalar progresivamente según la carga. La ejecución multihilo será suficiente al inicio, pero ante un aumento considerable de usuarios simultáneos se requerirá mayor inversión en infraestructura.
Comentarios	La arquitectura deberá permitir distribuir la ejecución de estrategias para mantener el rendimiento operativo.

Tabla 32: RNF-006, Escalabilidad técnica

3.3. Prototipado inicial. Mockups

Con el fin de anticipar visualmente el desarrollo del proyecto, se diseñó un prototipo inicial de carácter orientativo mediante el uso de la herramienta Figma. Esta elección se fundamentó en las capacidades que ofrece dicha plataforma para el diseño rápido de aplicaciones web, incluyendo una extensa biblioteca de elementos prediseñados como formularios, mapas interactivos, sistemas de mensajería, menús desplegables y botones, lo que facilitó la creación eficiente de una maqueta representativa del producto final.

Este esquema visual, conocido como mockup, no tenía como finalidad definir la interfaz definitiva, sino más bien actuar como una referencia preliminar que ayudara a estructurar conceptualmente la propuesta y comunicarla de manera clara al tutor del proyecto. Gracias a esta estrategia, fue posible revisar y validar las principales ideas que sustentan la aplicación, lo cual permitió establecer una base sólida para iniciar el diseño del entorno de usuario.

A medida que avanzó el trabajo durante los meses posteriores, la interfaz definitiva fue transformándose de manera sustancial respecto a aquella primera versión creada en Figma. Este cambio, lejos de ser inesperado, respondía a la función que cumplía el prototipo: no aspiraba a replicar el diseño final, sino a servir como herramienta inicial para analizar el recorrido del usuario, los elementos funcionales más relevantes y el esquema general de interacción. En la Figura 10 se muestra el prototipo que sirvió de referencia en las primeras etapas del proyecto.

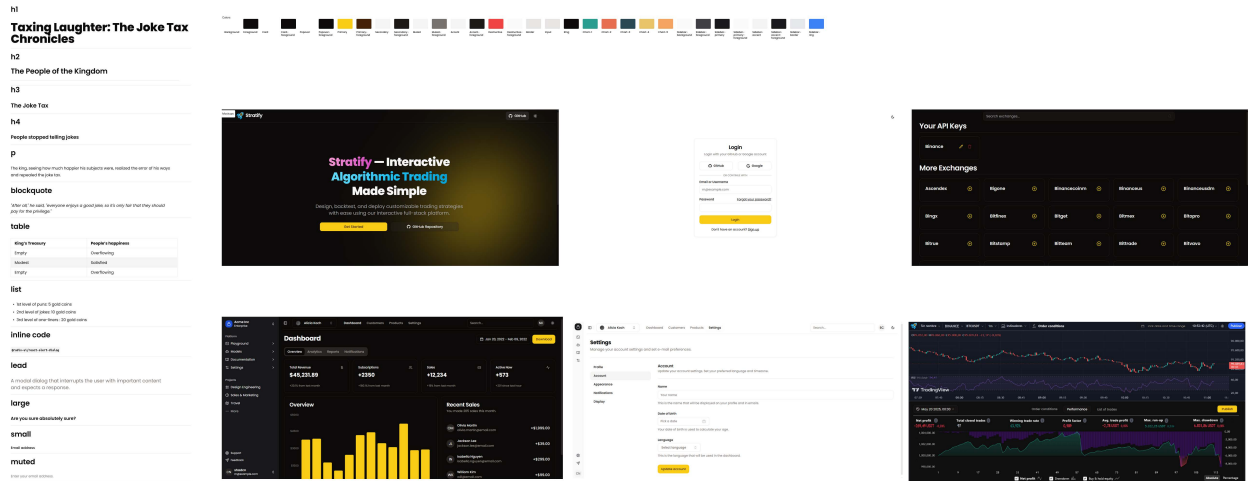


Figura 10: Mockups iniciales

4. Fundamentos del análisis técnico

Una vez definida la estructura y el alcance del proyecto, así como los objetivos y las especificaciones necesarias para su desarrollo, es imprescindible abordar los fundamentos teóricos que sustentan la lógica operativa del sistema. En este sentido, el análisis técnico representa una base esencial para la interpretación y el estudio del comportamiento del mercado financiero a partir de datos históricos. Antes de avanzar en la implementación técnica, es necesario comprender los principios clave que guían esta disciplina, garantizando así una correcta aplicación en el desarrollo del backend.

El análisis técnico se apoya principalmente en dos elementos fundamentales: las velas japonesas [24] y los indicadores técnicos. Las velas japonesas ofrecen una representación visual del movimiento de precios a lo largo del tiempo, facilitando la comprensión de la dinámica del mercado en diferentes intervalos temporales. Los indicadores técnicos, por su parte, son herramientas cuantitativas que derivan información valiosa a partir de dichos datos históricos para apoyar la toma de decisiones en estrategias de trading. Más adelante en este informe, se profundizará en estos conceptos básicos como base para el diseño y la implementación del algoritmo de simulación y ejecución de operaciones.

4.1. Funcionamiento y estructura de las velas japonesas

Las velas japonesas son una herramienta esencial dentro del análisis técnico que se utiliza para mostrar, de manera visual, cómo ha cambiado el precio de un activo financiero durante un periodo de tiempo determinado. Cada vela resume cuatro datos importantes: el precio al inicio del periodo (apertura), el precio al final (cierre), el valor más alto alcanzado (máximo) y el más bajo (mínimo). Esta forma de representar los precios facilita la comprensión rápida de lo que está ocurriendo en el mercado, ya que permite ver con claridad si los compradores o los vendedores han tenido más influencia durante ese intervalo y cómo ha sido el movimiento del precio.

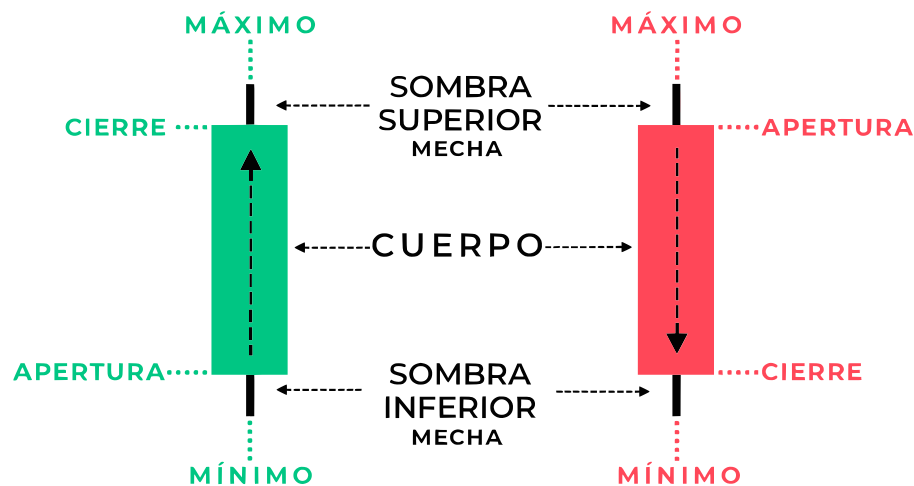


Figura 11: Estructura de una vela japonesa alcista y bajista

El cuerpo central de la vela refleja la diferencia entre el precio de apertura y el precio de cierre. Si el precio al cerrar es mayor que al abrir, la vela se denomina alcista, indicando que durante ese tiempo hubo más compra que venta; estas velas suelen mostrarse con colores claros o verdes para simbolizar crecimiento o subida. En cambio, si el precio de cierre es menor que el de apertura, la vela se considera bajista, lo que significa que predominó la venta y el precio bajó; estas velas suelen representarse en colores oscuros o rojos. Encima y debajo del cuerpo de la vela están las “mechas” o “sombras”, que representan los precios máximos y mínimos alcanzados durante ese mismo periodo. Estas mechas nos muestran la volatilidad, es decir, cuánto ha variado el precio dentro del intervalo, y reflejan posibles movimientos de rechazo o estabilización en ciertos niveles.

Cabe señalar que la duración de cada vela, conocida como intervalo temporal o timeframe, es un parámetro configurable que determina la cantidad de datos que se condensan en cada unidad gráfica. Dependiendo del

objetivo del análisis, se pueden utilizar marcos temporales muy breves, como velas de 1 minuto (1m) o 5 minutos (5m), útiles en estrategias de alta frecuencia, hasta intervalos más amplios como 1 hora (1h), 4 horas (4h) o 1 día (1d), más apropiados para estudios de tendencias a medio o largo plazo. La elección del intervalo afecta directamente a la interpretación de los movimientos del mercado, por lo que resulta esencial ajustarlo en función del contexto operativo o del tipo de estrategia a desarrollar.

Es importante destacar que la forma y el tamaño del cuerpo y las mechas proporcionan información adicional sobre la fuerza y la actividad del mercado. Por ejemplo, un cuerpo grande y mechas cortas indica que hubo un movimiento claro y fuerte en una dirección determinada, mientras que cuerpos pequeños con mechas largas pueden señalar dudas o indecisión entre compradores y vendedores. Esta información ayuda a los analistas a entender mejor la situación del mercado y a anticipar posibles cambios o continuaciones en la tendencia de los precios.

Sin embargo, aunque el análisis de las velas japonesas es muy útil, estas no deben interpretarse de manera aislada, ya que su efectividad aumenta cuando se combinan con indicadores técnicos. Estos permiten complementar la información visual con datos cuantitativos, mejorando la toma de decisiones.

4.2. Funcionamiento y clasificación de indicadores técnicos

Los indicadores técnicos son herramientas matemáticas aplicadas a datos de precios y volumen que permiten interpretar el comportamiento del mercado desde una perspectiva cuantitativa. Su función principal es complementar la información ofrecida por las velas japonesas, ayudando a identificar tendencias, confirmar señales de entrada o salida, y anticipar posibles cambios en la dirección del precio. Estos indicadores no predicen el futuro, pero proporcionan una base sólida para tomar decisiones más fundamentadas, especialmente en contextos de alta volatilidad o ambigüedad en el gráfico.

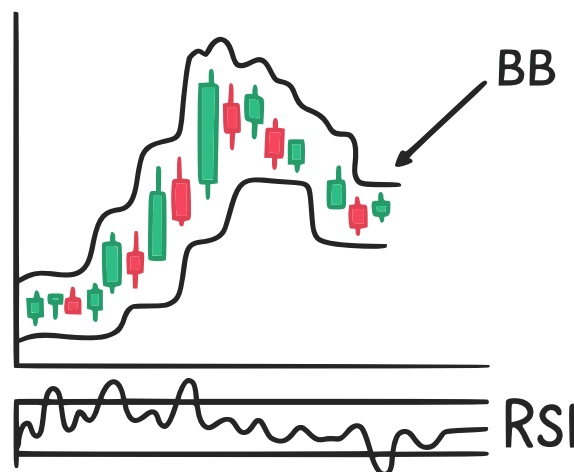


Figura 12: Boceto explicativo de los indicadores RSI y Bollinger Bands sobre el gráfico de velas

Existen múltiples formas de clasificar los indicadores técnicos, aunque una de las más extendidas se basa en la función principal que desempeñan. Bajo esta lógica, los indicadores implementados se agrupan en cuatro categorías: de tendencia, de momentum, de volatilidad y de volumen. Esta clasificación permite estructurar el análisis y adaptar su uso según el tipo de estrategia o el contexto del mercado.

- Los indicadores de tendencia están diseñados para identificar la dirección general del mercado, ya sea alcista, bajista o lateral. Son especialmente útiles en estrategias que buscan seguir la dirección predominante del precio, evitando operar contra la corriente del mercado. En esta categoría se han integrado indicadores como SMA (Simple Moving Average) y EMA (Exponential Moving Average), que suavizan los datos para resaltar la tendencia subyacente; MACD [25], que mide la convergencia y divergencia de medias móviles; ADX, que cuantifica la fuerza de la tendencia; así como Parabolic SAR, Aroon, y TRIX, que permiten detectar posibles cambios de dirección.
- Por otro lado, los indicadores de momentum evalúan la velocidad y fuerza con la que se están moviendo los precios. Su objetivo es detectar cuándo una tendencia está ganando o perdiendo impulso. Dentro de esta categoría se encuentran el RSI (Relative Strength Index) [26], uno de los más



conocidos, que indica condiciones de sobrecompra o sobreventa; Stochastic y Stochastic RSI, que comparan el precio actual con rangos anteriores; CCI, Momentum, ROC, Ultimate Oscillator, y Williams %R, cada uno con metodologías distintas para identificar desequilibrios temporales en la dinámica de precios.

- Los indicadores de volatilidad permiten medir cuánto varía el precio en un determinado intervalo temporal. Esto es clave para ajustar estrategias de gestión de riesgo o detectar momentos de compresión o expansión del mercado. En este grupo se encuentran Bollinger Bands [27], que definen rangos dinámicos en torno a una media móvil, y ATR (Average True Range), que calcula la volatilidad media del activo en función del rango diario.
- Por último, los indicadores de volumen vinculan los movimientos de precio con el volumen negociado, permitiendo evaluar si un movimiento es respaldado por una participación significativa del mercado. En este caso, se utilizan herramientas como OBV (On Balance Volume), que acumula el volumen según la dirección del precio; MFI (Money Flow Index), que combina volumen y precio para evaluar la presión de compra o venta; y Chaikin A/D Line (AD), que estima la acumulación o distribución de un activo.



5. Desarrollo del servidor (backend)

El lado del servidor constituye el núcleo funcional de la plataforma, responsable de orquestar la lógica de negocio, asegurar la integridad de los datos y facilitar una comunicación segura con el resto de componentes del sistema. Para su desarrollo, se ha empleado el framework Django junto con Django REST Framework (DRF), una herramienta que permite construir APIs RESTful de manera estructurada, flexible y alineada con estándares ampliamente utilizados en entornos profesionales.

Este apartado aborda de manera detallada la organización interna del backend, desde la definición de los modelos y serializadores hasta la configuración de las vistas, rutas y sistemas de autenticación. Se hace especial énfasis en la manera en que estos elementos se integran para ofrecer una interfaz coherente que centraliza la gestión de usuarios, estrategias, indicadores y órdenes de ejecución.

Asimismo, se describen los criterios seguidos en las decisiones arquitectónicas, poniendo en valor tanto la modularidad como la claridad del diseño, con el objetivo de facilitar la escalabilidad y el mantenimiento del sistema. Se expone cómo se ha estructurado cada módulo funcional y cómo la API expuesta permite una interacción segura y eficiente con el cliente.

Además de las funcionalidades del framework, el backend integra librerías especializadas como CCXT, para la comunicación con plataformas de intercambio de criptomonedas, y TA-Lib, para el cálculo de indicadores técnicos sobre series temporales financieras. Estas herramientas se incorporaron mediante servicios internos que abstraen la lógica de acceso, facilitando su uso desde endpoints específicos sin afectar la cohesión del código base.

5.1. Arquitectura del backend con Django REST Framework

Antes de abordar el funcionamiento específico del backend, es fundamental comprender cómo se organiza un proyecto estándar desarrollado con Django. Este paso es clave para interpretar adecuadamente la interacción entre sus distintos módulos internos.

Django emplea una arquitectura basada en el patrón de diseño MVT (Modelo–Vista–Plantilla), que guarda cierta semejanza con el modelo MVC (Modelo–Vista–Controlador) [28], aunque presenta una nomenclatura diferente. En este esquema:

- El modelo se encarga del manejo de datos, y se implementa mediante clases en Python que se corresponden automáticamente con las estructuras de la base de datos.
- La vista incorpora la lógica encargada de procesar solicitudes, controlar el flujo de información y generar respuestas adecuadas.
- La capa de plantilla, normalmente encargada de mostrar el contenido visual al usuario, no se emplea en este proyecto. En su lugar, la interfaz se gestiona íntegramente desde el lado del cliente mediante React, mientras que el servidor se limita a proporcionar los datos a través de una API.

En vez de generar contenido HTML como en un enfoque tradicional, el servidor opera únicamente como una interfaz REST, respondiendo con información en formato JSON estructurado.

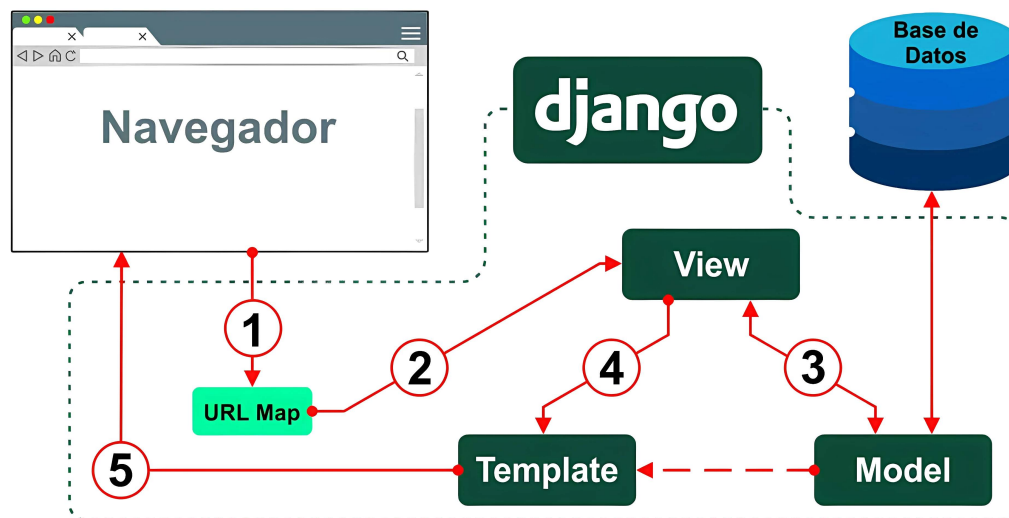


Figura 13: Estructura Modelo-Vista-Plantilla (MVT)

La Figura 13 ilustra el recorrido que sigue la arquitectura MVT, dejando claro que para interactuar con la base de datos desde la interfaz de usuario es imprescindible pasar por la capa intermedia, conocida como vista. Esta capa actúa como mediadora entre el frontend y la información almacenada, garantizando que el acceso a los datos se realice exclusivamente a través de los puntos definidos en la API REST. Así, se evita cualquier exposición directa de la base de datos desde la aplicación cliente.

La estructura de un proyecto en Django se compone de varios elementos esenciales:

- Un archivo llamado `manage.py`, que sirve como herramienta principal para ejecutar órdenes administrativas relacionadas con el entorno de desarrollo.
- Un directorio que representa el núcleo del proyecto (por ejemplo, `backend/`), el cual incluye archivos esenciales como `settings.py`, `urls.py`, `wsgi.py` y `asgi.py`, encargados de la configuración general y del despliegue. Cada nuevo sistema a desarrollar comienza con la creación de un único proyecto que agrupará diferentes aplicaciones internas.
- Dentro del proyecto, se desarrollan una o más aplicaciones (por ejemplo, `api/`, `user/`, etc.), cada una centrada en un conjunto específico de funciones. Estas aplicaciones contienen sus propios componentes, como `models.py`, `views.py`, `serializers.py`, `urls.py` y `admin.py`, donde se define el comportamiento particular de cada módulo que formará parte de la API.

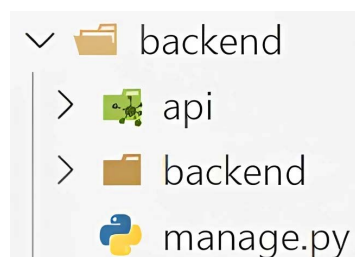


Figura 14: Estructura de carpetas en un proyecto Django

La Figura 14 muestra cómo se estructuró el proyecto llamado `backend`, el cual contiene una sola aplicación denominada `api`, todo ello bajo un directorio con el mismo nombre que el proyecto principal.

En los apartados siguientes se detallará la función de los distintos archivos fundamentales que conforman la aplicación. Cada uno de ellos contribuye al funcionamiento general de la API, desempeñando un papel específico en el procesamiento de datos y la entrega de respuestas. Para facilitar el desarrollo y mantener el proyecto organizado, se optó por consolidar toda la lógica en una única aplicación. No obstante, si el sistema llegara a crecer significativamente o fuera necesario incorporar nuevas capacidades, siempre existirá la opción de añadir otras aplicaciones adicionales, aparte de `api/`.



5.1.1. Definición de modelos de datos

En el marco de trabajo de Django, los modelos constituyen el fundamento sobre el cual se estructura y administra la información persistente de la aplicación. Estas entidades se implementan como clases que definen de forma declarativa la naturaleza y las reglas de los datos, estableciendo no solo sus tipos, sino también las relaciones lógicas y las restricciones necesarias para garantizar la coherencia del sistema.

Cada clase modela una unidad semántica del dominio del problema, y su definición se traduce automáticamente en una tabla relacional. Los atributos especificados en dichas clases determinan tanto los tipos de columnas como las propiedades de validación y enlace con otras estructuras. A su vez, este diseño proporciona una abstracción efectiva sobre el motor de base de datos subyacente, permitiendo a los desarrolladores operar sobre los datos mediante un lenguaje de alto nivel sin involucrarse directamente con instrucciones SQL.

La arquitectura del sistema de modelos admite también la incorporación de comportamientos personalizados mediante métodos definidos dentro de las propias clases. Estos pueden intervenir en momentos clave del ciclo de vida del objeto, como la creación, la modificación o la eliminación, o bien ofrecer representaciones textuales y lógicas que faciliten la interacción con otras capas del software. Además, las clases internas de configuración permiten ajustar diversos aspectos del modelo, tales como los criterios de ordenamiento por defecto, las restricciones de unicidad o la denominación explícita de las tablas.

Un elemento especialmente destacable es la capacidad de representar asociaciones complejas entre entidades, como las de cardinalidad múltiple o las relaciones jerárquicas. Estas se expresan mediante campos especializados que encapsulan los principios de integridad referencial, contribuyendo a una representación clara y modular del dominio de la aplicación.

Gracias al sistema de mapeo objeto-relacional (ORM), las operaciones de lectura, escritura y modificación se realizan de manera eficiente y transparente, asegurando la integridad transaccional sin comprometer la claridad del código. Esta estrategia de diseño permite, además, encapsular toda la lógica relacionada con los datos directamente en la capa de modelo, evitando la dispersión de responsabilidades y favoreciendo la mantenibilidad a largo plazo.

En la Figura 15 se muestra una tabla como ejemplo del sistema de datos, mientras que la Figura 16 presenta su correspondiente definición en el archivo `models.py`.

User	
PK	<u>id UUID NOT NULL</u>
U	email varchar(254) NOT NULL
U	username varchar(150) NOT NULL
	password varchar(128) NOT NULL
	timezone_offset decimal(4,2) NOT NULL
	dark_theme boolean NOT NULL
	dashboard_real_trading boolean NOT NULL
U	google_id varchar(255)
U	github_id varchar(255)

Figura 15: Tabla User del modelo de datos

```
class User(AbstractUser):
    id = models.UUIDField(default=uuid.uuid4, editable=False, primary_key=True)
    email = models.EmailField(max_length=254, unique=True)
    username = models.CharField(max_length=150, unique=True)
    password = models.CharField(max_length=128)
    timezone = models.CharField(max_length=255, default="UTC")
    dark_theme = models.BooleanField(default=True)
    dashboard_real_trading = models.BooleanField(default=False)
    google_id = models.CharField(max_length=255, null=True, blank=True,
unique=True)
    github_id = models.CharField(max_length=255, null=True, blank=True,
unique=True)

    def __str__(self):
        return self.username
```

Figura 16: Clase User en models.py

5.1.2. Implementación de vistas

En el sistema desarrollado con Django, se ha prestado especial atención a cómo se gestionan las solicitudes que llegan desde el cliente. Este papel lo cumplen las vistas, que actúan como el punto de entrada para procesar cada petición entrante y generar una respuesta adecuada. Funcionan como el puente entre el exterior y la lógica interna del servidor, encapsulando el comportamiento necesario para responder de forma precisa según el tipo de operación solicitada.

Se han empleado dos enfoques complementarios para estructurar estas vistas: por un lado, el uso de API-View ha permitido una definición explícita y detallada en endpoints más específicos, donde era necesario un control fino sobre el tratamiento de datos o una validación más compleja. Por otro lado, ModelViewSet ha sido la elección para gestionar conjuntos de datos completos con operaciones básicas como creación, modificación o eliminación, simplificando considerablemente la escritura de código sin perder consistencia.

En cada uno de estos puntos de entrada, el sistema responde con datos estructurados en formato JSON y un código HTTP que informa sobre el estado de la operación. Estos códigos cumplen una función comunicativa esencial, permitiendo al cliente interpretar correctamente el resultado de su acción. Entre los más utilizados están los que indican éxito (como el 200 para respuestas satisfactorias o el 201 para creaciones exitosas), los que advierten de problemas del lado del cliente (como el 400 en caso de errores de formato o el 404 si el recurso no existe), y los que reflejan restricciones de acceso (como el 401 o el 403).

En situaciones donde la complejidad del flujo lo requiriera, parte del procesamiento se ha trasladado a los serializadores, lo que ha permitido mantener las vistas más limpias y centradas en coordinar el proceso general, sin que se vieran saturadas de lógica interna.

Además, todas las rutas sensibles del sistema han sido protegidas mediante un sistema de permisos basado en autenticación. Esta capa de seguridad impide que usuarios no identificados accedan a funcionalidades restringidas, y devuelve respuestas adecuadas cuando se intenta realizar una operación sin los privilegios necesarios. La validación se realiza antes de ejecutar cualquier lógica, reforzando la integridad del sistema.

Más adelante se explicará con detalle el mecanismo de autenticación empleado, que se apoya en tokens JWT (JSON Web Token) [29] y cookies para mantener las sesiones de forma segura. Mientras tanto, en las Figuras 17 y 18 se pueden observar respectivamente tanto la implementación de una vista como el resultado visual de su funcionamiento al ser consultada desde un navegador.



```
class StrategyExecutionView(viewsets.ModelViewSet):
    queryset = StrategyExecution.objects.all()
    serializer_class = StrategyExecutionSerializer

    def get_permissions(self):
        if self.action in ['start', 'stop', 'destroy']:
            self.permission_classes = [IsOwner]
        elif self.action in ['list', 'retrieve']:
            self.permission_classes = [IsAuthenticated]
        return super().get_permissions()

    def list(self, request, *args, **kwargs):
        strategy_id = request.query_params.get('strategy_id')
        if not strategy_id:
            return Response({"detail": "Missing strategy parameter"}, status=status.HTTP_400_BAD_REQUEST)
        try:
            strategy = Strategy.objects.get(id=strategy_id)
        except Strategy.DoesNotExist:
            return Response({"detail": "Strategy not found."}, status=status.HTTP_404_NOT_FOUND)
        if not strategy.is_public and strategy.user != request.user:
            return Response({"detail": "Not authorized to view this strategy"}, status=status.HTTP_403_FORBIDDEN)
        executions = StrategyExecution.objects.filter(strategy_id=strategy_id).values('id', 'execution_timestamp')
        return Response(list(executions))
```

Figura 17: Endpoint de listar ejecuciones en views.py

localhost:8000/api/v1/strategy-execution/?strategy_id=fac07a32-4f5b-4157-b119-0e0c211c9278

Django REST framework admin

Strategy Execution

Strategy Execution

OPTIONS GET

GET /api/v1/strategy-execution/?strategy_id=fac07a32-4f5b-4157-b119-0e0c211c9278

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
[
  {
    "id": "d044f155-6ba3-450c-857d-733c38a358ff",
    "execution_timestamp": 1749740651983
  },
  {
    "id": "2f045c38-763b-4af1-b1c4-9c2fd279d8fd",
    "execution_timestamp": 1749740622832
  }
]
```

Figura 18: Interfaz generada por Django al realizar la llamada al endpoint



5.1.3. Configuración de rutas

Una vez se ha definido qué es un endpoint y cómo se implementa en el servidor, resulta fundamental comprender el proceso para acceder a dicho endpoint. En Django, esta gestión se realiza a través de dos archivos homónimos, `urls.py`, encargados de marcar las rutas generales del proyecto y de registrar las rutas correspondientes a cada endpoint de la API. En estos archivos se definen las direcciones que permiten la comunicación con los recursos disponibles.

En el contexto de este proyecto, donde se emplea una única aplicación, todas las rutas de la misma se concentran en un solo archivo de configuración. Este archivo, ubicado en la carpeta principal del proyecto, importa y agrupa las rutas de la aplicación bajo un prefijo común (`/api`), facilitando así la organización y la escalabilidad del sistema.

Esta estructura jerarquizada contribuye a mantener un código ordenado y fácilmente mantenible, al establecer una URL única para cada endpoint. Las Figuras 19 y 20 ilustran el proceso de construcción de la URL correspondiente a un endpoint específico, previamente presentado en la Figura 18.

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', RedirectView.as_view(url='/admin/')),  
    path('api/', include('api.urls')),  
]
```

Figura 19: Configuración general de prefijos en backend/urls.py

```
urlpatterns = [  
    path("v1/strategy-execution/", views.StrategyExecutionView.as_view({"get": "list"}), name="strategy-execution-list"),  
    ...  
]
```

Figura 20: Url asociada al endpoint de listar ejecuciones

5.1.4. Serialización y validación de datos

En el desarrollo de APIs que siguen el estilo REST, la información se intercambia normalmente en formato JSON, un estándar ligero y fácil de interpretar. Para controlar este flujo de datos de manera clara y segura, Django REST Framework ofrece los serializadores, herramientas fundamentales que permiten estructurar, transformar y validar la información que circula entre el cliente y el servidor, asegurando así la coherencia y la integridad de los datos transmitidos.

Un buen ejemplo de su utilidad es el serializador de usuarios. En este caso, no solo se definen los campos del modelo que deben incluirse en las operaciones, como el correo electrónico, el nombre de usuario o la configuración del tema oscuro, sino que también se aplican reglas de validación para evitar conflictos o inconsistencias. Durante este proceso se verifica, por ejemplo, que no existan otros usuarios con el mismo nombre o correo. Si alguno de estos criterios no se cumple, se detiene el proceso y se informa del error, garantizando así que los datos que ingresan al sistema sean válidos y seguros.


```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'email', 'username', 'dark_theme', 'dashboard_real_trading',
        'timezone', 'password']
        extra_kwargs = {
            'password': { 'write_only': True }
        }

    def validate(self, attrs):
        if User.objects.exclude(id=self.instance.id if self.instance else None).filter(
            username=attrs['username']).exists():
            raise serializers.ValidationError("Username already in use")

        if User.objects.exclude(id=self.instance.id if self.instance else None).filter(
            email=attrs['email']).exists():
            raise serializers.ValidationError("Email already in use")
```

Figura 21: Serializador para la validación de nombre de usuario y correo

5.2. Seguridad y autenticación

Uno de los aspectos fundamentales en cualquier entorno web que gestiona información sensible y funcionalidades avanzadas es el control de acceso. En el caso de este proyecto, donde los usuarios operan con datos confidenciales como claves API, estrategias personalizadas y resultados de ejecución, se ha prestado especial atención a la implementación de un sistema de autenticación seguro y eficaz. Para ello, se ha optado por un modelo basado en tokens, que permite identificar al usuario de forma fiable y proteger las operaciones críticas frente a accesos no autorizados, manteniendo a su vez una experiencia de uso fluida y coherente con el entorno web.

5.2.1. Autenticación mediante JWT

El sistema de autenticación adoptado en la plataforma se basa en el uso de JSON Web Tokens como mecanismo principal para validar la identidad del usuario en sus interacciones con el servidor. Esta técnica permite establecer sesiones seguras sin almacenar información sensible en el lado del cliente. Durante el proceso de inicio de sesión, el backend genera dos tipos de tokens diferenciados:

- Access Token, de corta duración, necesario para acceder a recursos protegidos.
- Refresh Token, con una validez mayor, que permite renovar el anterior sin que el usuario deba volver a introducir sus credenciales.

Este enfoque ofrece una capa adicional de protección frente a accesos no autorizados. El Access Token, al tener una validez limitada, reduce el margen de acción de posibles ataques en caso de interceptación. Por su parte, el Refresh Token actúa como mecanismo de renovación, permitiendo mantener la sesión activa de forma transparente para el usuario mientras se preserva la seguridad general del sistema.

En cada solicitud protegida, el cliente incluye el Access Token como prueba de identidad. Cuando caduca, se puede generar uno nuevo mediante el Refresh Token, sin que ello suponga una interrupción perceptible en la experiencia de uso. Así, se ofrecen sesiones continuas sin sacrificar buenas prácticas de seguridad.

5.2.2. Persistencia de tokens mediante cookies

Con el objetivo de reforzar la protección frente a vulnerabilidades típicas en entornos web, como el Cross-Site Scripting (XSS) [30], los tokens JWT generados no se exponen directamente al código del cliente. En

su lugar, se almacenan mediante cookies seguras de tipo HTTP-only, inaccesibles desde JavaScript, lo que impide su manipulación desde el frontend.

Además, estas cookies están configuradas con los atributos `Secure` y `SameSite='Strict'`. El primero garantiza su transmisión solo por conexiones HTTPS, mientras que el segundo limita su inclusión a solicitudes del mismo origen, mitigando riesgos asociados a ataques CSRF (Cross-Site Request Forgery).

Gracias a esta configuración, el usuario puede autenticarse una única vez y, mientras su sesión siga activa, realizar operaciones protegidas sin necesidad de gestionar tokens manualmente. Este modelo no solo simplifica el flujo de autenticación, sino que también mejora la experiencia general de navegación, manteniendo al mismo tiempo un alto nivel de protección en todos los intercambios entre cliente y servidor.

5.2.3. Inicio de sesión con Google y GitHub

Además del inicio de sesión convencional mediante correo y contraseña, el sistema incorpora autenticación mediante proveedores externos utilizando el protocolo OAuth 2.0 [31], permitiendo a los usuarios identificarse con sus cuentas de Google o GitHub. Esta opción agiliza significativamente el acceso a la plataforma, evitando la necesidad de crear nuevas credenciales o completar procesos de registro adicionales. El flujo completo de este proceso puede observarse en la Figura 22, donde se detallan las interacciones entre el cliente, el proveedor externo y el servidor durante la autenticación.

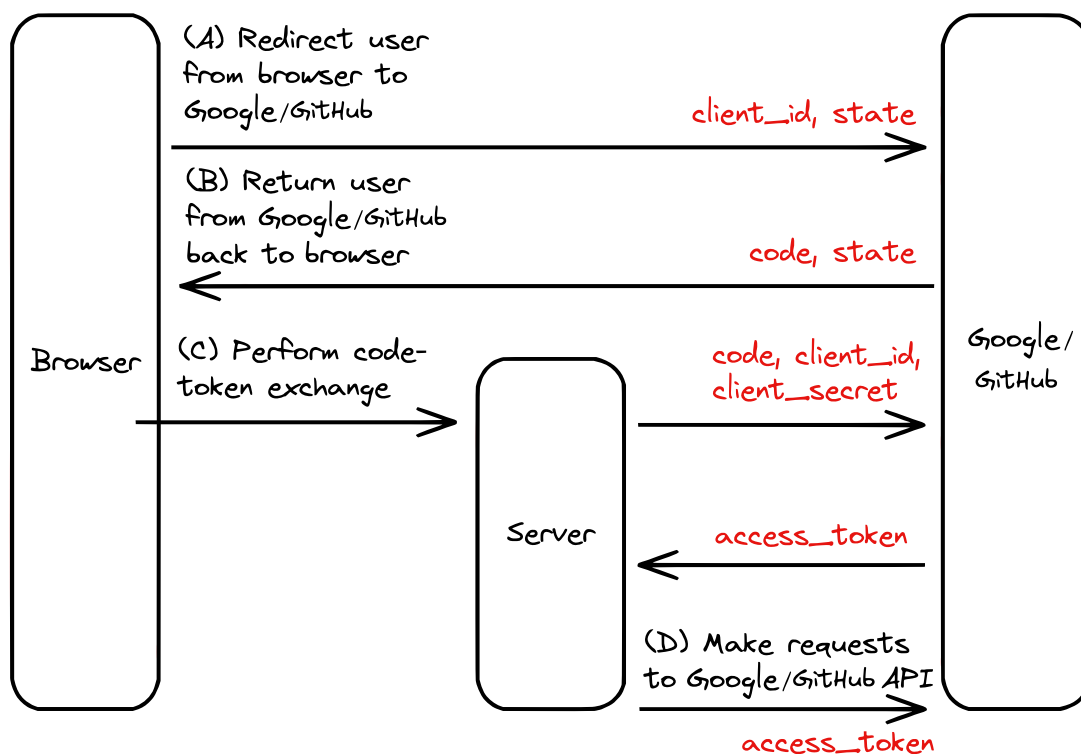


Figura 22: Flujo de autenticación con Google y GitHub

El procedimiento sigue el flujo habitual de OAuth: el usuario selecciona el proveedor deseado y es redirigido a su interfaz de autenticación. Tras conceder los permisos solicitados, el proveedor devuelve un token de identidad que es validado por el servidor. Si la validación resulta exitosa, se genera un par de tokens JWT (Access y Refresh) que permiten mantener la sesión activa y protegida, en consonancia con el sistema de autenticación principal.

Esta integración, además de aportar comodidad y velocidad, elimina pasos intermedios como la verificación por correo o la elección de contraseñas, lo que resulta especialmente útil para usuarios que desean comenzar a utilizar el sistema de inmediato. La habilitación de este tipo de autenticación ha requerido el registro previo del cliente en los paneles de desarrolladores de Google y GitHub, donde se han definido los entornos web autorizados, las rutas de redirección y las credenciales asociadas.

Gracias a esta funcionalidad, se ofrece una alternativa moderna y segura que simplifica el acceso sin comprometer los principios de seguridad y trazabilidad definidos para el resto del sistema.

5.2.4. Verificación de correo electrónico mediante código y protocolo SMTP

Como medida adicional de seguridad y fiabilidad, se ha incorporado un mecanismo de verificación de dirección de correo electrónico. Este proceso garantiza que las cuentas registradas estén asociadas a direcciones reales y accesibles, evitando el uso de correos ficticios o automatizados, y habilitando un canal de comunicación válido para futuras verificaciones.

Durante el registro inicial o en solicitudes de restablecimiento de contraseña, el sistema genera un código numérico aleatorio de seis cifras que es almacenado temporalmente en caché con una validez de diez minutos. Este código se envía por correo electrónico al usuario, quien debe introducirlo posteriormente en un campo habilitado específicamente para completar la validación.

El envío del mensaje se realiza a través del protocolo SMTP (Simple Mail Transfer Protocol) [32], estándar ampliamente adoptado para la transmisión de correos desde aplicaciones web. Esta integración permite establecer un flujo automatizado de confirmación que combina simplicidad de uso con garantías de autenticidad en el registro de nuevos usuarios.

El procedimiento puede resumirse en los siguientes pasos:

1. El usuario introduce una dirección de correo válida en el formulario correspondiente.
2. El servidor genera un código de verificación aleatorio.
3. El código se almacena temporalmente en el sistema, con expiración controlada.
4. Se envía el correo al destinatario con el código generado, como se puede observar en la Figura 23.

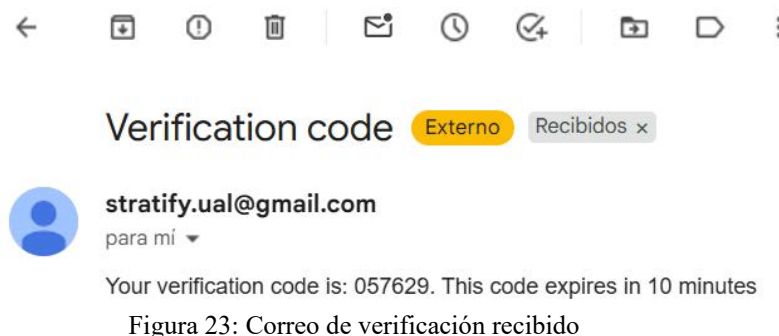


Figura 23: Correo de verificación recibido

5. El usuario introduce el código recibido en el formulario de verificación, como se puede ver en la Figura 24.

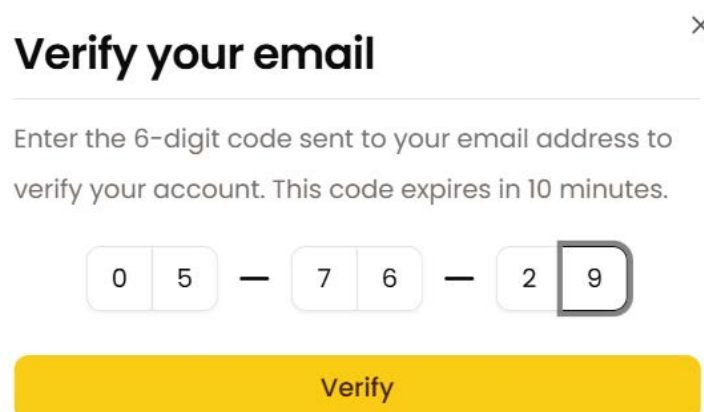


Figura 24: Entrada en la interfaz para verificación del código

6. El servidor compara el valor introducido con el almacenado en caché.
7. Si ambos coinciden y el código no ha expirado, la cuenta se considera verificada.



5.3. API REST: Endpoints

Ruta	Tipo	Requisitos	Descripción	Auth
validate-email/	POST	email	Envía un código de verificación temporal al correo electrónico.	No
recover-password/	POST	email	Envía un código temporal para cambiar la contraseña.	No
change-password/	POST	email, new_password, code	Cambia la contraseña si el código de verificación es correcto.	No
login/	POST	username, password	Autentica al usuario y devuelve tokens JWT en cookies.	No
logout/	POST	—	Cierra la sesión y elimina las cookies de autenticación.	Sí
google-login/	POST	Bearer token (Authorization header)	Inicia sesión mediante Google OAuth2.	No
github-login/	POST	code	Inicia sesión mediante GitHub OAuth2.	No
user/	POST	username, email, password, code	Registra una nueva cuenta tras verificar el correo.	No
user/	GET	—	Devuelve los datos del usuario autenticado si tiene sesión iniciada.	Sí
user/	PATCH	Campos a modificar + password + code	Actualiza el perfil del usuario autenticado tras verificar su correo.	Sí
user/	DELETE	password + code	Elimina la cuenta del usuario autenticado si el código es correcto.	Sí
user/send-verification-email/	POST	email, password	Envía un código de verificación para actualizar el correo.	Sí
user/send-verification-delete/	POST	password	Envía un código de verificación para eliminar la cuenta.	Sí
user/send-verification-signup/	POST	username, email, password	Envía un código de verificación para el registro.	No
user/send-verification-recover/	POST	email	Envía un código de verificación para recuperar la contraseña.	No



toggle-theme/	PATCH	–	Cambia entre modo claro y oscuro para el usuario autenticado.	Sí
update-timezone/	PATCH	timezone	Establece zona horaria preferida del usuario.	Sí
apikeys/	GET	–	Lista los exchanges asociados a claves API del usuario.	Sí
apikeys/	POST	exchange, api_key, secret (y opcionales)	Añade o actualiza una clave API para un exchange.	Sí
apikeys/<exchange>/	DELETE	exchange	Elimina la clave API asociada a un exchange.	Sí
exchanges/	GET	–	Devuelve los exchanges soportados con los métodos requeridos.	Sí
symbols/	GET	exchange	Devuelve los símbolos y intervalos de tiempo, disponibles para un exchange.	Sí
market-info/	GET	exchange, symbol	Devuelve información del mercado, como las comisiones	Sí
strategy/	GET	– (con filtros opcionales)	Lista estrategias públicas y del usuario.	Sí
strategy/	POST	–	Crea una nueva estrategia con nombre por defecto.	Sí
strategy/<id>/	GET	id	Devuelve una estrategia pública o propia por ID.	Sí
strategy/<id>/	PATCH	id, campos modificables	Modifica una estrategia propia.	Sí
strategy/<id>/	DELETE	id	Elimina una estrategia propia.	Sí
strategy/me/	GET	–	Lista todas las estrategias propias del usuario.	Sí
strategy/<id>/clone/	POST	execution_id (opcional)	Clona una estrategia pública, opcionalmente con su backtest.	Sí
candles/	GET	exchange, symbol, timeframe, timestamps	Devuelve datos de velas OHLCV desde el exchange o la base de datos.	Sí
indicator/	GET	strategy_id, indicator_id, timestamps	Devuelve los datos calculados para un indicador técnico específico.	Sí

Figura 25: Endpoints definidos en la API



5.4. Integración con librerías y servicios externos

El desarrollo de soluciones orientadas al análisis y ejecución automatizada en entornos de alta volatilidad, como el de los criptoactivos, requiere inevitablemente la interacción con componentes externos que aporten funcionalidad especializada y datos actualizados. En este contexto, se ha incorporado un conjunto de herramientas y servicios de terceros que permiten ampliar las capacidades del sistema más allá de lo que resultaría viable desarrollar localmente, tanto por su complejidad como por su coste de mantenimiento.

El empleo de estas librerías no solo ha facilitado la implementación de funcionalidades clave, sino que también ha contribuido a mantener la modularidad y escalabilidad del sistema. Gracias a esta estrategia de integración, ha sido posible acceder en tiempo real a información sobre mercados, operar sobre datos estructurados y simular comportamientos de trading bajo condiciones realistas. Asimismo, se ha evitado la sobrecarga innecesaria de procesamiento y almacenamiento local, delegando determinadas tareas a soluciones ya optimizadas y ampliamente contrastadas.

Las secciones siguientes describen diferenciadamente las principales integraciones, según el tipo de utilidad aportada al sistema: conexión con plataformas de intercambio, cálculo de métricas técnicas sobre series temporales y tratamiento avanzado de datos de mercado para simulación y ejecución automatizada.

5.4.1. Integración con plataformas de intercambio mediante CCXT

La integración con plataformas de intercambio de criptomonedas es un componente fundamental para el desarrollo de sistemas que requieran interacción directa con mercados financieros digitales. En este contexto, la biblioteca CCXT (CryptoCurrency eXchange Trading Library) se presenta como una solución robusta, eficiente y ampliamente adoptada para facilitar la conexión con múltiples exchanges mediante una única interfaz unificada.

CCXT es una biblioteca de código abierto desarrollada en JavaScript, Python y PHP, que permite acceder a más de 100 plataformas de intercambio diferentes, tales como Binance, Coinbase, Kraken, Kucoin, entre otras. La ventaja principal de utilizar CCXT radica en su capacidad para abstraer las complejidades propias de la API nativa de cada exchange, brindando un conjunto estandarizado de métodos para realizar operaciones comunes como consulta de balances, precios de mercado y consulta y ejecución de órdenes.

En la implementación de la integración mediante CCXT, el proceso inicia con la configuración de las credenciales API que cada plataforma requiere para la autenticación y autorización. Estas credenciales generalmente incluyen una clave pública, una clave secreta, pudiendo añadirse otros parámetros como contraseña o identificador de usuario en determinados casos. La correcta gestión y almacenamiento seguro de estas claves es crucial para evitar vulnerabilidades y accesos no autorizados.

Una vez completada la configuración de acceso, la creación de instancias de conexión a los exchanges se realiza a través de una clase de abstracción que encapsula el proceso de inicialización y validación de estado del servicio, ofreciendo así una capa intermedia que centraliza la lógica y facilita su mantenimiento.

```
class Exchange:
    def __new__(cls, exchange_name, user):
        try:
            api_key_instance = list(ApiKey.objects.filter(user=user, exchange=exchange_name).values('api_key', 'secret', 'password', 'uid'))
            exchange = getattr(ccxt, exchange_name)(api_key_instance[0] if api_key_instance else {})
            if exchange.has.get('fetchStatus'):
                status_response = exchange.fetch_status()
                if status_response.get('status') != 'ok':
                    raise Exception('Exchange service unavailable')
            exchange.load_markets()
            exchange.precisionMode = ccxt.TICK_SIZE
            exchange.roundingMode = ccxt.TRUNCATE
            exchange.name = exchange_name
            return exchange
        except Exception:
            raise Exception('Unable to initialize exchange')
```

Figura 26: Declaración de la clase Exchange

Este diseño permite, entre otras funcionalidades, verificar la disponibilidad del servicio antes de proceder con operaciones, cargar dinámicamente los mercados disponibles y configurar los modos de redondeo y precisión que deben utilizarse para los valores numéricos.

Una vez configurado el entorno, la biblioteca CCXT facilita la ejecución de operaciones según las necesidades del sistema. Por ejemplo, la consulta de los precios históricos de los activos se puede realizar mediante el método `fetch_ohlcv()`, mientras que la ejecución de órdenes de compra o venta se maneja con `create_order()`. Esta estandarización reduce significativamente el tiempo de desarrollo y minimiza los errores derivados de las diferencias en las especificaciones de cada API.

No obstante, la integración con múltiples exchanges exige atender a ciertas particularidades operativas. Una de las más relevantes es la gestión de los límites de tasa (rate limits), impuestos por los proveedores para prevenir abusos y garantizar la estabilidad del servicio. El incumplimiento de estas restricciones puede derivar en bloqueos temporales o respuestas erráticas por parte de la API.

Para mitigar este riesgo, se ha configurado la biblioteca CCXT con el parámetro `enableRateLimit` activado, lo cual permite a la propia biblioteca gestionar automáticamente los tiempos de espera entre peticiones, ajustándose así a las políticas específicas de cada plataforma. Además, se han añadido bloques de control de excepciones que permiten interceptar y registrar los errores derivados de estas restricciones, facilitando la implementación de reintentos controlados y asegurando una operativa resiliente.

5.4.2. Cálculo de indicadores técnicos a través de TA-Lib

El análisis técnico constituye uno de los pilares fundamentales en el diseño y evaluación de estrategias de trading automatizado. Para implementar este tipo de análisis, es necesario disponer de una herramienta que permita calcular de manera eficiente y precisa los indicadores más reconocidos en el ámbito financiero. En este proyecto, se ha utilizado la biblioteca TA-Lib (Technical Analysis Library), una solución ampliamente consolidada que ofrece más de 150 funciones matemáticas para el estudio de series temporales de precios, entre ellas osciladores, medias móviles, bandas de volatilidad y otros indicadores compuestos.

TA-Lib ha sido incorporada como dependencia del backend para permitir el procesamiento directo de los datos de mercado almacenados en la base de datos o recuperados desde los exchanges mediante CCXT. Su principal ventaja reside en el rendimiento computacional que ofrece, al estar implementada parcialmente en C y optimizada para manejar grandes volúmenes de datos con baja latencia. Esto resulta especialmente



relevante en escenarios donde se requiere realizar cálculos en tiempo real o aplicar múltiples indicadores simultáneamente sobre una misma serie histórica.

Para garantizar la fiabilidad de los resultados y mantener un enfoque práctico, se han seleccionado los 20 indicadores más empleados en entornos profesionales y académicos. Entre ellos se encuentran el RSI, las Bollinger Bands, el MACD, las medias móviles exponenciales y simples, el ADX y el CCI, entre otros. Esta selección responde tanto a su alta representatividad en estrategias reales como a la disponibilidad de documentación y casos de uso bien establecidos.

El cálculo de cada indicador se realiza de forma dinámica en función de la configuración definida por el usuario. A continuación, se muestra un ejemplo representativo correspondiente al cálculo del Relative Strength Index (RSI), que permite identificar zonas de sobrecompra y sobreventa a partir de los valores de cierre de las velas japonesas.

```
case 'RSI':
    if new_indicator:
        indicator['params'] = [
            {"key": "length", "value": 14},
            {"key": "upper_limit", "value": 70},
            {"key": "middle_limit", "value": 50},
            {"key": "lower_limit", "value": 30},
        ]
    length = int(next(p for p in indicator['params'] if p['key'] == 'length')['value'])
    candles_df = CandleView().get_candles(
        exchange=Exchange(exchange, user),
        symbol=symbol,
        timeframe=timeframe,
        timestamp_start=timestamp_start,
        timestamp_end=timestamp_end,
        db_search=True,
        extra_candles=2 * length
    )
    candles_df['rsi'] = ta.RSI(candles_df['close'].astype(float).values, time-
period=length)
```

Figura 27: Ejemplo de cálculo del indicador RSI con TA-Lib

El fragmento anterior muestra cómo se extraen los parámetros de configuración definidos por el usuario, se recuperan las velas necesarias y se aplica el cálculo del RSI mediante la función `ta.RSI()`. La obtención de datos se realiza con un margen adicional proporcional al periodo, lo que garantiza la precisión de los valores iniciales del indicador.

Esta lógica se replica para el resto de indicadores, encapsulando la ejecución en bloques independientes que permiten añadir o modificar cálculos sin afectar al resto de la estructura. Esta modularidad ha sido clave para mantener el código limpio y fácilmente extensible. Asimismo, se han implementado mecanismos de validación para asegurar que los parámetros definidos se encuentran dentro de rangos razonables, evitando resultados erróneos o distorsionados.

Gracias a esta integración, el sistema es capaz de enriquecer visual y analíticamente los gráficos de precios, facilitando la toma de decisiones tanto en entornos de simulación como en operaciones reales.



5.4.3. Algoritmo de procesamiento de velas para simulación y trading automático

La ejecución automatizada de estrategias de inversión requiere no solo la definición de reglas operativas, sino también la implementación de un sistema capaz de interpretar esas reglas en función de los datos del mercado en tiempo real o histórico. Con ese propósito, se ha desarrollado un algoritmo de procesamiento de velas que permite ejecutar estrategias tanto en modo de simulación como en entornos reales de trading.

Este algoritmo opera sobre estructuras de datos que representan series temporales de velas japonesas, previamente obtenidas mediante integración con exchanges. A partir de ellas se calculan indicadores técnicos definidos por el usuario, y se mantienen variables de estado para representar la posición actual, el valor realizado y no realizado, el capital disponible, así como métricas de rendimiento y riesgo acumuladas.

La lógica de ejecución se basa en una evaluación secuencial de condiciones de mercado, expresadas mediante operadores comparativos y lógicos. Estas condiciones se aplican sobre columnas derivadas de los precios e indicadores previamente calculados. En caso de cumplirse los criterios definidos, se ejecutan órdenes de compra, venta o cancelación, adaptadas según el tipo de orden (mercado o límite), el volumen, el apalancamiento y las comisiones establecidas.

En modo simulado, el sistema recorre las velas almacenadas y emula la evolución del portafolio evaluando condiciones y aplicando operaciones en cada instante. Las órdenes tipo límite se procesan solo si el precio de mercado toca el valor establecido, respetando el comportamiento realista del mercado. En trading en tiempo real, la ejecución se realiza en un hilo independiente que permanece activo mientras la estrategia esté marcada como “en ejecución”. Este hilo gestiona la espera entre intervalos, consulta el estado de órdenes abiertas y actualiza los datos con nuevas velas a medida que se publican.

Durante el proceso, se monitoriza el valor total del portafolio, incluyendo posiciones abiertas, órdenes pendientes y liquidez disponible. Esto permite calcular métricas como el beneficio neto, la tasa de operaciones ganadoras, el factor de beneficio, la ganancia media por operación y medidas de serie de ganancias y serie de pérdidas, tanto en términos absolutos como relativos. Dichas métricas son persistidas al finalizar la ejecución, proporcionando una base cuantitativa para evaluar el rendimiento de cada estrategia.

El sistema también incorpora mecanismos de validación y control de errores, tanto para la evaluación de condiciones como para la ejecución de órdenes. Esto incluye comprobaciones de sintaxis, control de paréntesis y detección de inconsistencias en las expresiones lógicas que definen las condiciones de entrada y salida. En caso de fallo, se registra el error y se interrumpe la ejecución de forma segura, evitando comportamientos no deseados.

Finalmente, todas las operaciones ejecutadas, reales o simuladas, se almacenan en una tabla de transacciones que permite su posterior análisis. Esta estructura modular y asíncrona asegura que la lógica de ejecución pueda ser reutilizada en múltiples escenarios y adaptada a diferentes estilos de trading, desde sistemas conservadores de reversión a la media hasta estrategias agresivas de seguimiento de tendencia.



6. Desarrollo del cliente (frontend)

Una vez desarrollada la lógica del sistema en el servidor y garantizada la integridad de la API, se abordó la implementación del cliente web, encargado de ofrecer una interfaz funcional que sirva de puente entre el usuario y las capacidades técnicas de la aplicación. Esta capa representa el punto de entrada principal para el usuario final, permitiendo gestionar estrategias de inversión, visualizar datos del mercado y controlar en tiempo real la ejecución automatizada.

La interfaz ha sido diseñada para ofrecer una experiencia fluida, clara y adaptable a distintos dispositivos, manteniendo la coherencia visual y funcional sin importar el entorno de acceso. En este apartado se analiza la estructura general del cliente, las tecnologías elegidas para su construcción y las decisiones clave que han condicionado su diseño e implementación.

Se describen también los mecanismos de navegación y gestión de estado, la interacción con la API REST expuesta por el backend, y los recursos utilizados para representar de forma dinámica gráficos, indicadores técnicos y resultados operativos. Se hace especial hincapié en la organización modular de los componentes, la utilización de herramientas modernas de desarrollo frontend, y la implementación de prácticas que favorecen la escalabilidad y mantenibilidad del código.

El resultado es una aplicación web capaz de traducir procesos complejos de análisis y ejecución de estrategias algorítmicas en un entorno accesible y visualmente comprensible para el usuario.

6.1. Arquitectura general del frontend

Antes de profundizar en la arquitectura y funcionalidad del cliente web, resulta fundamental comprender sobre qué tecnologías se apoya su construcción, qué herramientas lo sustentan y cuáles son los conceptos esenciales que permiten su correcta ejecución en un entorno de desarrollo moderno. En este apartado se analizan los pilares tecnológicos que sustentan la interfaz, con especial atención a la configuración inicial y a los mecanismos que facilitan la instalación y gestión de dependencias.

6.1.1. Node.js como entorno base para React

Uno de los elementos clave en el desarrollo frontend ha sido el uso de Node.js [33] como entorno de ejecución. Esta tecnología permite ejecutar código JavaScript fuera del navegador y se ha consolidado como base en numerosos proyectos web, no solo por su versatilidad, sino también por su integración con herramientas de construcción modernas. En este contexto, Node.js actúa como motor que permite el uso de frameworks como React, facilitando el arranque del proyecto y la ejecución de scripts automatizados.

Junto a Node.js, el gestor de paquetes npm ha desempeñado un papel central. Esta herramienta permite instalar y mantener un ecosistema de librerías externas, que incluyen desde el propio React hasta sistemas de rutas, gestión de estado y componentes visuales. Gracias a npm, la aplicación cuenta con un entorno modular, actualizado y fácilmente reproducible, adaptado a los estándares actuales de desarrollo web.

Dentro de esta estructura, el archivo `package.json` constituye el núcleo del proyecto. Su función principal es declarar las dependencias necesarias, especificar versiones concretas para garantizar la estabilidad del sistema y definir scripts personalizados que automatizan tareas como el despliegue, la compilación o la limpieza de archivos temporales. En este trabajo, este archivo ha permitido establecer una base coherente y controlada para todas las operaciones relacionadas con el cliente, asegurando que cada fase del desarrollo pueda repetirse sin inconsistencias.

A continuación, en la Figura 28 se muestra un fragmento representativo de dicho archivo, donde se recogen las configuraciones globales y los scripts de ejecución.

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  ...
}
```

Figura 28: Sección de configuración y arranque del package.json

6.1.2. Configuración del entorno: Vite y TypeScript

La elección del entorno de desarrollo ha desempeñado un papel clave en la eficiencia y organización del cliente web. En este proyecto se ha optado por Vite como herramienta de construcción y empaquetado, en combinación con TypeScript como lenguaje de programación tipado sobre JavaScript. Esta decisión responde a la necesidad de contar con un flujo de trabajo ágil, estructurado y fácilmente escalable, manteniendo un equilibrio entre rendimiento en tiempo de desarrollo y robustez en tiempo de ejecución.

Vite es una herramienta moderna que ha ganado popularidad en el ecosistema JavaScript por su rapidez en la recarga en caliente, su configuración mínima y su integración nativa con módulos ECMAScript. A diferencia de otros bundlers tradicionales como Webpack, Vite realiza la carga de módulos de forma progresiva y directa desde el navegador durante el desarrollo, lo que reduce significativamente los tiempos de arranque y mejora la experiencia del desarrollador. Además, su sistema de plugins permite extender funcionalidad con facilidad, ofreciendo compatibilidad con linters, transformadores de código y análisis estático.

La estructura inicial del proyecto se genera con una plantilla adaptada al uso conjunto de React y TypeScript. Esto permite desde el inicio contar con un sistema de tipos estáticos que aporta mayor seguridad en la codificación y facilita la detección temprana de errores. Gracias a TypeScript, es posible definir interfaces para las propiedades de los componentes, tipar correctamente el estado global y garantizar que las funciones se invocan con los parámetros adecuados. Esta capa adicional de control resulta especialmente útil en proyectos con múltiples módulos y flujos de datos, como ocurre en el presente trabajo.

La configuración de Vite, alojada en el archivo `vite.config.ts`, ha sido ajustada para incorporar funcionalidades específicas del entorno de desarrollo, como la resolución de rutas personalizadas, el uso de alias para evitar referencias relativas extensas y la integración con bibliotecas de componentes. Asimismo, se ha habilitado el soporte para hojas de estilo con Tailwind CSS [34], permitiendo definir estilos de forma declarativa y altamente reutilizable.

En conjunto, la combinación de Vite y TypeScript proporciona una base sólida para el desarrollo del cliente web. Mientras Vite garantiza velocidad y simplicidad en la construcción, TypeScript aporta claridad, prevención de errores y mejores capacidades de documentación interna. Esta configuración se adapta plenamente a las exigencias de una aplicación que debe mantener su estabilidad en un entorno dinámico y orientado al usuario final.

6.1.3. Estructura de componentes con Shadcn/ui y Tailwind CSS

El diseño visual de la aplicación web no se limita a criterios estéticos, sino que está estrechamente ligado a la experiencia de usuario, la accesibilidad y la organización del código. Para garantizar una interfaz coherente, personalizable y fácil de mantener, se ha optado por una estrategia basada en el uso combinado de Shadcn/ui como librería de componentes y Tailwind CSS como sistema de estilos utilitario.

Shadcn/ui es una colección moderna de componentes de interfaz diseñados con accesibilidad y personalización en mente, desarrollados sobre Radix UI y estilizados mediante Tailwind. Su principal ventaja radica en que no actúa como una librería opaca y cerrada, sino como una fuente de componentes cuyo código se integra directamente en el proyecto, permitiendo una modificación precisa sin renunciar a buenas prácticas. Esta aproximación facilita un control total sobre los estilos, el comportamiento y la estructura de los elementos visuales, lo que ha permitido adaptar cada componente a las necesidades específicas del sistema, tanto en términos funcionales como de diseño.

La organización del frontend parte de una estructura clara en la carpeta `src/components/ui`, donde se alojan los bloques reutilizables, como botones, formularios, tarjetas, menús desplegables o modales. Cada uno de estos elementos ha sido importado progresivamente mediante la CLI de Shadcn, lo que ha permitido mantener una carga controlada y evitar la inclusión de código innecesario. La integración con React se ha realizado siguiendo una lógica de componentes funcionales, lo que favorece la reutilización y la composición modular de vistas más complejas.

Complementariamente, se ha empleado Tailwind CSS como motor de estilos. A diferencia de los enfoques clásicos basados en hojas de estilo globales, Tailwind propone una metodología utilitaria que permite aplicar estilos directamente en la clase de cada elemento HTML. Esta técnica facilita una lectura inmediata del diseño visual desde el propio JSX, reduce la necesidad de definiciones externas y promueve una coherencia estilística que se mantiene a lo largo de toda la aplicación. Además, al estar completamente tipado y optimizado en tiempo de compilación, Tailwind contribuye a minimizar el tamaño del CSS final y acelera los tiempos de carga en producción.

El sistema de personalización ha sido ampliado mediante la redefinición de variables CSS en el archivo de configuración global. Estas variables controlan aspectos como los colores del tema, los radios de borde, las tipografías y los estilos oscuros, asegurando así una consistencia cromática y visual incluso en modos de visualización alternativos. Se han definido variantes personalizadas tanto para temas claros como oscuros, aplicando adaptaciones automáticas en función del esquema de preferencias del usuario o de la configuración del sistema operativo.

```
import path from "path"
import tailwindcss from "@tailwindcss/vite"
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
})
```

Figura 29: Configuración de Vite con Tailwind y alias de rutas en vite.config.ts

Para evitar problemas derivados de rutas relativas extensas, se ha configurado un sistema de alias en `vite.config.ts` y en los archivos de configuración de TypeScript, permitiendo importar componentes



mediante prefijos lógicos como `@/components/ui`, en lugar de rutas relativas profundas, como se aprecia en la Figura 29. Esta decisión mejora la claridad del código, reduce errores comunes en refactorizaciones y favorece la escalabilidad del proyecto a medio plazo.

En términos funcionales, el uso conjunto de Shadcn/ui y Tailwind ha permitido construir una interfaz intuitiva, limpia y reactiva, optimizada tanto para escritorios como para dispositivos móviles. Cada elemento visual responde de forma inmediata a las acciones del usuario, respetando las pautas de accesibilidad y ofreciendo retroalimentación visual coherente. Este enfoque garantiza no solo una buena apariencia, sino también una interacción fluida y robusta.

6.1.4. Integración de gráfico de velas japonesas con Lightweight Charts

El sistema incorpora una herramienta de visualización avanzada basada en gráficos de velas japonesas, diseñada para ofrecer al usuario una lectura clara y detallada del comportamiento del mercado. Este tipo de representación, común en el análisis técnico financiero, permite observar con precisión la evolución del precio de un activo en distintos periodos, mostrando de forma compacta la información relativa a apertura, cierre, máximos y mínimos.

Para llevar a cabo esta visualización se ha empleado Lightweight Charts, una librería de gráficos financieros desarrollada por TradingView. Esta solución destaca por ofrecer representaciones limpias, reactivas y altamente optimizadas para la web, lo que la convierte en una opción ideal para aplicaciones donde la claridad visual y la eficiencia son prioritarias. Su enfoque minimalista permite centrarse en los datos sin elementos superfluos, favoreciendo una experiencia de usuario fluida y enfocada en el análisis.

Desde la perspectiva del usuario, este gráfico cumple una doble función. Por un lado, permite comprender de un vistazo cómo se han comportado los precios durante una ventana temporal determinada, lo que resulta clave para contextualizar las decisiones automatizadas de la estrategia. Por otro, sirve como soporte visual sobre el que se superponen elementos complementarios como los indicadores técnicos o las operaciones realizadas, facilitando así un análisis integrado.

Cada vez que el usuario selecciona una estrategia específica, el gráfico se actualiza para mostrar la evolución del precio en el periodo correspondiente, junto con los puntos de entrada y salida de las operaciones ejecutadas. Estas operaciones aparecen marcadas directamente sobre el gráfico mediante iconos visuales y etiquetas que indican el tipo de orden (compra o venta) y su volumen relativo. Esta representación permite evaluar fácilmente la eficacia de cada decisión tomada por la estrategia, en relación con el contexto del mercado en ese momento.

Además, el sistema permite al usuario añadir indicadores técnicos personalizados, como medias móviles, osciladores o bandas de volatilidad. Cada indicador se representa en el gráfico por separado, ya sea sobre el precio o en paneles adicionales. Esto le permite adaptar la visualización a sus preferencias, activando solo los elementos que considere relevantes para su análisis.

La interacción con el gráfico es completamente dinámica. Al mover el cursor, el sistema muestra en tiempo real los valores exactos de las velas y los indicadores en ese instante, lo que facilita un análisis detallado sin necesidad de cálculos adicionales. También es posible reorganizar los distintos paneles para dar mayor protagonismo a ciertos indicadores, u ocultarlos temporalmente para simplificar la vista.

Este enfoque visual mejora significativamente la comprensión de los resultados y el comportamiento de las estrategias, especialmente para aquellos usuarios que no disponen de experiencia previa en análisis cuantitativo. Al transformar datos complejos en elementos gráficos intuitivos, el sistema hace accesible el análisis de rendimiento y ayuda a identificar patrones, anomalías o momentos clave de cada ejecución.

En definitiva, la integración del gráfico de velas japonesas aporta una capa esencial de visibilidad al funcionamiento interno del sistema. Su utilidad no radica únicamente en mostrar datos, sino en ofrecer una representación interactiva que conecta al usuario con la lógica de decisión que subyace a cada estrategia. Esta conexión entre acción, contexto y resultado es clave para generar confianza en el sistema y favorecer una toma de decisiones informada por parte del usuario.

6.2. Gestión de rutas y navegación en React

La navegación dentro de la aplicación se ha estructurado utilizando la biblioteca `react-router-dom`, una solución consolidada en el ecosistema de React que permite definir rutas de forma declarativa y flexible. Esta herramienta permite asociar cada ruta a un componente concreto, facilitando la construcción de una estructura lógica y coherente para el acceso a las distintas vistas del sistema.

Una de las ventajas de este enfoque es la capacidad de controlar el flujo de navegación según el estado del usuario. En este proyecto, se han implementado restricciones de acceso para determinadas rutas, como el portal principal o el panel de edición de estrategias, de forma que solo los usuarios autenticados puedan acceder a ellas. En caso contrario, se redirige automáticamente a la pantalla de inicio de sesión, garantizando así la integridad del flujo de uso.

El sistema también contempla redirecciones condicionadas. Por ejemplo, si un usuario ya autenticado intenta acceder al formulario de registro o de inicio de sesión, se le redirige directamente al portal, evitando rutas redundantes o incoherentes dentro de la experiencia de uso. Esta lógica se integra de forma clara en la estructura de rutas, lo que permite mantener una navegación fluida y controlada.

Como se muestra en la Figura 30, la definición de rutas contempla todas las posibles vistas de la aplicación, incluyendo accesos públicos, rutas protegidas y una redirección final para cualquier ruta no reconocida, que conduce de vuelta a la pantalla de inicio. Esta configuración contribuye a una navegación robusta, intuitiva y alineada con el comportamiento esperado por el usuario.

```
<Routes>
  <Route path="/api/*" />
  <Route path="/home" element={<Home />} />
  <Route path="/recover-password" element={<RecoverPassword />} />
  <Route path="/login" element={user ? <Navigate to="/portal" /> : <Login />} />
  <Route path="/signup" element={user ? <Navigate to="/portal" /> : <Signup />} />
  <Route path="/portal" element={user ? <Portal /> : <Navigate to="/login" />} />
  <Route path="/strategy/:id" element={user ? <Strategy /> : <Navigate to="/login" />} />
  <Route path="*" element={<Navigate to="/home" />} />
</Routes>
```

Figura 30: Gestión de rutas con `react-router-dom` en `App.tsx`

6.3. Autenticación y manejo de cookies en el navegador

La autenticación en la aplicación ha sido diseñada para ofrecer una experiencia de uso continua, evitando que el usuario tenga que iniciar sesión repetidamente en cada interacción. Para ello, se emplea un sistema de verificación basado en sesiones persistentes que aprovechan el uso de cookies seguras. Una vez el usuario proporciona sus credenciales, el servidor responde con la información necesaria para mantener su estado autenticado, la cual queda almacenada de forma transparente en el navegador.

A lo largo de toda la navegación, el cliente se comunica con el servidor mediante peticiones HTTP autenticadas. Para facilitar esta interacción, se ha configurado una instancia personalizada de `axios` que centraliza todas las llamadas a la API. Esta configuración incluye el parámetro `withCredentials`, lo que permite que las cookies HTTP-only se incluyan automáticamente en cada petición, sin necesidad de intervención explícita por parte del desarrollador en cada llamada individual. En la Figura 31 se muestra el fragmento del código que configura las llamadas mediante `axios`.

```
const api = axios.create({
  baseURL: URL,
  withCredentials: true,
});
```

Figura 31: Instancia de Axios con cookies incluidas en cada petición.

Gracias a esta estrategia, la verificación del estado de sesión se realiza de forma silenciosa y continua. Cada vez que el usuario accede a una sección protegida de la aplicación, como el portal principal o el panel de estrategias, el sistema consulta de forma interna si la sesión sigue siendo válida. En caso de que no lo sea, el usuario es redirigido al inicio de sesión sin interrupciones visibles. Esta comprobación se lleva a cabo a través de un endpoint específico, asegurando que los tokens siguen siendo válidos antes de permitir el acceso a contenido sensible.

Este enfoque permite mantener la sesión activa durante periodos prolongados sin afectar a la seguridad ni a la fluidez del uso. Mientras no se cierre la sesión de forma manual, el usuario puede navegar por todas las funcionalidades del sistema sin interrupciones ni pasos adicionales, mejorando notablemente la experiencia general en comparación con sistemas tradicionales de autenticación basados en almacenamiento local.

6.4. Integración con la API REST del backend

Para centralizar la comunicación entre el cliente y el servidor, todas las llamadas a la API se han organizado en un único archivo llamado `api.ts`. Este actúa como punto de acceso común a la lógica de red, permitiendo encapsular detalles como rutas, métodos HTTP y manejo de errores. Por ejemplo, la obtención del usuario autenticado se realiza mediante una función reutilizable que oculta la complejidad de la petición.

Además, se ha configurado un interceptor global con Axios para capturar errores de red o autenticación, tal como se muestra en la Figura 32. Esto permite mostrar notificaciones automáticas o redirigir al usuario sin necesidad de añadir lógica repetida en cada llamada.

```
// Interceptor to handle errors globally
api.interceptors.response.use(
  response => response,
  error => {
    if (axios.isCancel(error)) return Promise.resolve();
    if (error.response) {
      console.error("API Error:", error.response.data);
      if (error.response.status === 401) {
        window.location.href = "/login";
      }
    } else {
      console.error("Network Error:", error.message);
    }
    return Promise.reject(error);
  }
);

// Get Authenticated User
export const getAuthUser = () => api.get<User>("user/me/");
```

Figura 32: Gestión global de errores y llamada a la API

6.5. Interfaz de usuario

Este apartado recoge una visión general de las principales pantallas que conforman la aplicación, junto con ejemplos de interacción que reflejan distintas funcionalidades integradas. A través de estas representaciones, se busca mostrar cómo se estructura visualmente el sistema y cómo se lleva a cabo el flujo de acciones habituales dentro de la plataforma, ofreciendo así una perspectiva clara del comportamiento de la interfaz en situaciones reales de uso.

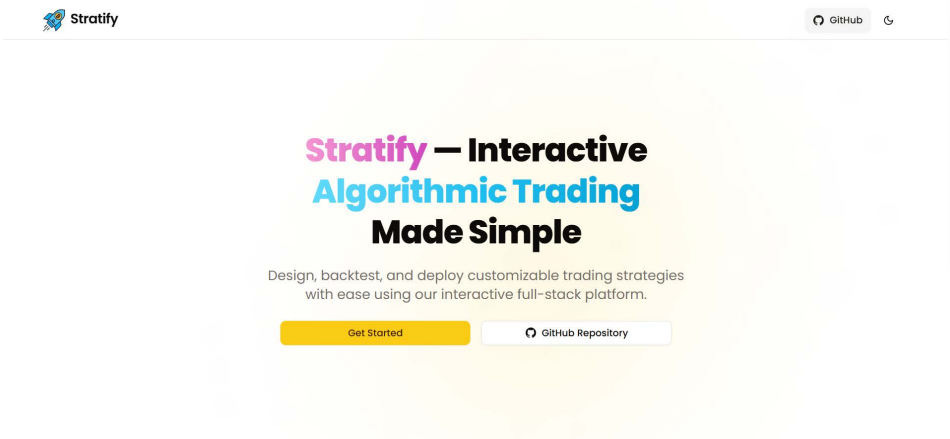
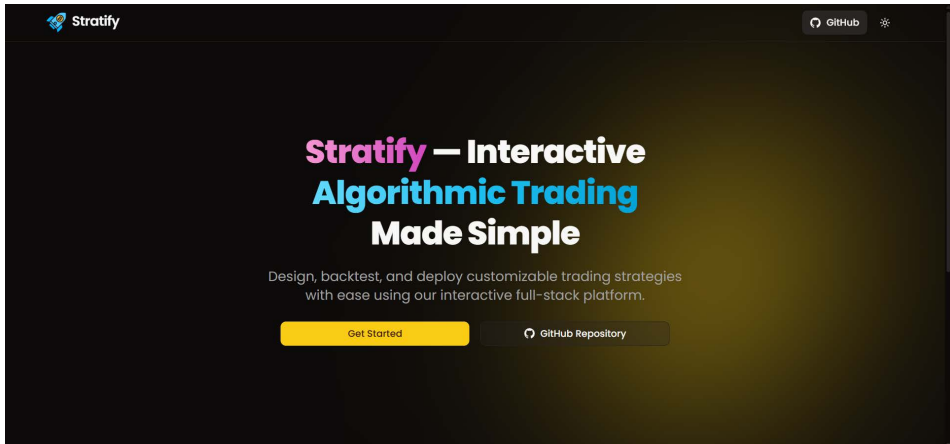
Historia de usuario	HU-001: Página principal y tema oscuro
Actores	ACT-001, ACT-002
Característica / Funcionalidad	Visualizar la página principal de la aplicación, acceder a funcionalidades básicas y alternar entre tema claro y oscuro.
Escenarios	1. Como usuario no autenticado, quiero ver la página principal con información general de la plataforma. 2. Como cualquier usuario, quiero cambiar el tema visual de la interfaz a modo oscuro para una mejor experiencia desde el inicio.
Resultados esperados	1.  Figura 33: Página principal para usuario no autenticado 2.  Figura 34: Interfaz en modo oscuro

Tabla 33: HU-001, Página principal y tema oscuro



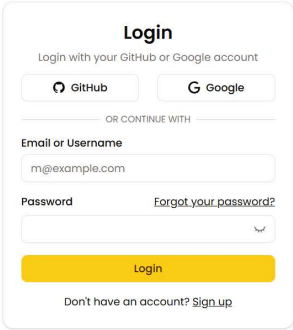
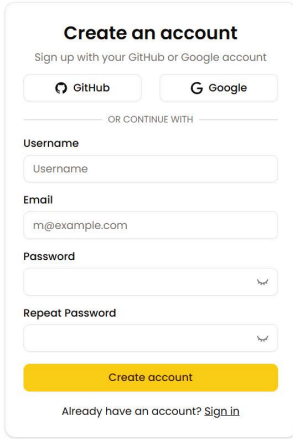
Historia de usuario	HU-002: Identificación en la plataforma
Actores	ACT-001
Característica / Funcionalidad	Acceder a la plataforma mediante los formularios de inicio de sesión o registro, permitiendo autenticar al usuario y habilitar su perfil.
Escenarios	1. Como usuario no autenticado, quiero acceder al formulario de registro para crear una cuenta. 2. Como usuario autenticado, quiero iniciar sesión para acceder a las funcionalidades personalizadas de la plataforma.
Resultados esperados	1.  Figura 35: Formulario de registro 2.  Figura 36: Formulario de inicio de sesión

Tabla 34: HU-002, Identificación en la plataforma



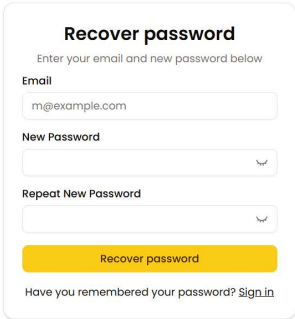
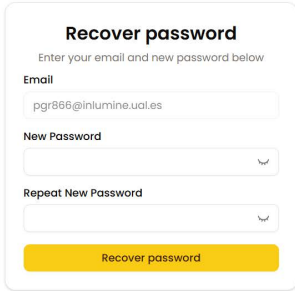
Historia de usuario	HU-003: Recuperación de contraseña
Actores	ACT-001, ACT-002
Característica / Funcionalidad	Recuperar el acceso a la cuenta a través de un formulario que permite solicitar el restablecimiento de contraseña, adaptado al estado de autenticación del usuario.
Escenarios	1. Como usuario no autenticado, quiero introducir mi correo electrónico para solicitar la recuperación de mi contraseña. 2. Como usuario autenticado, quiero acceder al formulario de recuperación con mi correo ya rellenado y bloqueado para evitar cambios.
Resultados esperados	1.  Figura 37: Recuperación de contraseña para usuario no autenticado 2.  Figura 38: Recuperación de contraseña con correo prellenado

Tabla 35: HU-003, Recuperación de contraseña



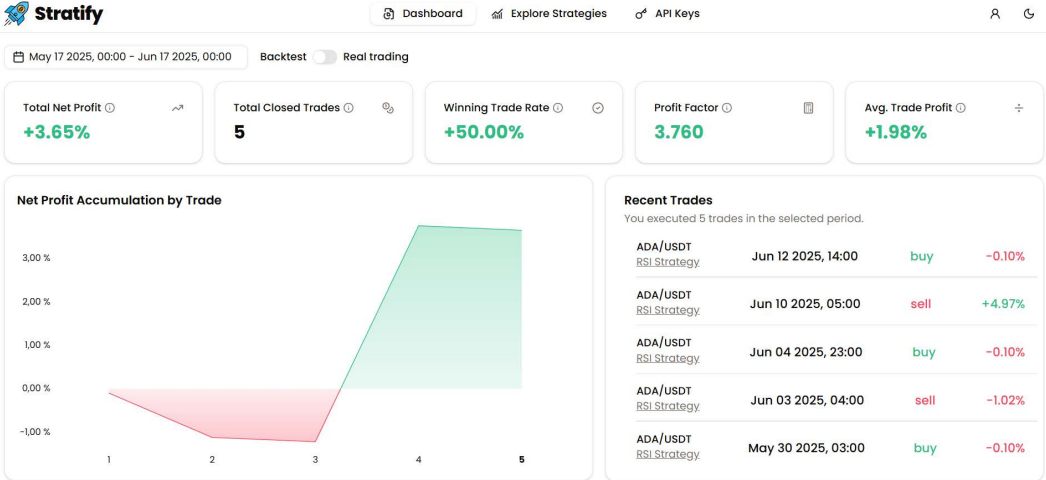
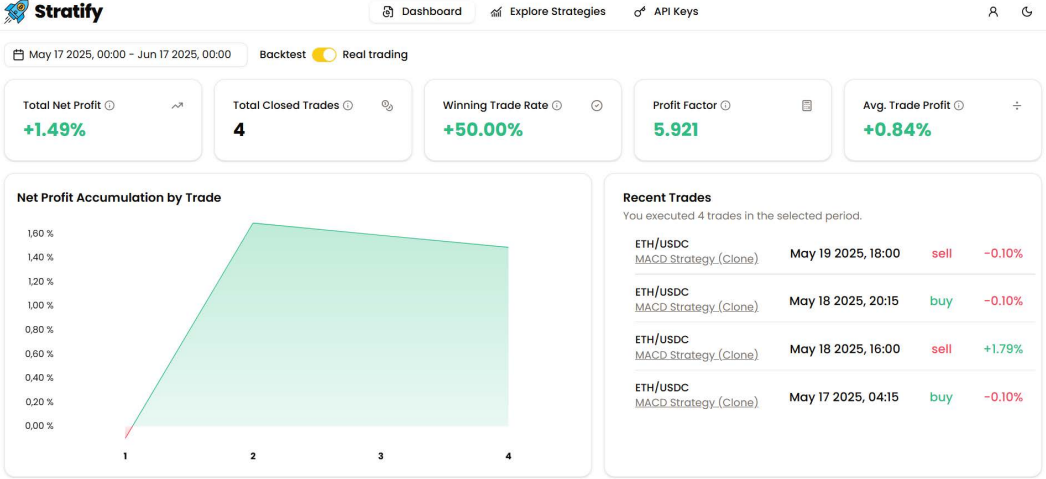
Historia de usuario	HU-004: Portal de métricas globales
Actores	ACT-002
Característica / Funcionalidad	Visualizar en el portal principal un resumen global con métricas de rendimiento, gráfico de evolución y operaciones recientes según el tipo de ejecución seleccionada (simulada o real).
Escenarios	1. Como usuario autenticado, quiero consultar el portal con datos filtrados por ejecuciones en modo simulado (backtest). 2. Como usuario autenticado, quiero consultar el portal con datos filtrados por ejecuciones reales.
Resultados esperados	1.  Figura 39: Dashboard con métricas en modo simulado 2.  Figura 40: Dashboard con métricas en modo real

Tabla 36: HU-004, Portal de métricas globales



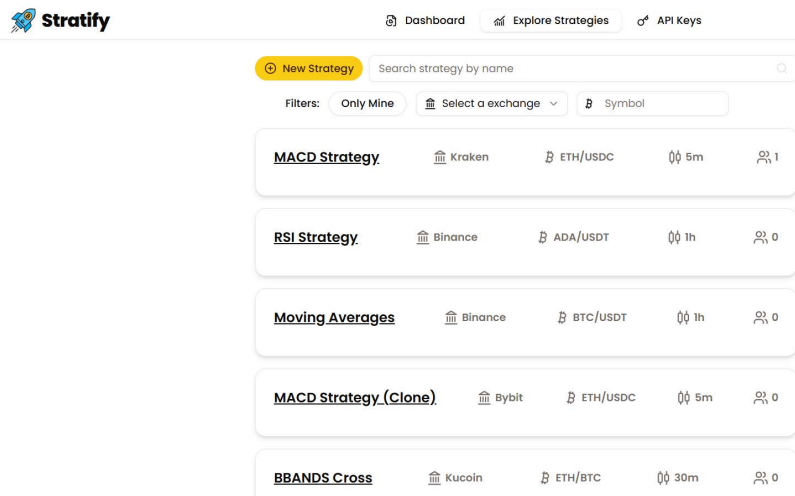
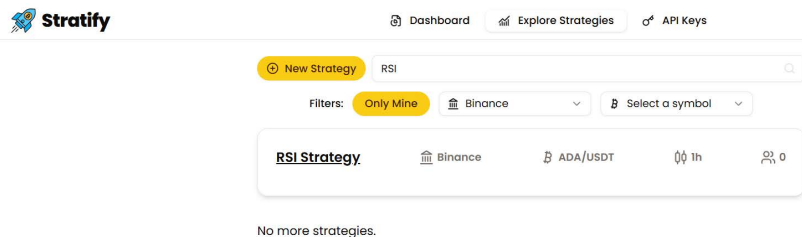
Historia de usuario	HU-005: Exploración de estrategias
Actores	ACT-002
Característica / Funcionalidad	Explorar estrategias disponibles en la plataforma mediante un sistema de búsqueda y filtrado por nombre, tipo de visibilidad y exchange, permitiendo identificar y acceder a estrategias relevantes.
Escenarios	1. Como usuario autenticado, quiero ver todas las estrategias públicas ordenadas por número de clonaciones sin aplicar ningún filtro. 2. Como usuario autenticado, quiero aplicar filtros para visualizar únicamente mis estrategias, seleccionar un exchange o par específico y buscar por nombre en la barra de búsqueda.
Resultados esperados	1.  Figura 41: Listado de estrategias públicas 2.  Figura 42: Exploración con filtros aplicados

Tabla 37: HU-005, Exploración de estrategias

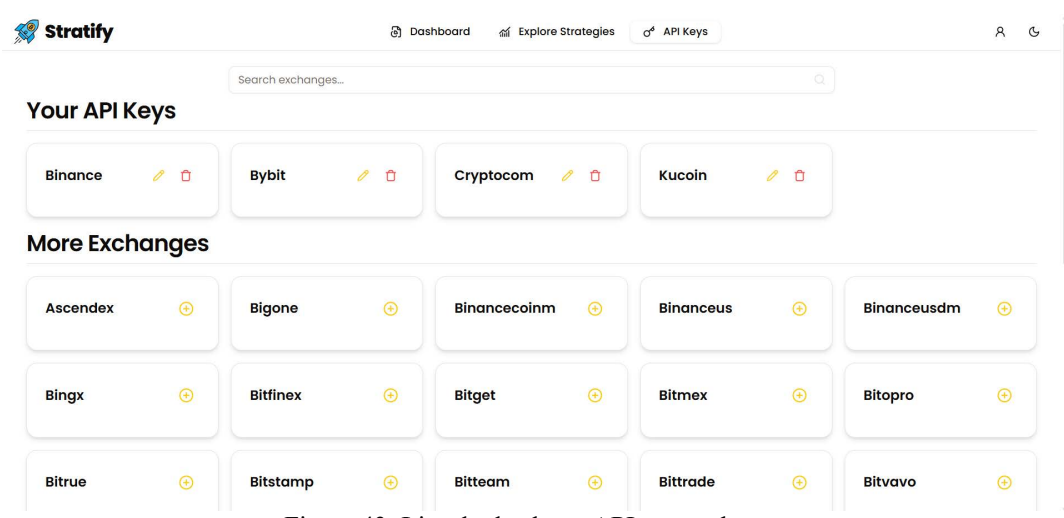
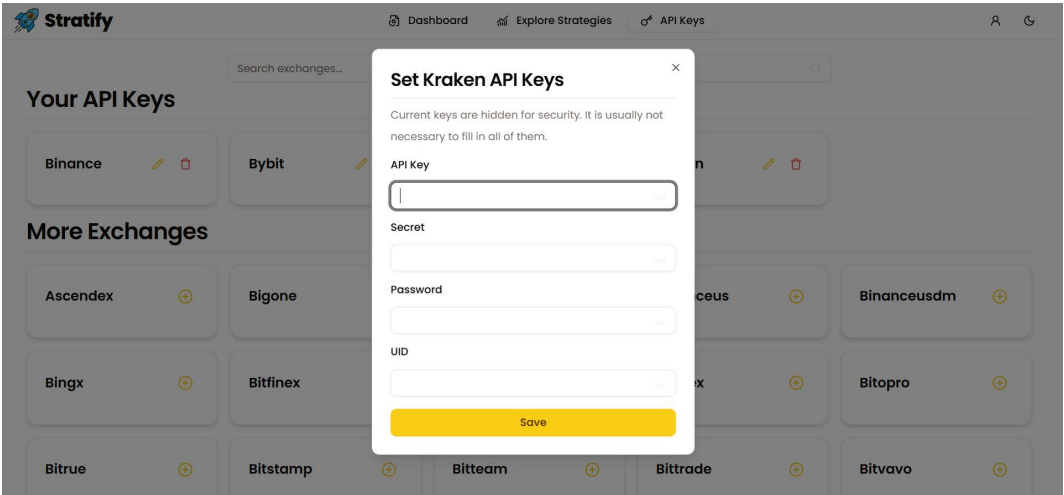
Historia de usuario	HU-006: Gestión de claves API
Actores	ACT-002
Característica / Funcionalidad	Gestionar las claves API asociadas a diferentes exchanges, permitiendo al usuario añadir, modificar o eliminar claves para habilitar la ejecución de estrategias automáticas, además de una búsqueda de exchange por nombre.
Escenarios	1. Como usuario autenticado, quiero ver un listado de los exchanges donde ya he configurado claves API y aquellos en los que aún no lo he hecho, con opciones para modificar o eliminar las existentes. 2. Como usuario autenticado, quiero acceder a un formulario para añadir o modificar un conjunto de claves API de un exchange específico.
Resultados esperados	1.  Figura 43: Listado de claves API por exchange 2.  Figura 44: Formulario de gestión de clave API

Tabla 38: HU-006, Gestión de claves API



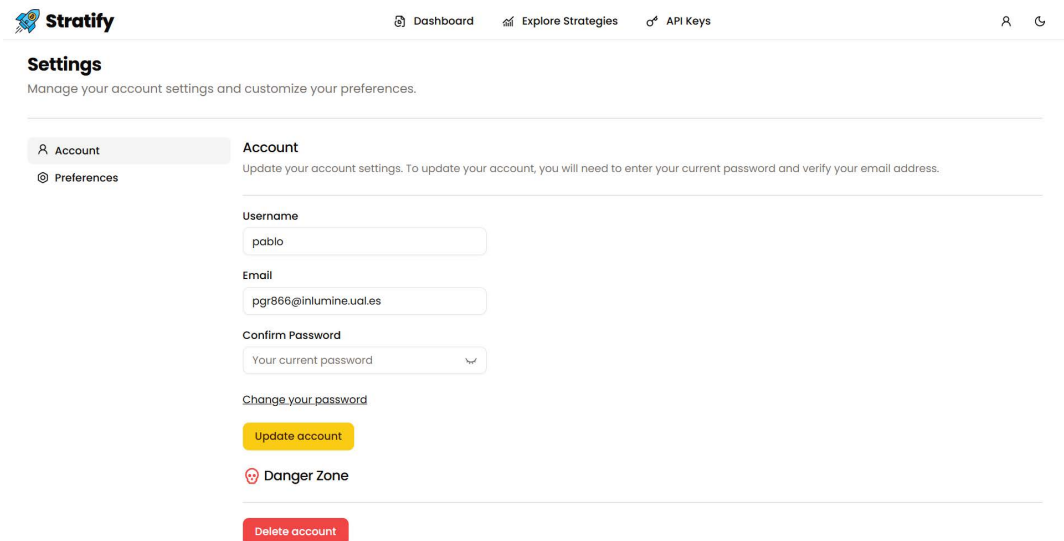
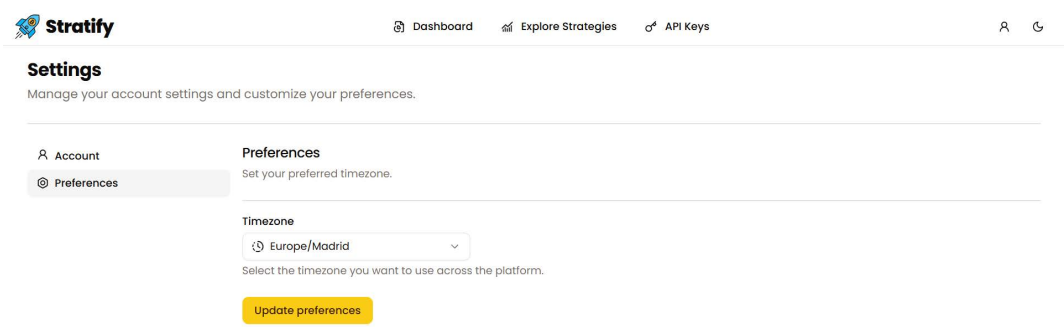
Historia de usuario	HU-007: Configuración de usuario
Actores	ACT-002
Característica / Funcionalidad	Gestionar opciones personales del usuario desde el apartado de configuración, incluyendo datos de cuenta y preferencias generales como la zona horaria.
Escenarios	1. Como usuario autenticado, quiero modificar mi nombre de usuario, correo electrónico, contraseña o eliminar mi cuenta, con verificación por correo electrónico para cada acción. 2. Como usuario autenticado, quiero establecer mi zona horaria desde el apartado de preferencias para que la plataforma muestre las fechas y horas ajustadas a mi región.
Resultados esperados	1.  Figura 45: Configuración de datos de usuario 2.  Figura 46: Selección de zona horaria

Tabla 39: HU-007, Configuración de usuario



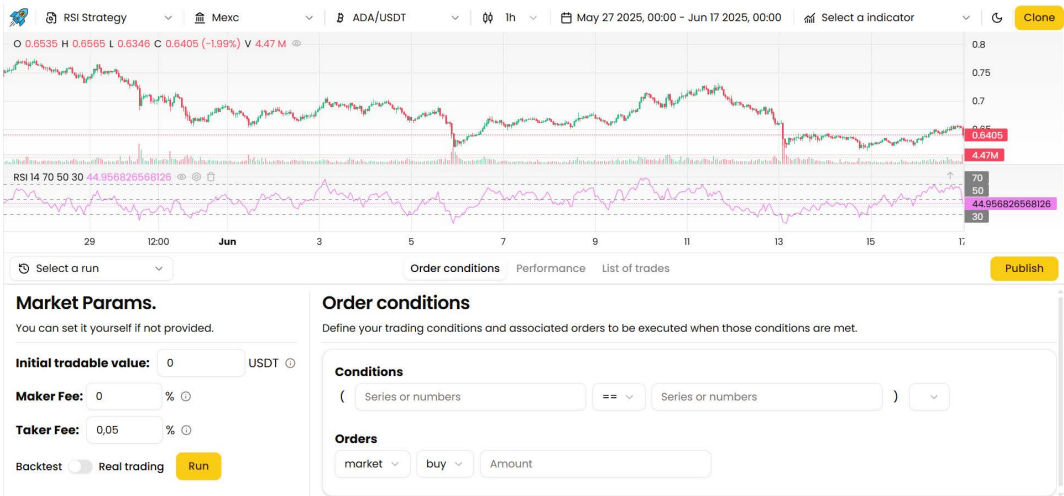
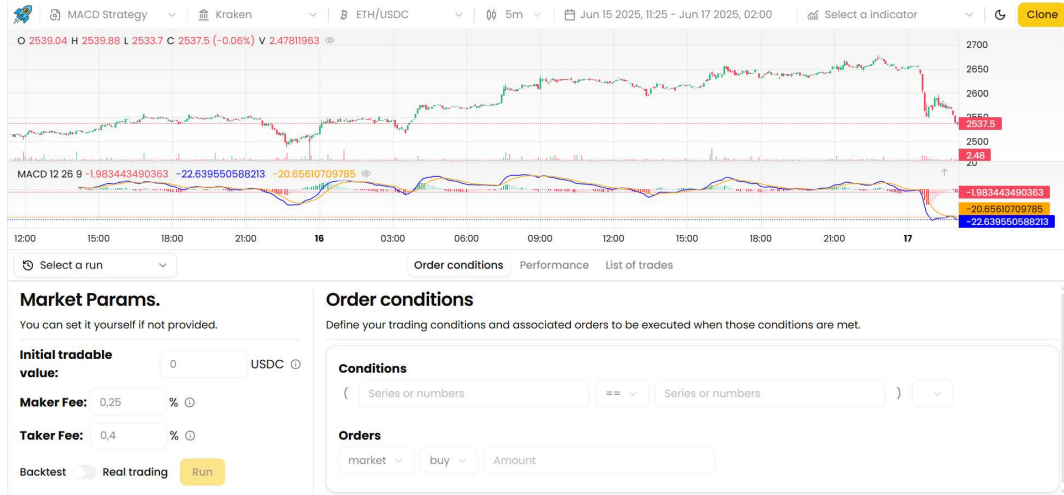
Historia de usuario	HU-008: Visualización de estrategia
Actores	ACT-002
Característica / Funcionalidad	Acceder a la vista detallada de una estrategia, incluyendo sus parámetros operativos, gráfico de velas y condiciones lógicas, con distintos niveles de interacción según la autoría.
Escenarios	1. Como usuario autenticado, quiero acceder a una de mis estrategias para poder editar todos sus parámetros, gestionar condiciones y ejecutar acciones sobre ella. 2. Como usuario autenticado, quiero visualizar una estrategia creada por otro usuario, pero sin poder modificarla ni ejecutar ninguna acción.
Resultados esperados	1.  2. 

Figura 47: Vista editable de estrategia propia

Figura 48: Vista restringida de estrategia ajena

Tabla 40: HU-008, Visualización de estrategia

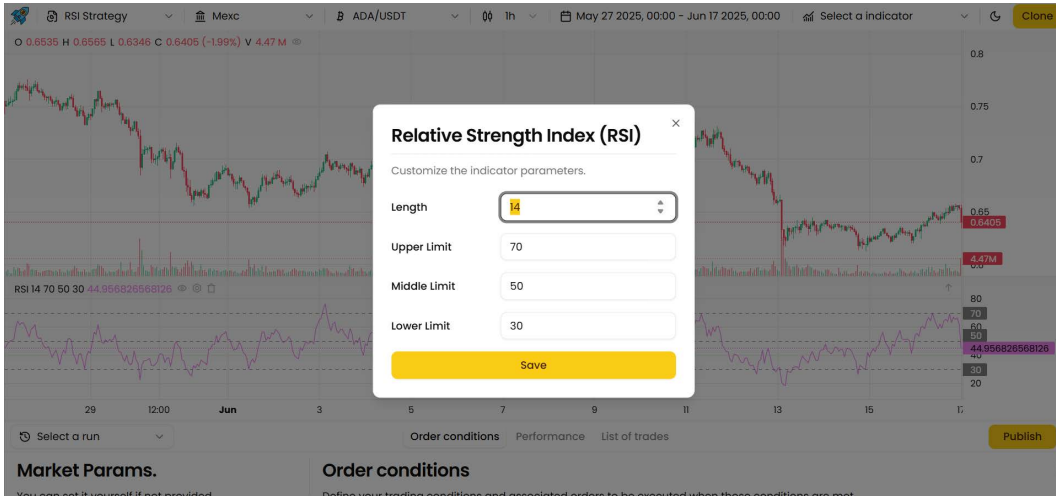
Historia de usuario	HU-009: Configuración general e indicadores
Actores	ACT-002
Característica / Funcionalidad	Modificar parámetros clave de una estrategia seleccionada (como exchange, par, timeframe, rango temporal e indicadores), así como editar individualmente la configuración de cada indicador visualizado.
Escenarios	1. Como usuario autenticado, quiero seleccionar una estrategia, cambiar su nombre o eliminarla, y ajustar parámetros generales como el exchange, par de monedas, duración de las velas, fechas de análisis e indicadores. 2. Como usuario autenticado, quiero modificar los parámetros de un indicador específico ya añadido al gráfico para ajustar su comportamiento, y ver los cambios reflejados visualmente al guardar.
Resultados esperados	1.  Figura 49: Parámetros generales de estrategia y vista de gráfico de velas japonesas 2.  Figura 50: Modificación de parámetros de un indicador

Tabla 41: HU-009, Configuración general e indicadores



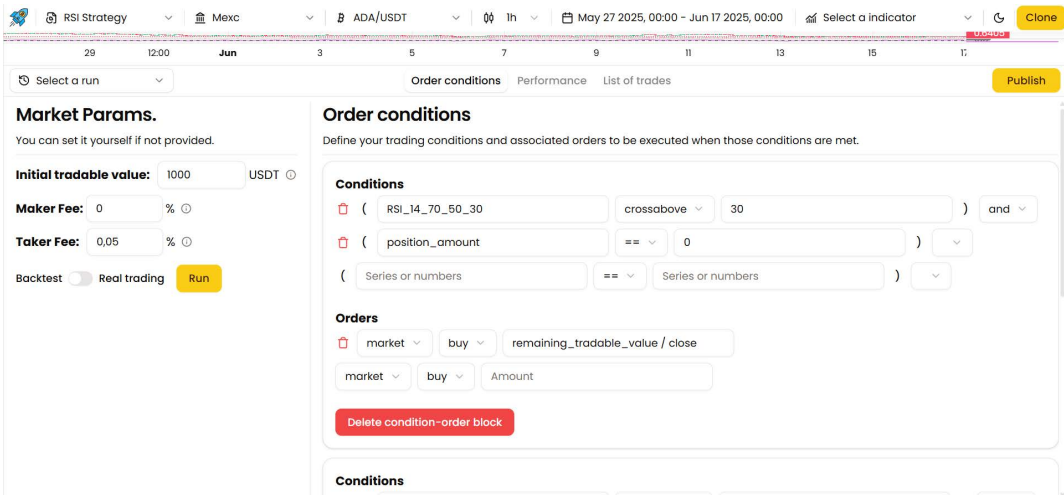
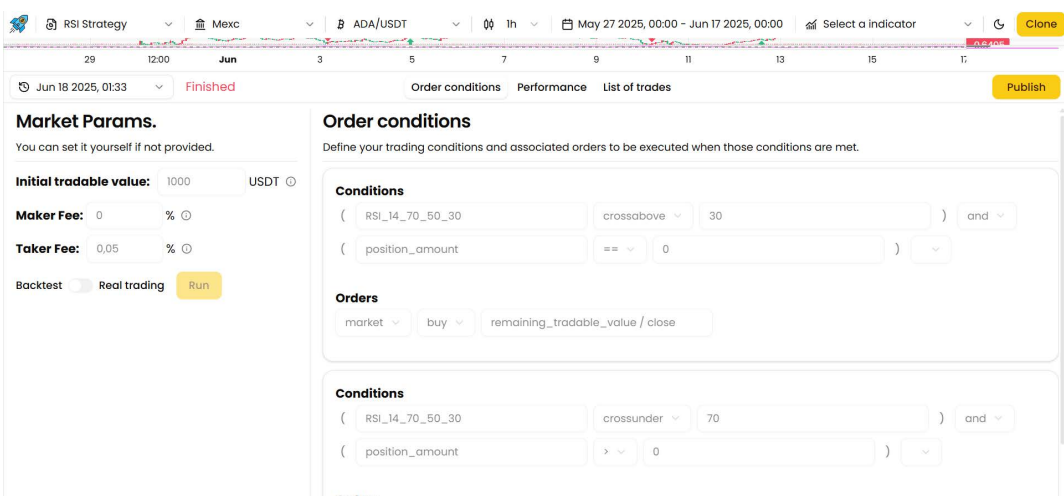
Historia de usuario	HU-010: Configuración y visualización de condiciones de ejecución
Actores	ACT-002
Característica / Funcionalidad	Configurar condiciones lógicas de ejecución y sus órdenes asociadas antes de lanzar una estrategia, y visualizar esos parámetros una vez la ejecución ya ha sido realizada.
Escenarios	1. Como usuario autenticado, quiero editar las condiciones lógicas y órdenes asociadas a una estrategia antes de ejecutarla, incluyendo la cantidad inicial, el modo (real o simulado) y las reglas condicionales. 2. Como usuario autenticado, quiero revisar los parámetros utilizados en una ejecución pasada, sin poder modificarlos, para analizar el comportamiento de la estrategia en ese contexto.
Resultados esperados	1.  Figura 51: Configuración de condiciones antes de ejecutar 2.  Figura 52: Visualización de condiciones tras ejecución

Tabla 42: HU-010, Configuración y visualización de condiciones de ejecución



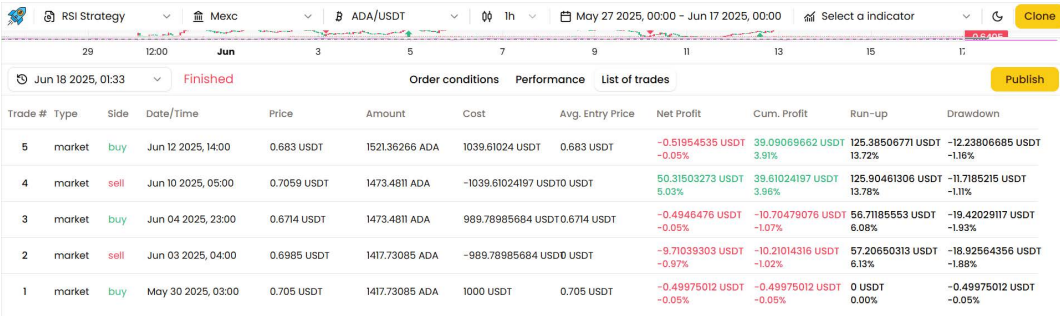
Historia de usuario	HU-011: Resultados y operaciones de una ejecución
Actores	ACT-002
Característica / Funcionalidad	Consultar el resultado de una ejecución de estrategia, accediendo tanto a métricas agregadas como a la lista detallada de operaciones realizadas durante el periodo analizado.
Escenarios	1. Como usuario autenticado, quiero visualizar las métricas de rendimiento de una ejecución de estrategia, incluyendo gráficos que representen visualmente los indicadores clave como beneficio acumulado o series de pérdidas. 2. Como usuario autenticado, quiero revisar una lista ordenada de todas las operaciones realizadas durante la ejecución, con datos como precio, cantidad, ganancia y momento de apertura/cierre.
Resultados esperados	1. Figura 53: Métricas de rendimiento de una ejecución 2. Figura 54: Lista de operaciones ejecutadas

Tabla 43: HU-011, Resultados y operaciones de una ejecución



7. Pruebas

Durante el desarrollo de la plataforma, se ha dado una atención especial a la fase de pruebas, entendida como un proceso esencial para garantizar la estabilidad y fiabilidad del sistema. Dado que la arquitectura se basa en la interacción entre el cliente y una API REST en el backend que gestiona operaciones críticas, cualquier fallo en los puntos de conexión puede derivar en errores graves, pérdida de datos o incluso decisiones financieras incorrectas.

Por ello, se ha realizado una validación rigurosa de los endpoints expuestos por el backend, comprobando que cada ruta responde correctamente ante distintos escenarios. Se ha prestado especial atención al manejo de errores, la integridad de los datos retornados y el cumplimiento de las reglas definidas. Un fallo en este nivel, como una excepción no capturada que devuelva un error HTTP 500, puede interrumpir completamente la operativa de la plataforma. En consecuencia, se ha priorizado la cobertura funcional de los servicios mediante pruebas diseñadas para evaluar su comportamiento individual y su integración conjunta.

Además de los aspectos técnicos, también se han llevado a cabo pruebas centradas en el flujo de uso previsto por el usuario. En este caso, el objetivo no ha sido solamente validar el correcto funcionamiento de las funcionalidades, sino también asegurar que la interacción entre módulos resulte coherente y predecible. Este enfoque permite detectar errores lógicos que no necesariamente se manifiestan como fallos técnicos, pero que afectan directamente a la experiencia de uso y, por tanto, a la utilidad práctica de la aplicación.

7.1. Introducción a Postman

Para la realización de estas pruebas se ha empleado Postman, una herramienta especializada en la validación de APIs REST que goza de gran aceptación en entornos profesionales por su versatilidad, capacidad de automatización y facilidad de uso. Postman permite simular solicitudes HTTP completas, definir entornos de prueba, reutilizar variables y organizar los distintos casos en colecciones temáticas, lo que facilita una revisión sistemática de la lógica del backend.

Entre sus principales ventajas se encuentra la posibilidad de realizar pruebas tanto unitarias como de integración. Las primeras validan la respuesta individual de cada endpoint ante una entrada concreta, mientras que las segundas permiten verificar la interacción y coherencia general del sistema mediante múltiples llamadas consecutivas. Estas pruebas han sido fundamentales para confirmar que las rutas del backend responden correctamente y respetan las restricciones de seguridad definidas.

Durante el proceso de validación, se han ejecutado peticiones que simulan escenarios reales, como la creación de estrategias, la configuración de indicadores, la recuperación de datos históricos o el inicio de ejecuciones automáticas. Para cada uno de estos casos, se ha comprobado que el backend responde correctamente en cuanto al código HTTP, formato de los datos retornados y tratamiento de errores. Además, se ha validado el comportamiento ante casos incorrectos, como entradas inválidas o ausencia de credenciales.

Postman también ha resultado útil para gestionar variables como claves API, tokens de acceso o identificadores dinámicos, facilitando así la repetibilidad de las pruebas y asegurando una detección más eficiente de errores en fases tempranas del desarrollo. Esta capacidad ha contribuido significativamente a mejorar la calidad y estabilidad del software desde sus primeras etapas.

Gracias a este conjunto de pruebas sistemáticas, ha sido posible asegurar la solidez del backend antes de cada despliegue, minimizando el riesgo de errores en producción y mejorando la calidad global del sistema.

7.2. Creación de pruebas en Postman

Además de permitir el envío manual de peticiones HTTP, Postman ofrece un entorno orientado a la automatización de pruebas que resulta especialmente útil para comprobar el comportamiento esperado de los distintos endpoints de una API. Esta funcionalidad permite verificar no solo la validez del código de respuesta recibido, sino también el contenido específico del cuerpo de la respuesta, garantizando que la lógica de negocio se ejecuta correctamente bajo diferentes escenarios.

Cada solicitud en Postman puede complementarse con comprobaciones que se ejecutan automáticamente al recibir la respuesta del servidor. Estas validaciones se configuran en una sección específica donde se redactan scripts en JavaScript para verificar aspectos clave del resultado. Esta capacidad convierte a Postman en una herramienta útil no solo para desarrolladores, sino también para equipos de calidad que garantizan que los servicios web cumplen con sus especificaciones.

En el caso concreto del presente proyecto, se ha hecho uso del sistema de pruebas integrado de Postman para validar endpoints relacionados con la gestión de estrategias de trading. A modo de ejemplo, se diseñó una prueba para verificar la correcta recuperación del conjunto de estrategias asociadas a un usuario autenticado. En este contexto, se espera que la ruta responda con un código HTTP 200 y que el cuerpo de la respuesta contenga una lista no vacía de objetos, cada uno representando una estrategia previamente registrada por el usuario.

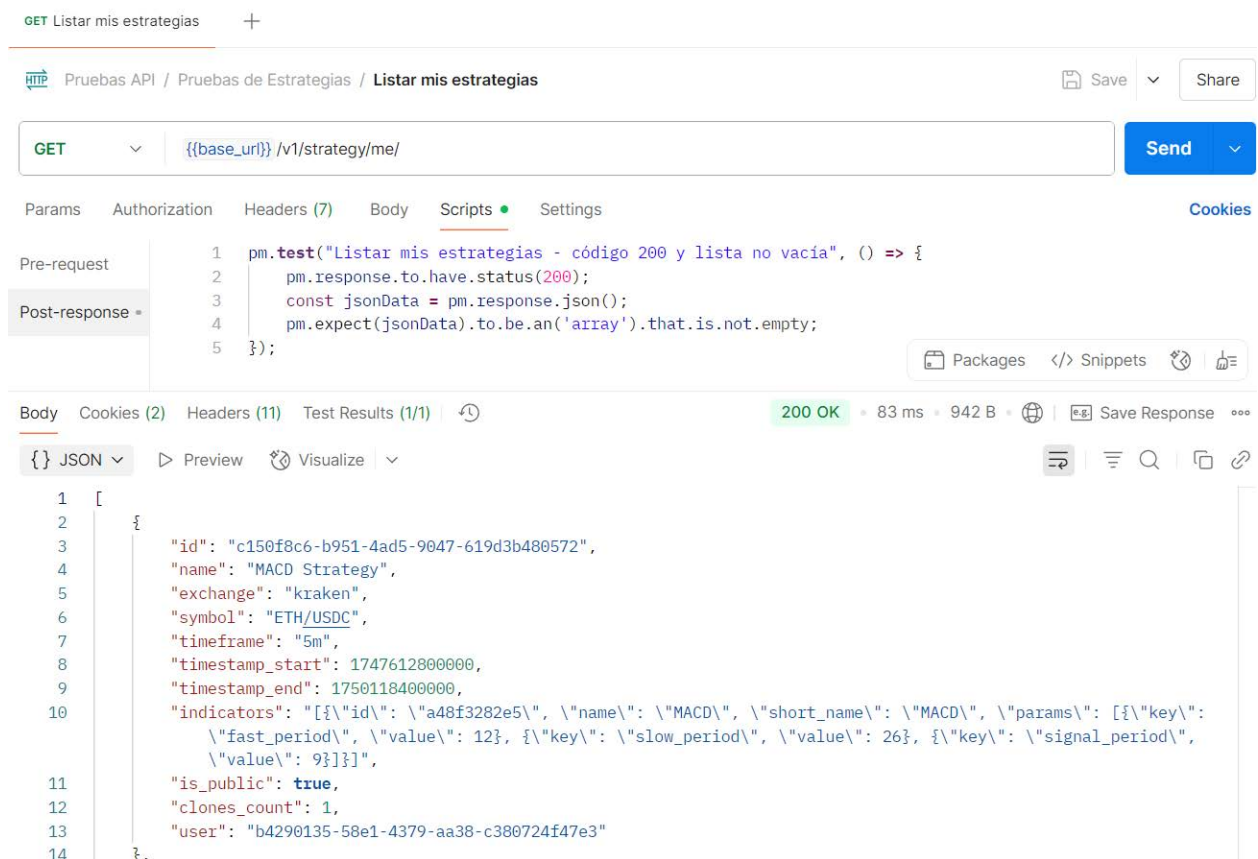


Figura 55: Ejemplo de prueba para el endpoint de consulta de estrategias personales

En esta prueba se evalúan dos aspectos esenciales: por un lado, que la respuesta sea satisfactoria desde el punto de vista del protocolo HTTP, y por otro, que el contenido devuelto sea coherente con la funcionalidad esperada. Este tipo de validaciones es particularmente útil para garantizar que los servicios continúan funcionando correctamente después de introducir cambios en el backend, permitiendo detectar de forma temprana posibles regresiones o errores lógicos.

Postman permite organizar estas pruebas en colecciones estructuradas y asociarlas a diferentes entornos de ejecución, como pueden ser los entornos de desarrollo, preproducción o producción. También es posible reutilizar variables definidas globalmente, como la base de las URLs, tokens de autenticación o identificadores dinámicos obtenidos en peticiones anteriores, lo cual facilita la automatización de flujos más complejos y mejora la eficiencia en la validación continua del sistema.

Gracias a la incorporación de este tipo de pruebas automatizadas, ha sido posible mantener un control preciso sobre la evolución del sistema, garantizando que cada modificación o ampliación en la lógica del backend conserva la compatibilidad con las funcionalidades existentes. Esta metodología ha resultado clave para reforzar la fiabilidad del sistema y asegurar una experiencia de usuario coherente en cada versión desplegada.



7.3. Listado de pruebas en Postman

Pruebas de Autenticación de Usuario

Nº	Nombre de la prueba	Descripción breve	Endpoint asociado	Resultado esperado
1	Login incorrecto	Devuelve error con credenciales inválidas	/v1/login/ (POST)	Código HTTP 400
2	Login correcto	Devuelve cookies y status 200	/v1/login/ (POST)	Código HTTP 200
3	Logout correcto	Borra cookies y devuelve 200	/v1/logout/ (POST)	Código HTTP 200
4	Obtener datos usuario no autenticado	Devuelve 200 pero con cuerpo vacío si no está autenticado	/v1/user/me/ (GET)	Código 200 y cuerpo vacío
5	Obtener datos de usuario autenticado	Devuelve código 200 y contiene id, nombre de usuario, email si está autenticado	/v1/user/me/ (GET)	Código 200 + propiedades presentes si hay datos

Tabla 44: Pruebas de Autenticación de Usuario

Pruebas de Preferencias del Usuario

Nº	Nombre de la prueba	Descripción breve	Endpoint asociado	Resultado esperado
1	Alternar tema	Cambia entre tema claro/oscuro	/v1/toggle-theme/ (PATCH)	Campo de tema cambiado
2	Zona horaria no enviada	Devuelve error si no se pasa la zona horaria	/v1/update-time-zone/ (PATCH)	Código HTTP 400
3	Zona horaria válida	Actualiza correctamente la zona horaria	/v1/update-time-zone/ (PATCH)	Campo de zona horaria actualizado

Tabla 45: Pruebas de Preferencias del Usuario

Pruebas de API Keys

Nº	Nombre de la prueba	Descripción breve	Endpoint asociado	Resultado esperado
1	Crear clave nueva	Crea una clave nueva	/v1/apiKey/ (POST)	Código HTTP 201
2	Listar claves API	Devuelve exchanges con claves activas	/v1/apiKey/ (GET)	Lista de exchanges no vacía
3	Reemplazar clave	Actualiza clave si ya existe para el exchange	/v1/apiKey/ (POST)	Código HTTP 200
4	Eliminar clave existente	Elimina correctamente la clave	/v1/apiKey/{exchange}/ (DELETE)	Código HTTP 204
5	Eliminar clave inexistente	Devuelve error si no existe	/v1/apiKey/{exchange}/ (DELETE)	Código HTTP 400

Tabla 46: Pruebas de API Keys



Pruebas de Datos de Mercado

Nº	Nombre de la prueba	Descripción breve	Endpoint asociado	Resultado esperado
1	Listar exchanges CCXT	Devuelve lista de exchanges filtrados	/v1/exchanges/ (GET)	Lista no vacía
2	Listar pares por exchange	Devuelve pares y timeframes	/v1/symbols/?exchange={exchange} (GET)	Campos par, timeframes
3	Info de mercado completa	Devuelve comisiones y más datos del mercado	/v1/market-info/?exchange={exchange}&symbol={symbol} (GET)	Campos taker_fee, maker_fee, etc.
4	Info de mercado faltan parámetros	Falla si no se pasa exchange o par	/v1/market-info/ (GET)	Código HTTP 404

Tabla 47: Pruebas de Datos de Mercado

Pruebas de Estrategias

Nº	Nombre de la prueba	Descripción breve	Endpoint asociado	Resultado esperado
1	Crear nueva estrategia	Devuelve una nueva estrategia con nombre único	/v1/strategy/me/ (POST)	Código HTTP 201
2	Editar estrategia propia	Permite modificarla si es del usuario	/v1/strategy/{id}/ (PUT)	Código HTTP 200
3	Listar mis estrategias	Devuelve solo las del usuario	/v1/strategy/me/ (GET)	Lista no vacía
4	Listar estrategias públicas	Devuelve públicas y propias	/v1/strategy/ (GET)	Lista no vacía
5	Clonar estrategia pública	Clona una estrategia pública	/v1/strategy/{id}/clone/ (POST)	Código HTTP 201
6	Eliminar estrategia propia	Permite borrarla si es del usuario	/v1/strategy/{id}/ (DELETE)	Código HTTP 204

Tabla 48: Pruebas de Estrategias



Pruebas de Ejecución de Estrategias

Nº	Nombre de la prueba	Descripción breve	Endpoint asociado	Resultado esperado
1	Iniciar ejecución	Lanza una ejecución de estrategia	/v1/strategy-execution/start/ (POST)	Código HTTP 201
2	Detener ejecución	Finaliza una ejecución activa	/v1/strategy-execution/{id}/stop/ (PATCH)	Código HTTP 200
3	Ver detalles de ejecución	Incluye métricas y lista de operaciones	/v1/strategy-execution/{id}/ (GET)	JSON con operaciones, beneficio, etc.
4	Listar ejecuciones por estrategia	Devuelve ejecuciones para una estrategia	/v1/strategy-execution/?strategy_id={id} (GET)	Lista no vacía de ejecuciones

Tabla 49: Pruebas de Ejecución de Estrategias

Durante la fase de pruebas, se ha optado por aplicar un mayor grado de verificación a aquellas áreas del sistema donde la lógica es más dinámica y propensa a variaciones, como ocurre en el tratamiento de estrategias y sus condiciones de ejecución. Este enfoque ha permitido detectar posibles inconsistencias en escenarios complejos, donde intervienen múltiples parámetros y reglas combinadas.

En cambio, otras partes del backend con menor nivel de complejidad estructural, como la autenticación o la recuperación de datos básicos, han requerido un número más reducido de comprobaciones, centradas principalmente en validar la respuesta esperada ante entradas válidas e inválidas.

Por último, en la Figura 60 se muestra una colección de pruebas ejecutadas en Postman, donde se observa una validación completa sin errores, reflejo del correcto comportamiento de los endpoints evaluados.

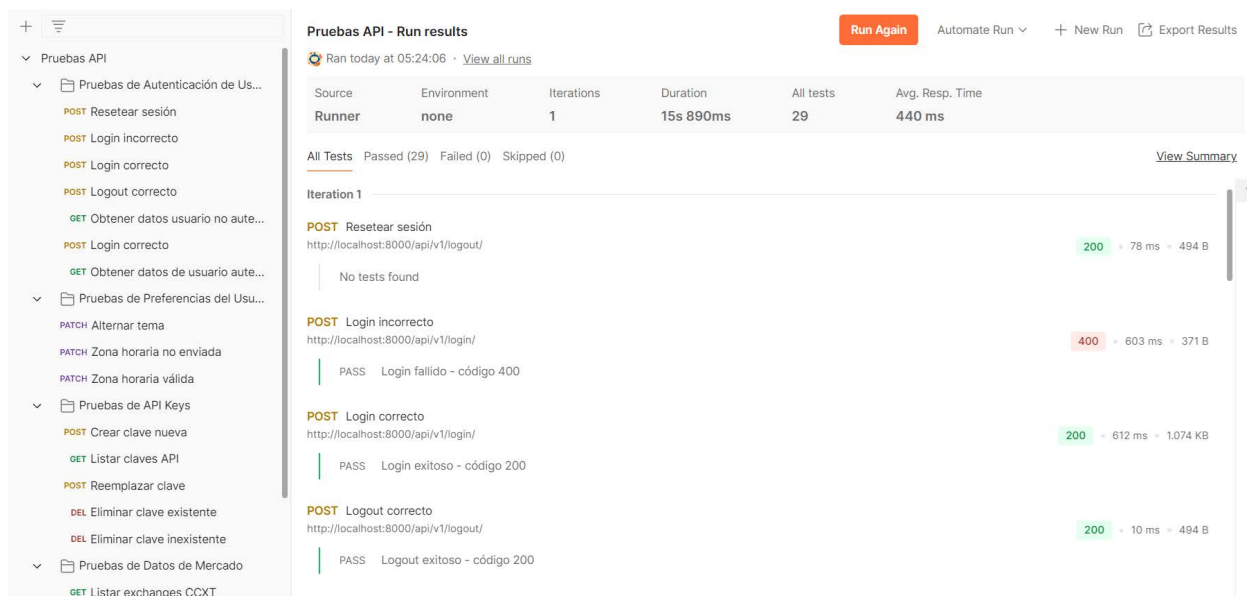


Figura 56: Resultado de ejecución de pruebas en Postman



7.4. Pruebas de usabilidad

Como complemento a las pruebas técnicas y de integración realizadas durante el desarrollo, se llevó a cabo una fase de pruebas de usabilidad con el objetivo de evaluar la experiencia de usuario en situaciones reales de uso. Esta etapa resulta especialmente importante en el contexto de este proyecto, donde uno de los objetivos clave es facilitar el acceso al trading algorítmico a perfiles no necesariamente técnicos.

Se diseñó un conjunto de tareas representativas de los flujos principales en la plataforma, evaluando factores como la claridad de la interfaz, la curva de aprendizaje, la capacidad de personalización y la comprensión de los resultados operativos.

Nº	Tarea a realizar	Objetivo evaluado	Métrica de éxito esperada
1	Crear una nueva estrategia desde cero	Validar facilidad de uso del formulario y comprensión de campos configurables	Estrategia creada sin ayuda externa
2	Editar y guardar cambios en una estrategia existente	Evaluar la localización de elementos y edición de datos	Edición completada en menos de 5 minutos
3	Ejecutar una estrategia y consultar resultados de rendimiento	Comprobar accesibilidad y claridad de la sección de resultados y métricas	Usuario interpreta correctamente los resultados
4	Clonar una estrategia pública	Validar comprensión de la funcionalidad y correcto flujo para clonado y modificación	Estrategia clonada y adaptada sin incidencias
5	Consultar el historial de ejecuciones	Evaluar acceso a datos históricos y comprensión del panel de actividad pasada	Historial accedido y comprendido correctamente
6	Personalizar preferencias de usuario	Medir la capacidad de adaptar la interfaz a nivel visual y horario	Preferencias aplicadas y reflejadas correctamente
7	Añadir un nuevo indicador técnico al gráfico	Comprobar la interacción con el gráfico de velas y configuración visual	Indicador añadido correctamente y parámetros visibles
8	Navegar entre diferentes pares de criptomonedas	Evaluar la navegación y filtrado en la interfaz	Cambio realizado sin pérdida de contexto

Tabla 50: Pruebas de usabilidad realizadas

Los participantes fueron observados de forma controlada mientras realizaban estas tareas. La muestra incluyó usuarios con diferentes niveles de experiencia en tecnología y finanzas, reflejando el público objetivo de la plataforma.



Usuario	Perfil técnico	Experiencia en trading	Observaciones durante las pruebas	Valoración global (1–5)
Usuario A	Usuario no técnico	Baja	Mostró una notable dificultad para comprender la interfaz y los conceptos básicos. Solo completó 3 de las 8 tareas sin ayuda. Le resultó confusa la terminología usada.	2
Usuario B	Inversor con nociones básicas	Alta	Ejecutó correctamente las estrategias y comprendió los resultados; tuvo dudas al configurar parámetros avanzados.	5
Usuario C	Estudiante de ingeniería informática	Media	Completó todas las tareas sin problemas. Destacó la eficiencia del sistema y propuso mejoras menores en la interfaz.	4
Usuario D	Estudiante de economía	Baja	Comprendió bien el análisis de resultados, aunque necesitó orientación para navegar entre pares de criptomonedas.	3

Tabla 51: Participantes de las pruebas de usabilidad

Conclusiones de las pruebas de usabilidad

La plataforma ha demostrado una buena capacidad de adaptación a distintos perfiles de usuario, especialmente en lo que respecta a perfiles técnicos o con cierta experiencia en inversiones digitales. Las tareas relacionadas con la ejecución de estrategias, la personalización básica y la visualización de resultados fueron completadas satisfactoriamente por la mayoría de los participantes.

No obstante, los resultados evidencian que la curva de aprendizaje para perfiles no técnicos es elevada, especialmente en tareas que implican la configuración de estrategias o el uso de terminología específica como “clonar”, “par de criptomonedas” o “indicador técnico”. Estos usuarios mostraron una mayor dependencia de asistencia externa o exploración guiada para avanzar.

A pesar de ello, se ha detectado un potencial claro de aprendizaje progresivo mediante el uso del modo simulación, que permite experimentar sin riesgo en un entorno controlado. Esta funcionalidad representa una vía eficaz para que usuarios con menos experiencia se familiaricen con los conceptos clave y el funcionamiento de la plataforma, reforzando así la accesibilidad a largo plazo.

Entre las mejoras identificadas durante el proceso de pruebas se destacan:

- Incluir ayudas contextuales (tooltips) al configurar estrategias o indicadores.
- Mejorar la segmentación visual entre estrategias propias y públicas.
- Añadir un modo de introducción guiada o “tour” para nuevos usuarios.

Estas observaciones serán especialmente valiosas en futuras iteraciones del proyecto, ya que permitirán reforzar la usabilidad y ampliar el alcance de la plataforma a un espectro más amplio de usuarios.



8. Proceso de despliegue

El despliegue del sistema se ha abordado con una arquitectura modular y reproducible, que facilita tanto la ejecución local como la transición a entornos de producción. Para lograr este objetivo, se ha optado por un enfoque basado en contenedores, que permite encapsular los distintos componentes del sistema, como el servidor web y la base de datos, en entornos aislados y controlados.

La coordinación de todos estos servicios se ha realizado mediante Docker Compose, herramienta que posibilita la definición conjunta de múltiples contenedores, sus redes internas y sus volúmenes persistentes. Esta estrategia favorece la escalabilidad, el mantenimiento y la portabilidad del sistema, a la vez que simplifica su despliegue en diferentes infraestructuras.

Como parte esencial de esta infraestructura, se ha incorporado un servidor Nginx como proxy inverso y punto de entrada externo a la plataforma. Además, se ha considerado desde el diseño inicial la compatibilidad con entornos en la nube, con vistas a su integración en servicios como Microsoft Azure, cuya descripción se aborda en este apartado.

8.1. Despliegue con Docker y Nginx

En el presente proyecto se ha empleado Docker como tecnología principal para la contenerización de los distintos componentes del sistema. Docker permite empaquetar aplicaciones junto con sus dependencias en contenedores ligeros, portables y aislados, lo que garantiza que el entorno de ejecución sea idéntico en desarrollo, pruebas y producción. Esta característica resulta especialmente útil en arquitecturas distribuidas, donde conviven múltiples servicios con requisitos específicos.

La coordinación entre estos contenedores se ha gestionado mediante Docker Compose, herramienta que permite definir la infraestructura completa mediante un único archivo de configuración. En él se especifican tanto los servicios implicados como sus relaciones, volúmenes persistentes, variables de entorno y redes internas, asegurando un control claro y ordenado del sistema.

Entre los contenedores desplegados, se encuentra uno dedicado a la base de datos, que ejecuta PostgreSQL. Este servicio se encarga del almacenamiento estructurado y seguro de toda la información gestionada por la plataforma. Su configuración contempla el uso de cifrado SSL en producción, con certificados generados mediante Let's Encrypt, y la persistencia de datos mediante volúmenes locales.

El servicio de backend, también ejecutado en un contenedor, aloja la lógica de servidor construida con Django y Django REST Framework. Durante el arranque, este componente se encarga de aplicar las migraciones de base de datos y preparar el entorno de ejecución, ajustando su comportamiento según se trate de un entorno de desarrollo o producción. En este último caso, se utiliza un servidor WSGI de alto rendimiento para gestionar las peticiones concurrentes.

El frontend, por su parte, se ejecuta en otro contenedor independiente que gestiona la construcción de la interfaz web mediante React y Vite. En producción, el resultado del proceso de compilación se comparte con el servidor web a través de un volumen común, lo que permite servir los archivos estáticos de forma optimizada y eficiente.

Como punto de entrada a toda la aplicación se ha empleado Nginx, un servidor web ampliamente utilizado por su eficiencia, su bajo consumo de recursos y su capacidad para actuar como proxy inverso. Nginx recibe todas las peticiones externas y las dirige al servicio correspondiente: ya sea entregando archivos estáticos del frontend o reenviando solicitudes a la API del backend. Además, gestiona el cifrado mediante HTTPS, implementa redirecciones automáticas desde HTTP, y aplica diversas cabeceras de seguridad para proteger la comunicación entre cliente y servidor frente a ataques comunes.

8.2. Configuración de Microsoft Azure

Microsoft Azure es una plataforma de servicios en la nube que proporciona una infraestructura robusta y escalable para el despliegue de aplicaciones web, combinando soluciones de computación, almacenamiento, redes y monitorización en un entorno centralizado. Esta versatilidad ha permitido implementar el sistema de forma eficiente y con un control total sobre los recursos utilizados.

En este caso, se ha configurado una máquina virtual con sistema operativo Ubuntu Server, utilizando la SKU Standard B2s, que ofrece dos vCPUs, 4 GiB de memoria RAM y un disco duro de 64 GiB con almacenamiento estándar HDD LRS. Esta configuración ha resultado suficiente para alojar la aplicación web, manejar la lógica del servidor y garantizar tiempos de respuesta adecuados en un contexto de uso no intensivo. La elección de estos recursos responde al equilibrio buscado entre rendimiento y coste, considerando las necesidades reales del entorno de pruebas y demostración.

Un elemento clave que ha favorecido el uso de esta plataforma ha sido la disponibilidad de una suscripción gratuita con un crédito anual de 100€, proporcionada por la Universidad de Almería a su alumnado. Esta ayuda ha posibilitado desplegar la solución en un entorno profesional, sin limitaciones presupuestarias, permitiendo experimentar con servicios reales de infraestructura en la nube, lo que enriquece notablemente el valor formativo del trabajo.

Durante el proceso de despliegue, se ha realizado la instalación de Docker para contenerizar los servicios y Nginx como proxy inverso, junto con la configuración de reglas de red que habilitan el acceso seguro desde el exterior. Todo ello ha sido gestionado desde el portal de Azure, que ofrece una experiencia unificada para el control de recursos, facilitando tareas como la supervisión de uso, la apertura de puertos o el reinicio de la máquina virtual en caso necesario.

El resultado de esta configuración puede visualizarse en la figura siguiente, que muestra la página de inicio de la aplicación desplegada y accesible públicamente a través de la dirección:

<https://stratify.eastus.cloudapp.azure.com>

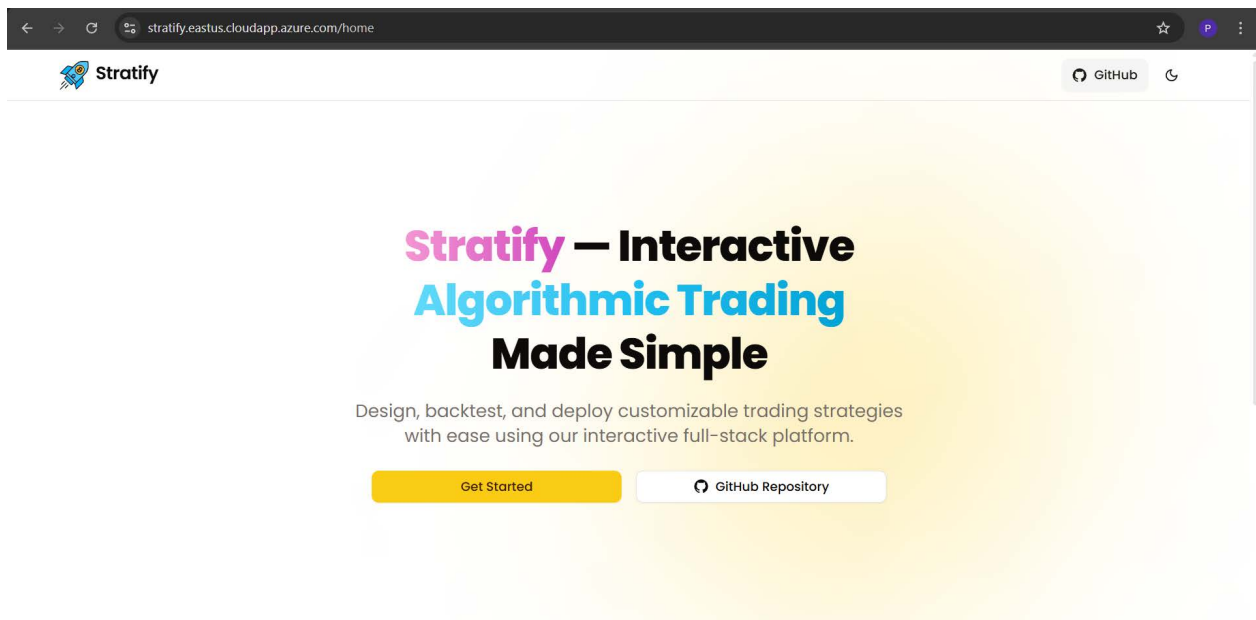


Figura 57: Página principal del sistema desplegado en Azure

8.3. Estimación de costes

El uso de servicios en la nube implica asumir una serie de costes derivados de la infraestructura utilizada y del mantenimiento requerido para garantizar la disponibilidad, seguridad y rendimiento de la plataforma. En este proyecto, se ha optado por Microsoft Azure como entorno de despliegue principal, dado que ofrece una solución profesional, con herramientas integradas para la gestión de recursos, y una interfaz amigable que permite un control detallado sobre las operaciones del sistema.

Gracias al programa de créditos educativos promovido por la Universidad de Almería, se ha dispuesto de un bono anual de 100 € para el uso de servicios en Azure, lo cual ha sido determinante para poder realizar pruebas y desplegar la aplicación sin incurrir en costes personales. Este crédito ha sido suficiente para cubrir todas las necesidades del proyecto durante su fase de desarrollo y validación.

La plataforma se encuentra desplegada en una máquina virtual Ubuntu Server, configurada con una instancia Standard B2s. Esta configuración incluye 2 vCPUs, 4 GiB de memoria RAM y 64 GiB de almacenamiento estándar en disco duro HDD (con redundancia local). Según los precios actuales ofrecidos por Azure en la región East US, esta instancia genera un coste estimado de entre 35 y 45 € mensuales, en función del uso horario y las posibles optimizaciones en la ejecución. Al tratarse de una máquina con una relación equilibrada entre coste y rendimiento, se considera adecuada para entornos de prueba o para un primer despliegue en producción con un número limitado de usuarios.

Durante el desarrollo, se ha utilizado Nginx como servidor proxy inverso, permitiendo canalizar las peticiones web y facilitar la configuración de HTTPS. Asimismo, se ha integrado Docker como herramienta de contenedorización, lo que facilita la organización de los servicios y su despliegue de forma reproducible. Ambas tecnologías son gratuitas y de código abierto, por lo que no generan costes directos, aunque requieren cierta dedicación para su configuración y mantenimiento.

En cuanto al acceso público de la aplicación, se ha utilizado la dirección IP dinámica que proporciona Azure por defecto. No se ha contratado ningún dominio personalizado, lo que evita costes adicionales asociados a su compra o renovación. En caso de optar por esta opción en el futuro, los dominios con extensiones comunes como .com o .org tienen un coste aproximado de entre 10 y 15 € anuales, que equivaldrían a unos 1–1,5 € mensuales prorrateados. La seguridad de la conexión está asegurada mediante certificados SSL gratuitos emitidos por Let's Encrypt, sin costes asociados.

Se presenta a continuación una estimación del coste mensual necesario para mantener la infraestructura actual en funcionamiento:

Concepto	Coste estimado mensual
Máquina virtual (Standard B2s en Azure)	35–45 €
Certificado SSL (Let's Encrypt)	0 €
Mantenimiento técnico (autogestionado)	0 €

Tabla 52: Estimación costes mensuales en producción

En total, el coste mensual estimado estaría entre 36 € y 46,5 €.

Esta estimación parte de un escenario en el que el mantenimiento es asumido de forma personal, como ha sido el caso a lo largo de este trabajo. En caso de escalar el servicio o delegar tareas técnicas a terceros, estos costes podrían incrementarse significativamente. Aun así, el modelo actual permite sostener el sistema de forma eficiente y con un gasto muy reducido, especialmente mientras se mantenga el acceso al programa de créditos académicos. Esta circunstancia convierte a Azure en una opción muy favorable para proyectos universitarios que requieran entornos reales de despliegue sin comprometer la viabilidad económica.



9. Conclusiones

9.1. Resultados

El desarrollo de este Trabajo de Fin de Grado ha representado una experiencia integral, tanto a nivel técnico como personal. La construcción de Stratify ha exigido la aplicación coordinada de múltiples competencias adquiridas durante el grado, incluyendo el diseño de interfaces, el desarrollo backend con Django, la integración de librerías externas para el análisis de mercado, y la arquitectura de sistemas web escalables. Este proyecto ha supuesto una oportunidad excepcional para consolidar conocimientos y afrontar un reto completo de Ingeniería del Software.

Uno de los principales logros ha sido la implementación de una plataforma funcional que permite a los usuarios diseñar, simular y ejecutar estrategias de trading algorítmico sobre criptoactivos, todo ello a través de una interfaz intuitiva y accesible. El uso de tecnologías modernas como React, Vite, Django REST Framework y PostgreSQL ha permitido garantizar un desarrollo coherente, modular y mantenible. Además, se ha conseguido integrar de forma efectiva la biblioteca CCXT para el acceso a datos de mercado en tiempo real y la operativa sobre exchanges reales, aportando un valor añadido clave para los objetivos del sistema.

En términos técnicos, destacan los avances en seguridad (autenticación JWT, verificación de correo, manejo cifrado de claves API), así como en eficiencia operativa, gracias a la estructura dockerizada y al despliegue en la nube mediante Microsoft Azure. También ha sido especialmente enriquecedor el diseño de mecanismos para la representación gráfica del mercado (gráfico de velas e indicadores), permitiendo al usuario una interacción visual directa con sus estrategias.

Desde un punto de vista formativo, este proyecto ha reforzado mi capacidad para abordar problemas reales, planificar iterativamente, organizar código de forma profesional y aplicar buenas prácticas en entornos de desarrollo Full Stack. La experiencia adquirida me motiva a seguir profundizando en el desarrollo de soluciones tecnológicas orientadas al ámbito financiero, con especial atención a la accesibilidad y escalabilidad.

Asimismo, el proceso ha puesto de manifiesto la importancia de adoptar una mentalidad crítica y orientada a la mejora continua, especialmente en proyectos de base tecnológica. Afrontar decisiones de diseño, resolver errores inesperados y mantener la cohesión entre frontend y backend ha sido tan formativo como implementar funcionalidades, permitiéndome desarrollar una visión más estratégica del ciclo completo de desarrollo de software.

9.2. Trabajos futuros

Si bien la plataforma ha alcanzado un grado significativo de desarrollo y funcionalidad, existe un amplio margen para futuras ampliaciones. Uno de los puntos más relevantes sería incorporar un sistema de backtesting más robusto, capaz de evaluar estrategias sobre grandes volúmenes de datos históricos con métricas más sofisticadas. Esto permitiría al usuario validar sus configuraciones de forma más precisa antes de operar en tiempo real.

Otra línea de mejora consistiría en implementar un sistema de alertas y notificaciones push, que informe al usuario de manera inmediata sobre eventos relevantes, como el inicio o la finalización de una ejecución, o incluso condiciones críticas del mercado. Esta funcionalidad incrementaría considerablemente la utilidad operativa de la plataforma y su capacidad de adaptabilidad a escenarios dinámicos y cambiantes.

La experiencia de usuario también puede seguir perfeccionándose mediante la inclusión de elementos gráficos más elaborados, animaciones y una mayor personalización de la interfaz. Asimismo, el diseño de un modo oscuro más personalizable y adaptable mejoraría la accesibilidad en entornos de uso prolongado.

Por otro lado, resulta prometedor plantear la extensión del sistema hacia un modelo colaborativo, permitiendo a los usuarios compartir y valorar estrategias dentro de una comunidad. Para ello, sería interesante diseñar un sistema de puntuaciones, comentarios y validaciones cruzadas, fomentando el aprendizaje compartido y la mejora continua.

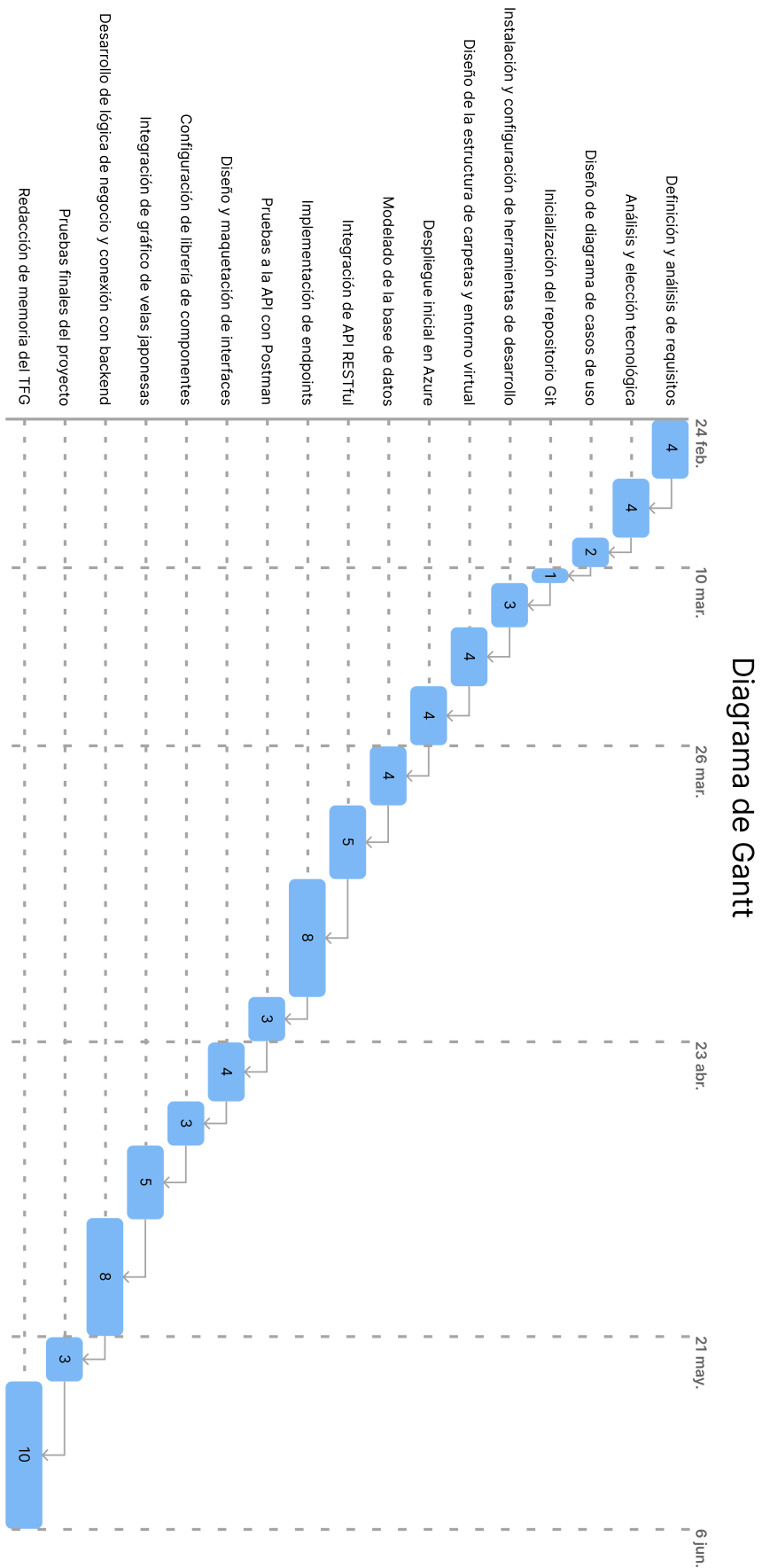


Finalmente, con vistas a su publicación como producto final, sería recomendable realizar una auditoría completa en aspectos de rendimiento, accesibilidad, y cumplimiento normativo (como la protección de datos). Esto incluiría mejoras específicas para su posible adaptación a dispositivos móviles mediante frameworks como React Native [35] o PWA.

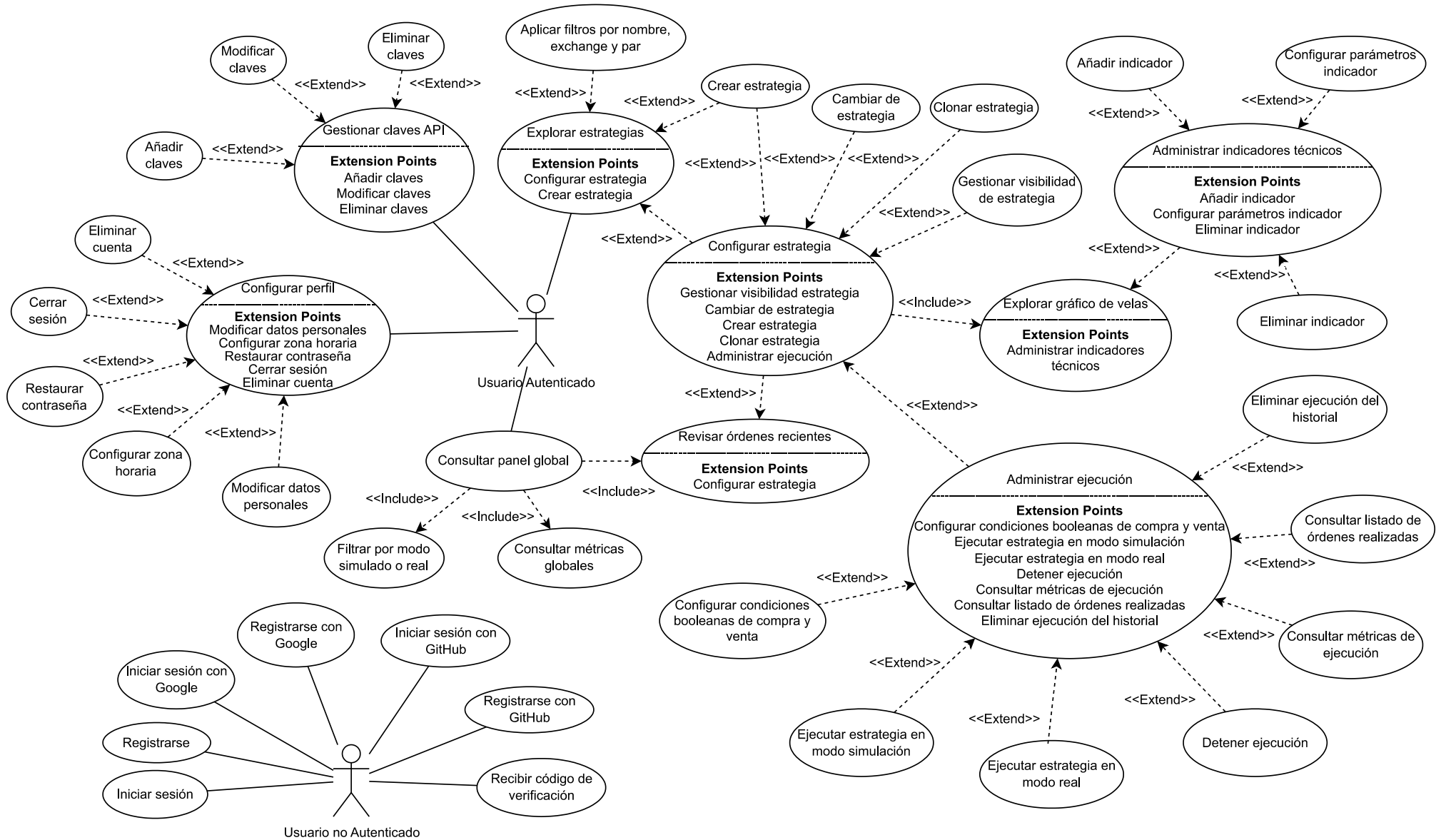
En resumen, Stratify ha sentado unas bases sólidas y escalables que permiten, con tiempo y recursos adicionales, evolucionar hacia una plataforma avanzada de trading algorítmico accesible, robusta y orientada a un público amplio.



Anexo 1



Anexo 2



Anexo 3





10. Bibliografía

- [1] Richardson, L., Amundsen, M., & Ruby, S. (2013). RESTful web APIs: services for a changing world. " O'Reilly Media, Inc."
- [2] Schwaber, K., & Sutherland, J. (2013). La guía de Scrum. Scrumguides. Org, 1, 21.
- [3] Anderson, D. J., & Carmichael, A. (2016). Essential kanban condensed. Blue Hole Press.
- [4] Astels, D., Miller, G., & Novak, M. (2002). The practical guide to extreme programming. Prentice Hall PTR.
- [5] Kuhlman, D. (2009). A python book: Beginning python, advanced python, and python exercises (pp. 1-227). Lutz: Dave Kuhlman.
- [6] Goldberg, J. (2022). Learning TypeScript. " O'Reilly Media, Inc."
- [7] Hillar, G. C. (2018). Django RESTful Web Services: The Easiest Way to Build Python RESTful APIs and Web Services with Django. Packt Publishing Ltd.
- [8] Vincent, W. S. (2022). Django for APIs: Build web APIs with Python and Django. WelcomeToCode.
- [9] Adeshina, A. A. (2022). Building Python Web APIs with FastAPI: A fast-paced guide to building high-performance, robust web APIs with very little boilerplate code. Packt Publishing Ltd.
- [10] Heckler, M. (2021). Spring Boot: Up and Running. O'Reilly Media.
- [11] Banks, A., & Porcello, E. (2020). Learning React: modern patterns for developing React apps. O'Reilly Media.
- [12] Ferrari, L., & Pirozzi, E. (2020). Learn PostgreSQL: Build and manage high-performance database solutions using PostgreSQL 12 and 13. Packt Publishing Ltd.
- [13] Suehring, S. (2002). MySQL bible. John Wiley & Sons, Inc..
- [14] Gehani, N., & Annamalai, M. (2011). The Database Book: Principles & Practice Using the Oracle Database. Silicon Press.
- [15] Carpenter, J., & Hewitt, E. (2022). Cassandra. " O'Reilly Media, Inc."
- [16] Andersson, J. C. (2023). Learning microsoft azure. " O'Reilly Media, Inc."
- [17] Wittig, A., & Wittig, M. (2023). Amazon Web Services in Action: An in-depth guide to AWS. Simon and Schuster.
- [18] Geewax, J. J. J. (2018). Google Cloud platform in action. Simon and Schuster.
- [19] DeJonghe, D. (2020). Nginx cookbook. O'Reilly Media.
- [20] Ristic, I. (2014). Bulletproof SSL and TLS: Understanding and deploying SSL/TLS and PKI to secure servers and web applications. Feisty Duck.
- [21] Poulton, N. (2023). Docker Deep Dive: Zero to Docker in a single book. NIGEL POULTON LTD.
- [22] Westerveld, D. (2021). API Testing and Development with Postman: A practical guide to creating, testing, and managing APIs for automated software testing. Packt Publishing Ltd.
- [23] Staiano, F. (2022). Designing and Prototyping Interfaces with Figma: Learn essential UX/UI design principles by creating interactive prototypes for mobile, tablet, and desktop. Packt Publishing Ltd.
- [24] Tam, F. K. (2015). The Power of Japanese Candlestick Charts: Advanced Filtering Techniques for Trading Stocks, Futures, and Forex. John Wiley & Sons.
- [25] Sami, H. M., Ahshan, K. A., Rozario, P. N., & Ashrafi, N. (2022). Evaluating the prediction accuracy of MACD and RSI for different stocks in terms of standard market suggestions. Canadian Journal of business and information studies, 7820(1), 137-143.



- [26] Gumparthi, S. (2017). Relative strength index for developing effective trading strategies in constructing optimal portfolio. *International Journal of Applied Engineering Research*, 12(19), 8926-8936.
- [27] Bollinger, J. (2002). *Bollinger on Bollinger bands* (p. 0). New York: McGraw-Hill.
- [28] Ding, Y. H., Liu, C. H., & Tang, Y. X. (2012). MVC pattern based on JAVA. *Applied Mechanics and Materials*, 198, 537-541.
- [29] Biehl, M. (2019). *OpenID Connect & JWT* (Vol. 6). API-University Press.
- [30] Grossman, J. (2007). *XSS attacks: cross site scripting exploits and defense*. Syngress.
- [31] Archer, G., Kahrer, J., & Trojanowski, M. (2025). *Cloud Native Data Security with OAuth: A Scalable Zero Trust Architecture*. " O'Reilly Media, Inc."
- [32] Rhoton, J. (1999). *Programmer's guide to internet mail: SMTP, POP, IMAP, and LDAP*. Digital Press.
- [33] Banks, A., & Porcello, E. (2020). *Learning React: modern patterns for developing React apps*. O'Reilly Media.
- [34] Bhat, K. (2023). *Ultimate Tailwind CSS Handbook: Build sleek and modern websites with immersive UIs using Tailwind CSS*. Orange Education Pvt Limited.
- [35] Sakhniuk, M., & Boduch, A. (2024). *React and React Native: Build cross-platform JavaScript and TypeScript apps for the web, desktop, and mobile*. Packt Publishing Ltd.

Stratify es una aplicación web que ha sido desarrollada como herramienta tecnológica para facilitar la gestión de estrategias de trading algorítmico en el ámbito de las criptomonedas. Su propósito es ofrecer un entorno accesible y automatizado que permita a los usuarios diseñar, probar y ejecutar algoritmos basados en indicadores técnicos. Con ello, se busca democratizar el uso de modelos de inversión avanzados y transformar la alta volatilidad de estos activos en una oportunidad, en lugar de un riesgo, mediante un enfoque sistemático.

Stratify is a web application developed as a technological tool to facilitate the management of algorithmic trading strategies in the field of cryptocurrencies. Its purpose is to provide an accessible and automated environment that allows users to design, test, and execute algorithms based on technical indicators. The aim is to democratize the use of advanced investment models and turn the high volatility of these assets into an opportunity rather than a risk, through a systematic and efficient approach.

